Lattice-Based Post-Quantum Cryptography

Complete Lecture Notes

Dr. Faisal Aslam

Lessons 1–6

Contents

Lesson 1: Mathematical Foundations for Lattice-Based Cryptography

Groups, Rings, Fields, Modular Arithmetic, Vector Spaces, Basis, and Lattices

Purpose of this lecture. Every technical term used later in the thesis proposal — "lattice," "modulus," "ring $\mathbb{Z}_q[x]/(x^n + 1)$" — rests on a small set of algebraic building blocks. This lecture builds those blocks, one at a time, from the ground up: *Group* → *Subgroup* → *Ring* → *Field* → *Modular Arithmetic* → *Vector Space* → *Basis* → *Lattice*. Nothing here requires prior abstract algebra; every definition is followed immediately by concrete, worked examples.

1.1 Why Start With Algebra?

Lattice-based cryptography is, at its core, about doing arithmetic on *vectors* whose coordinates live in a *finite, wraparound number system*, arranged according to strict algebraic rules. Before we can even state what a lattice is, we need precise answers to questions like:

- What does it mean for a set of numbers to have a well-behaved notion of "add" and "multiply"? (**Groups, Rings, Fields**)
- What happens when arithmetic "wraps around," like a clock? (**Modulus**)
- What does it mean to build every point in a space out of a small number of building-block directions? (**Vector spaces, Basis**)
- What happens when we only allow *whole-number* combinations of those building blocks, instead of any real-number combination? (**Lattice**)

This lecture answers each of these in order. By the end, the definition of a lattice (Section 1.8) should feel like the natural, inevitable next step, not an isolated new idea.

1.2 Sets and Binary Operations (Quick Refresher)

A **set** is just a collection of objects (numbers, vectors, symbols — anything). A **binary operation** on a set S is a rule that takes two elements of S and combines them into a third element, e.g., ordinary addition $+$ takes two numbers and produces their sum. We write $a * b$ for a general binary operation.

An operation is **closed** on S if combining two elements of S always gives another element of S (never something outside S). This sounds obvious but fails more often than you'd expect — see Example 1 below.

Example: Closure Can Fail

Let $S = \{1, 2, 3, 4, 5\}$ (just these five numbers) and let $*$ be ordinary addition. Then $2 * 4 = 6$, but $6 \notin S$. So ordinary addition is *not* closed on S. This is exactly the kind of problem modular arithmetic (Section 1.5) is built to fix.

1.3 Groups

Definition: Group

A set G together with a binary operation $*$ is called a **group**, written $(G, *)$, if all four of the following hold:
(G1) **Closure:** for all $a, b \in G$, $a * b \in G$.
(G2) **Associativity:** for all $a, b, c \in G$, $(a * b) * c = a * (b * c)$.
(G3) **Identity:** there exists an element $e \in G$ such that $a * e = e * a = a$ for all $a \in G$.
(G4) **Inverses:** for every $a \in G$, there exists $a^{-1} \in G$ such that $a * a^{-1} = a^{-1} * a = e$.
If, in addition, $a * b = b * a$ for all $a, b \in G$ (the order doesn't matter), the group is called **abelian** (commutative).

Example: Groups You Already Know

- $(\mathbb{Z}, +)$ — the integers under addition. Identity is 0; the inverse of a is $-a$. This is an abelian group.
- $(\mathbb{R} \setminus \{0\}, \times)$ — nonzero real numbers under multiplication. Identity is 1; the inverse of a is $1/a$. Also abelian. (We had to remove 0 because 0 has no multiplicative inverse.)
- $(\mathbb{Z}, \times)$ — integers under multiplication — is **not** a group: 2 has no inverse in $\mathbb{Z}$ (we'd need $1/2$, which isn't an integer). This fails axiom (G4).

Example: Groups Hiding in Less Obvious Places: Matrices and Polynomials

The three examples above were all sets of plain numbers. Groups show up in far less numeric-looking settings too — the axioms (G1)–(G4) don't care what the elements *are*, only that closure, associativity, identity, and inverses all hold. Four examples worth having in your back pocket, each a little "fancier" than the last:

- 3×3 **real matrices under addition,** $(M_{3\times 3}(\mathbb{R}), +)$**, is a group.** Adding two 3×3 matrices gives another 3×3 matrix (closure); matrix addition is associative; the identity is the all-zeros matrix $\mathbf{0}$; the inverse of A is $-A$ (negate every entry). It's even abelian, since $A + B = B + A$ entrywise. Nothing about this is different in spirit from $(\mathbb{Z}, +)$ — just 9 numbers moving in lockstep instead of 1.
- **The same set under multiplication is *not* a group.** Try $(M_{3\times 3}(\mathbb{R}), \times)$ (ordinary matrix multiplication): closure holds (multiplying two 3×3 matrices gives another 3×3 matrix) and multiplication is associative, and there's an identity, I_3. But axiom (G4) fails: a **singular** matrix (one with $\det = 0$, e.g. the all-zeros matrix, or $\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$) has no inverse matrix at all — there is no B with $AB = BA = I_3$. So the full set of 3×3 matrices under multiplication is not a group, for exactly the same structural reason $(\mathbb{Z}, \times)$ wasn't: some elements simply lack an inverse. (Also worth noting for later: even restricting to the invertible matrices, this group would *not* be abelian in general — $AB \neq BA$ for most matrix pairs — unlike every example so far.)
- **Invertible matrices under multiplication, on the other hand, *do* form a group.** Restrict to only the invertible 3×3 matrices, $GL_3(\mathbb{R}) = \{A \in M_{3\times 3}(\mathbb{R}) : \det(A) \neq 0\}$. Now every element has an inverse *by construction* (that's what "invertible" means), the product of two invertible matrices is again invertible (closure restored), and I_3 is still the identity. This is called the **general linear group**, and it's the standard fix for the multiplication problem above: throw away the elements that were breaking (G4), and check that what's left is still closed.
- **Unitary matrices under multiplication form a group too — a more specialized cousin of** GL_3**.** A (complex) $n \times n$ matrix U is **unitary** if $U^*U = I$, where U^* is the conjugate transpose (Lecture 1, Section 1.7.2's Hermitian inner product remark is exactly the conjugate this definition needs). The set of $n \times n$ unitary matrices, under multiplication, is a group: the product of two unitary matrices is unitary (check: $(U_1U_2)^*(U_1U_2) = U_2^*U_1^*U_1U_2 = U_2^*IU_2 = U_2^*U_2 = I$), I itself is unitary, and every unitary U has an inverse — namely $U^{-1} = U^*$, which is itself unitary. This group, $U(n)$, sits *inside* $GL_n(\mathbb{C})$ as a much smaller, highly-structured subgroup (Section 1.3.1's subgroup idea, one level up): every unitary matrix is invertible, but very few invertible matrices are unitary.
- **Degree-**$< n$ **polynomials under addition form a group; under multiplication they do not.** Let $P_n = \{a_0 + a_1x + \cdots + a_{n-1}x^{n-1} : a_i \in \mathbb{R}\}$ (real polynomials of degree strictly less than n). Under addition, $(P_n, +)$ is a group exactly like matrices under addition above: closed (adding two such polynomials keeps the degree below n), associative, identity is the zero polynomial, and the inverse of $p(x)$ is $-p(x)$. Under multiplication, it immediately fails on *two* separate grounds: closure fails (multiplying two degree-$(n-1)$ polynomials can produce degree up to $2n - 2$, which is no longer in P_n at all), and even ignoring that, most individual polynomials have no multiplicative inverse within polynomials (e.g., there is no polynomial $q(x)$ with $x \cdot q(x) = 1$ for every x). This is exactly the polynomial-ring preview from Section 1.4 below: $(P_n, +)$ being a group but $(P_n, \times)$ not being one is precisely why we need the weaker notion of a **ring** (two operations, only one of which needs full group structure) to describe polynomials properly, rather than expecting both operations to behave like a group.

Remark: Why Cryptography Cares About Groups

Many classical cryptographic schemes (Diffie–Hellman, ElGamal, elliptic-curve cryptography) are built directly on top of group structure — the security relies on certain problems (like "given g and g^a, find a") being hard inside a specific group. Lattice-based cryptography instead builds on richer structures (rings and modules, coming next), but the group is always the first layer underneath.

Exercise: Try This Before Next Lecture

Is $(\mathbb{Z}_5 \setminus \{0\}, \times \bmod 5) = (\{1,2,3,4\}, \times \bmod 5)$ a group? Check all four axioms directly by building the multiplication table. (You'll need Section 1.6 for what "$\bmod 5$" means if it's new to you — come back to this after reading that section.)

1.3.1 Subgroups

There is one more group-related idea we need before we can properly define a lattice: a group living *inside* another group.

Definition: Subgroup

Let $(G, *)$ be a group. A subset $H \subseteq G$ is a **subgroup** of G if H is itself a group under the same operation $*$. In practice, this is easiest to check with a simple test: H is a subgroup of G if and only if (1) H is nonempty (usually: it contains the identity e), (2) H is closed under $*$ (for $a, b \in H$, $a * b \in H$), and (3) H is closed under inverses (for $a \in H$, $a^{-1} \in H$). Associativity is automatic, since it already held for all of G.

Example: Subgroups You Can Picture

- For any integer n, the set $n\mathbb{Z} = \{\ldots, -2n, -n, 0, n, 2n, \ldots\}$ (all multiples of n) is a subgroup of $(\mathbb{Z}, +)$: it contains 0, adding two multiples of n gives another multiple of n, and $-(kn)$ is also a multiple of n. For example, $2\mathbb{Z}$ is "all even integers."
- The natural numbers $\mathbb{N} = \{0, 1, 2, \ldots\}$ are **not** a subgroup of $(\mathbb{Z}, +)$: $\mathbb{N}$ is closed under addition, but it fails closure under inverses — $1 \in \mathbb{N}$, but $-1 \notin \mathbb{N}$.

Looking Ahead: Why This Matters in Two Sections

Keep the example $n\mathbb{Z} \subseteq \mathbb{Z}$ in mind. In Section 1.8, we will define a lattice as a special kind of subgroup of $\mathbb{R}^n$ — and $n\mathbb{Z}$ is exactly a simple, one-dimensional lattice in disguise: an evenly-spaced, discrete set of points, closed under addition and negation, sitting inside the larger group $\mathbb{Z}$ (or $\mathbb{R}$).

1.3.2 Cosets and Quotient Groups: Where $\mathbb{Z}_n$ Really Comes From

Section 1.6 will introduce $\mathbb{Z}_n$ informally, as "remainders with wraparound arithmetic." That's the right intuition, but it's worth seeing the fully rigorous construction now, because it is a direct, satisfying payoff of the subgroup idea — and it is the same construction ("take a group, collapse it by a subgroup") that shows up again later when we build the ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ that Kyber and Dilithium actually use.

Remark: The One-Sentence Idea, Before Any Symbols

A coset is a "shifted copy" of a subgroup. You take every element of H and slide it over by adding a fixed amount a. That's all $a + H$ is: H, translated — nothing more exotic than that.

Definition: Coset

Let H be a subgroup of an abelian group $(G, +)$. For $a \in G$, the **coset** $a+H$ is the set $\{a+h : h \in H\}$ — "shift H by a." Two elements $a, b \in G$ turn out to produce the *same* coset, $a+H = b+H$, exactly when $a - b \in H$. This gives a clean way to sort all of G into non-overlapping "buckets," each one a shifted copy of H.

Example: Example 1 — The Simplest Possible Coset: Evens and Odds

Let $G = \mathbb{Z}$ and $H = 2\mathbb{Z} = \{\ldots, -4, -2, 0, 2, 4, \ldots\}$ (the even numbers; check it's a subgroup exactly as in the "Subgroups You Can Picture" example: contains 0, sum of two evens is even, negative of an even is even). Now form cosets by picking different a's:

- $0 + H = \{\ldots, -4, -2, 0, 2, 4, \ldots\}$ — same set as H itself (the evens)
- $1 + H = \{\ldots, -3, -1, 1, 3, 5, \ldots\}$ — the odds
- $2 + H = \{\ldots, -2, 0, 2, 4, 6, \ldots\}$ — same set as $0 + H$ again!
- $3 + H = \{\ldots, -1, 1, 3, 5, 7, \ldots\}$ — same set as $1 + H$ again!

No matter which integer you pick as a, you only ever land on **one of two** possible sets: "the evens" or "the odds." Every integer belongs to exactly one of these two buckets — that's the whole idea of cosets, in the simplest case there is.

Why did $0+H$ and $2+H$ turn out to be the same set? This is exactly the rule from the defbox above: $a + H = b + H$ iff $a - b \in H$. Check: $2 - 0 = 2$, and 2 is even, so $2 \in H$ — that's *why* they collapsed to the same coset. Meanwhile $1 - 0 = 1$, which is odd, not in H — so $1 + H$ is genuinely a different coset from $0 + H$.

Example: Example 2 — A "Clock" Version: The Cosets of $5\mathbb{Z}$ in $\mathbb{Z}$

Same idea as Example 1, one notch more general: let $H = 5\mathbb{Z} = \{\ldots, -10, -5, 0, 5, 10, \ldots\}$. There are exactly **5** distinct cosets:

Coset	What's in it	"Bucket"
$0 + 5\mathbb{Z}$	$\ldots, -5, 0, 5, 10, \ldots$	remainder 0
$1 + 5\mathbb{Z}$	$\ldots, -4, 1, 6, 11, \ldots$	remainder 1
$2 + 5\mathbb{Z}$	$\ldots, -3, 2, 7, 12, \ldots$	remainder 2
$3 + 5\mathbb{Z}$	$\ldots, -2, 3, 8, 13, \ldots$	remainder 3
$4 + 5\mathbb{Z}$	$\ldots, -1, 4, 9, 14, \ldots$	remainder 4

You have secretly been computing cosets your whole life: **a coset of $n\mathbb{Z}$ is just "everything with the same remainder when divided by n."** Every integer sorts into exactly one bucket, and there are exactly n buckets. This is precisely $\mathbb{Z}_n$ (Section 1.6), and it's exactly the same pattern as the $3\mathbb{Z}$ case used later in this lecture — there is nothing special about 5 here, only that it makes the "n buckets" pattern easy to see with more than 2 or 3 of them.

Example: Example 3 — A Picture You Can Draw: Lines in the Plane

This one is worth sitting with, because it connects straight to the lattice tilings of Section 1.8.5. Let $G = \mathbb{R}^2$ (the plane, under vector addition) and let H be a line through the origin — say $H = \{(t, t) : t \in \mathbb{R}\}$, the line $y = x$ (check it's a subgroup: contains $(0, 0)$; sum of two points on the line is on the line; negation of a point on the line is on the line).
Pick a point $a = (1, 0)$, not on the line, and form the coset:

$$(1, 0) + H = \{(1 + t,\ t) : t \in \mathbb{R}\}.$$

This is the line $y = x - 1$ — **the same line H, just shifted to pass through $(1, 0)$ instead of the origin.** Pick a different point on that same shifted line, say $a' = (3, 2)$: then $a' + H$ is *also* the line $y = x - 1$ — same coset, different name. Check the rule: $a - a' = (1, 0) - (3, 2) = (-2, -2)$, and $(-2, -2) = (t, t)$ with $t = -2$, so $(-2, -2) \in H$ — confirming $(1, 0) + H = (3, 2) + H$.
The picture: the cosets of H are exactly all the lines parallel to H. The whole plane $\mathbb{R}^2$ gets sliced into infinitely many parallel lines, and every point of the plane lies on exactly one of them. This "slicing the whole space into non-overlapping parallel copies" picture is *exactly* the fundamental-domain tiling picture of Section 1.8.5 — cosets and lattice tilings are the same idea, one dimension apart in this example.

Example: The Cosets of $3\mathbb{Z}$ in $\mathbb{Z}$ (the Version Used Later in This Lecture)

Take $G = \mathbb{Z}$ and $H = 3\mathbb{Z} = \{\ldots, -3, 0, 3, 6, \ldots\}$ (a subgroup, per the "Subgroups You Can Picture" example in Section 1.3.1). The cosets are:
- $0 + 3\mathbb{Z} = \{\ldots, -3, 0, 3, 6, \ldots\}$ (multiples of 3)
- $1 + 3\mathbb{Z} = \{\ldots, -2, 1, 4, 7, \ldots\}$ (remainder 1 when divided by 3)
- $2 + 3\mathbb{Z} = \{\ldots, -1, 2, 5, 8, \ldots\}$ (remainder 2 when divided by 3)

Every integer falls into exactly one of these three buckets — and which bucket it falls into is precisely its remainder mod 3. There is no fourth distinct coset: $3 + 3\mathbb{Z} = 0 + 3\mathbb{Z}$ (check: $3 - 0 = 3 \in 3\mathbb{Z}$), so the pattern repeats.

Remark: The Two Facts Worth Holding Onto

1. **Same-coset test:** $a + H = b + H$ if and only if $a - b \in H$ — this is how you check whether two things you've written down are secretly the same coset (used explicitly in all three examples above).
2. **No partial overlap:** two cosets are either *identical* or *completely disjoint* — never overlapping-but-not-equal. That's what lets cosets cleanly partition G into non-overlapping buckets, with nothing left over and nothing double-counted.

Definition: Quotient Group

The set of all distinct cosets of H in G, with addition defined by $(a+H)+(b+H) := (a+b)+H$, is itself a group — the **quotient group**, written G/H ("G mod H"). Its identity element is the coset $0 + H = H$ itself.

Remark: $\mathbb{Z}_n$ Is Not Just *Like* $\mathbb{Z}/n\mathbb{Z}$ — It *Is* $\mathbb{Z}/n\mathbb{Z}$

Compare the coset example above to Section 1.6: the three cosets of $3\mathbb{Z}$ in $\mathbb{Z}$ correspond *exactly* to the three elements $\{0, 1, 2\}$ of $\mathbb{Z}_3$, and coset addition matches addition mod 3 exactly. This is not a coincidence or an analogy — it is a theorem: $\mathbb{Z}_n$, as defined informally in Section 1.6, is precisely the quotient group $\mathbb{Z}/n\mathbb{Z}$. "Wraparound arithmetic" and "sorting integers into cosets of $n\mathbb{Z}$" are two descriptions of the exact same object. This is the rigorous justification for a claim we made without proof in Section 1.6: that $\mathbb{Z}_n$ really is a well-defined group (in fact ring) under addition and multiplication with wraparound — it inherits this structure directly from being a quotient of $\mathbb{Z}$.

Exercise: Practice

Let $H = 4\mathbb{Z} \subseteq \mathbb{Z}$. List all distinct cosets of H in $\mathbb{Z}$, and confirm there are exactly 4 of them, matching $\mathbb{Z}_4 = \{0, 1, 2, 3\}$. Which coset does -5 belong to?

1.4 Rings

A ring is what you get when you ask a set to support *two* binary operations that play two structurally different roles — one behaves like addition (fully invertible), the other behaves like multiplication (weaker, no inverses required). We always *label* these two operations "$+$" and "$\times$," but — and this is worth being precise about, since it's easy to misread — **the labels do not mean the operations have to literally be school-arithmetic addition and multiplication**. They can be almost any two operations at all, as long as together they satisfy the axioms below. "$+$" and "$\times$" are names for the *roles* the two operations play, not a requirement about what the operations must concretely do.

Definition: Ring

A set R with two binary operations $+$ and $\times$ is a **ring** $(R, +, \times)$ if:

(R1) $(R, +)$ is an abelian group (with identity element usually written 0).

(R2) $\times$ is associative: $(a \times b) \times c = a \times (b \times c)$.

(R3) **Distributivity of $\times$ over $+$:** $\times$ distributes over $+$, not the other way around — i.e., "multiplying a sum" can be split into "sum of the multiplications," but there is no requirement (and generally no meaning) for $+$ to distribute over $\times$. This comes in two versions, since $\times$ need not be commutative (see the note below), so both the left and right versions must be stated separately:

$$\underbrace{a \times (b + c) = (a \times b) + (a \times c)}_{\text{left distributivity}}, \qquad \underbrace{(a + b) \times c = (a \times c) + (b \times c)}_{\text{right distributivity}}.$$

Note: a ring does *not* need multiplicative inverses, and multiplication need not be commutative in general (though in every ring we use in this thesis, it will be — these are called **commutative rings**). If there is an element $1 \in R$ with $a \times 1 = 1 \times a = a$ for all a, we say R is a **ring with unity**.

Remark: Why Two Distributive Laws, and Why Only Over $+$

When $\times$ is commutative (as in every ring used in this course), left and right distributivity say the same thing in different order, so either one implies the other. But the *axioms* state both, because a ring is not required to be commutative in general (Section 1.3's matrix example under multiplication is a case where order matters). What never appears, in any ring, is a law like $a + (b \times c) = (a + b) \times (a + c)$ — $+$ never needs to distribute over $\times$. Distributivity is a one-way bridge: it tells you how $\times$ interacts with $+$, not the reverse.

Remark: "$+$" and "$\times$" Are Role Labels, Not a Restriction to Arithmetic

Compare this to groups (Section 1.3): a group has *one* operation, and we wrote it as a generic $*$ precisely to emphasize that it could be anything (addition, multiplication, matrix multiplication, function composition — whatever satisfies the axioms). A ring needs *two* operations, and it turns out we always call them "$+$" and "$\times$" — but this is a naming convention for the two *roles*, not a claim that the operations must be ordinary numeric addition and multiplication. Concretely:

- $n \times n$ **matrices:** "$+$" is ordinary matrix addition (this forms an abelian group), and "$\times$" is matrix *multiplication* (associative, distributes over $+$) — but matrix multiplication is **not** commutative in general, which is fine, since a ring's axioms above don't require it (only *commutative* rings require it, and this one wouldn't qualify as one).
- **Boolean rings** (used in digital logic): "$+$" is XOR and "$\times$" is AND on $\{0, 1\}$ — nothing to do with school arithmetic at all, yet every ring axiom above holds.

So whenever you meet a new ring in this course, the first question to ask is not "is $+$ really addition and is $\times$ really multiplication?" but rather: *what are the two operations here, and do they satisfy axioms (R1)–(R3)?* The labels "$+$"/"$\times$" just tell you which operation is playing which role once you've checked that.

Example: Rings You Already Know

- $(\mathbb{Z}, +, \times)$ — the integers form a commutative ring with unity 1. Notice 2 still has no multiplicative inverse, and that's fine — rings don't require one.
- **Polynomial rings**, e.g. $\mathbb{Z}[x]$: the set of *all* polynomials with integer coefficients, **of every degree at once** — no upper bound on degree at all. It contains 7 (degree 0), $3x - 1$ (degree 1), $3x^2 - x + 7$ (degree 2), and so on, all mixed together in one set, with ordinary polynomial addition and multiplication. This is a ring for exactly the same reason $\mathbb{Z}$ is: closure holds because adding or multiplying two polynomials, of *any* finite degrees, always produces another polynomial of *some* finite degree — still inside $\mathbb{Z}[x]$, whatever that degree turns out to be.

  Contrast this with P_n from Section 1.3's "Groups Hiding in Less Obvious Places" example, the set of polynomials of degree *strictly less than* a fixed n. There, closure under addition holds (adding two degree-$(n-1)$ polynomials keeps the degree below n), but closure under multiplication fails (the product can reach degree $2n-2$, escaping P_n) — which is precisely why $(P_n, +)$ is only a group, never a ring. $\mathbb{Z}[x]$ avoids this problem by simply not capping the degree in the first place, so there is no ceiling for a product to break through.

Looking Ahead: The Ring That Kyber and Dilithium Actually Use

Both Kyber and Dilithium do their arithmetic inside a specific ring of the form

$$R_q = \mathbb{Z}_q[x]/(x^n + 1),$$

read as: "polynomials with coefficients taken modulo q (Section 1.6), where we also reduce powers of x using the rule $x^n \equiv -1$." This looks intimidating right now, but it is built from exactly the pieces in this lecture: a polynomial ring (this section), with coefficients from $\mathbb{Z}_q$ (Section 1.6), further "folded" by a fixed rule. We will unpack this fully in a later lecture — for now, just notice that "ring" is not an abstract curiosity; it's the literal object the cryptography is built on.

1.4.1 Zero Divisors: A Structural Reason Some Rings Aren't Fields

We will soon see (Section 1.5) that $\mathbb{Z}_n$ is a field exactly when n is prime. Section 1.6.1 will justify this computationally, via gcd. Here is the deeper, structural reason *why* — worth seeing now, while we're still inside general ring theory.

Definition: Zero Divisor

In a ring R, a nonzero element a is a **zero divisor** if there exists some *nonzero* $b \in R$ with $a \times b = 0$. A commutative ring with unity that has *no* zero divisors (other than needing a or b to be 0) is called an **integral domain**.

Example: Zero Divisors in $\mathbb{Z}_6$, Absent in $\mathbb{Z}_5$

In $\mathbb{Z}_6$: $2 \times 3 = 6 \equiv 0 \pmod 6$, but neither 2 nor 3 is 0. So 2 and 3 are both zero divisors — $\mathbb{Z}_6$ is *not* an integral domain. Contrast this with $\mathbb{Z}_5$: checking every pair of nonzero elements $(1, 2, 3, 4)$, no product is ever $\equiv 0 \pmod 5$ (e.g., $2 \times 3 = 6 \equiv 1$, $4 \times 4 = 16 \equiv 1$, etc.) — $\mathbb{Z}_5$ has no zero divisors.

Remark: Why This Explains the Prime/Composite Split

Fact: every field is automatically an integral domain (if $a \neq 0$ and $ab = 0$, multiply both sides by a^{-1} to get $b = 0$ — so no nonzero b can pair with a nonzero a to give 0). Contrapositive: *if a ring has a zero divisor, it cannot be a field.* Now, $\mathbb{Z}_n$ has a zero divisor exactly when n is composite: write $n = jk$ with $1 < j, k < n$; then $j \times k = n \equiv 0 \pmod{n}$, with neither j nor k equal to 0 in $\mathbb{Z}_n$. So every composite modulus produces zero divisors, which immediately rules out being a field — a purely structural explanation, with no gcd computation needed, of the same fact Section 1.6.1 reaches computationally.

Exercise: Practice

Find a pair of zero divisors in $\mathbb{Z}_{12}$ (there are several valid answers). Then explain, using the Fact above, why $\mathbb{Z}_{12}$ cannot be a field — without computing any inverses directly.

1.5 Fields

Definition: Field

A **field** is a set F with two binary operations $+$ and $\times$ such that:
(F1) $(F, +)$ is an abelian group, over *all* of F (identity element 0).
(F2) $(F \setminus \{0\}, \times)$ is an abelian group, over F **with** 0 **removed** (identity element 1).
(F3) **Distributivity of $\times$ over $+$:** $a \times (b + c) = (a \times b) + (a \times c)$. (Only this one direction needs stating here, unlike the Ring definition above: (F2) already makes $\times$ commutative on $F \setminus \{0\}$, and $0 \times x = x \times 0 = 0$ always, so left and right distributivity coincide automatically — there's no separate "right" version to write down.)
Informally: a field is a set where you can add, subtract, multiply, and divide (by anything except 0) and always stay inside the set. In short — **a field is two abelian groups sharing one set, glued together by distributivity.**

Remark: Why 0 Has to Be Excluded, and It's Not Optional

In *any* structure with a distributive law, $0 \times x = 0$ for every x (this follows from distributivity itself, not from an extra rule: $0 \times x = (0 + 0) \times x = 0 \times x + 0 \times x$, so subtracting $0 \times x$ from both sides gives $0 = 0 \times x$). So 0 could only have a multiplicative inverse if $0 \times x = 1$ for some x — but the left side is always 0, never 1 (as long as $1 \neq 0$, which we always assume). So 0 is permanently excluded from having an inverse, in every field, without exception; it isn't a special case someone chose, it's forced by the arithmetic.

Definition: Subtraction and Division Are *Derived*, Not Extra

Neither subtraction nor division is a new, independent operation requiring its own axiom — both are just notation for "combine with an inverse," one level for each operation:

$$a - b := a + (-b), \qquad \frac{a}{b} := a \times b^{-1} \ \ (b \neq 0),$$

where $-b$ is guaranteed to exist by $(F, +)$ being a group (F1), and b^{-1} is guaranteed to exist, for every *nonzero* b, by $(F \setminus \{0\}, \times)$ being a group (F2). So "a field supports division" is not an extra requirement on top of the definition — it falls straight out of (F2) once every nonzero element is already promised an inverse. This is exactly why (F2) excludes 0: since 0 can never have an inverse (see the remark above), "dividing by 0" can never be defined this way, no matter what field you're in.

Example: Division in $\mathbb{Z}_5$, Worked Out

What is $4 \div 3$ in $\mathbb{Z}_5$? First find $3^{-1} \bmod 5$: $3 \times 2 = 6 \equiv 1 \pmod 5$, so $3^{-1} = 2$. Then $4 \div 3 := 4 \times 3^{-1} = 4 \times 2 = 8 \equiv 3 \pmod 5$. Check: $3 \times 3 = 9 \equiv 4 \pmod 5$ — correct, exactly as "$4 \div 3 = 3$" should verify.

Remark: The One-Line Takeaway

Subtraction is the additive inverse; division is the multiplicative inverse. Same trick, applied once per operation — $a - b$ means "add a and b's additive inverse," a/b means "multiply a by b's multiplicative inverse." The one asymmetry between them traces directly back to (F1) vs. (F2): *every* element of F has an additive inverse, since $(F, +)$ is a group over *all* of F — so subtraction always works. But *only nonzero* elements have a multiplicative inverse, since $(F \setminus \{0\}, \times)$ is a group only after removing 0 — so division by b only ever works when $b \neq 0$.

Fact: Every Field Is a Ring

Every field F (Definition above) is automatically a commutative ring with unity — in fact, an **integral domain** (Section 1.4.1). *Why:* (F1) is exactly ring axiom (R1). Associativity and distributivity of $\times$ (ring axioms (R2)–(R3)) are inherited directly from $\times$ being a group operation on $F \setminus \{0\}$ together with (F3). So a field satisfies every ring axiom, plus two extra guarantees stacked on top: $\times$ is commutative, and every *nonzero* element is invertible. This is why some textbooks instead *define* a field as "a commutative ring with unity in which every nonzero element has a multiplicative inverse" — that phrasing and the definition above pick out exactly the same objects; it's just built in the reverse order (starting from "ring" and adding a condition, instead of starting from two groups and gluing them together). Finally, a field is automatically an integral domain too: if $ab = 0$ with $a \neq 0$, multiplying both sides by a^{-1} forces $b = a^{-1}(ab) = a^{-1} \cdot 0 = 0$, so there can be no zero divisors.

Example: Fields and Non-Fields

- $\mathbb{Q}$ (rationals), $\mathbb{R}$ (reals), $\mathbb{C}$ (complex numbers) are all fields — you can divide by any nonzero element and stay inside the set.
- $\mathbb{Z}$ is **not** a field: $2 \in \mathbb{Z}$ has no multiplicative inverse in $\mathbb{Z}$ (again, $1/2 \notin \mathbb{Z}$). $\mathbb{Z}$ is a ring, but not a field.
- $\mathbb{Z}_5 = \{0, 1, 2, 3, 4\}$ under addition and multiplication mod 5 *is* a field — every nonzero element has an inverse (you checked this, or will check this, in the Section 1.3 subgroup/group exercise). Fields built this way, from $\mathbb{Z}_p$ with p prime, are called **finite fields** or **Galois fields**, written $\mathbb{F}_p$ or $GF(p)$.

Remark: The One-Sentence Summary So Far

Group $\subset$ Ring $\subset$ Field, in the sense that a field is a ring with extra structure (division), and a ring is built from a group with extra structure (a second operation). Each step adds a capability: groups let you undo one operation (inverses); rings add a second, distributive operation; fields guarantee *both* operations are fully invertible (except division by 0).

1.6 The Modulus: Arithmetic That Wraps Around

Definition: Congruence Modulo n

For a fixed positive integer n (the **modulus**), two integers a and b are **congruent modulo** n, written

$$a \equiv b \pmod{n},$$

if n divides $(a - b)$ evenly — equivalently, a and b leave the same remainder when divided by n. The set of possible remainders, $\{0, 1, 2, \ldots, n-1\}$, together with addition and multiplication performed "with wraparound," is written $\mathbb{Z}_n$ (or $\mathbb{Z}/n\mathbb{Z}$).

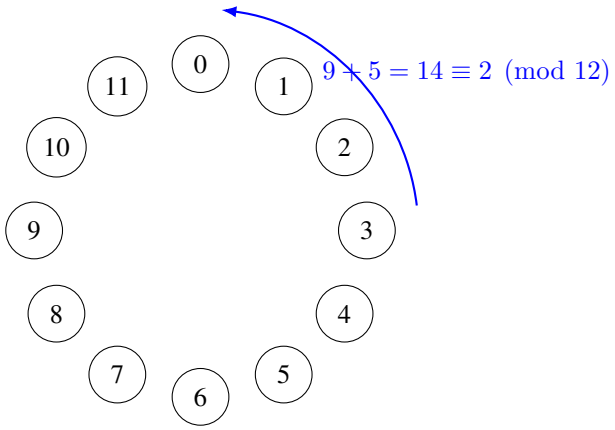


Figure 1.1: The clock analogy for modular arithmetic: $\mathbb{Z}_{12}$ works exactly like a 12-hour clock face. 9 o'clock plus 5 hours wraps around to 2 o'clock, i.e., $9 + 5 = 14 \equiv 2 \pmod{12}$.

Example: Computing Modulo

$17 \bmod 5$: divide 17 by 5 to get 3 remainder 2, so $17 \equiv 2 \pmod{5}$. Similarly, $23 \equiv 3 \pmod{10}$, and $-1 \equiv n - 1 \pmod{n}$ (e.g., $-1 \equiv 4 \pmod{5}$) — negative numbers wrap around too.

Remark: When Is $\mathbb{Z}_n$ a Ring? When Is It a Field?

$\mathbb{Z}_n$ is *always* a commutative ring with unity, for any $n \geq 2$. But it is a *field* only when $n = p$ is **prime**. If n is composite (e.g., $n = 6$), some nonzero elements fail to have multiplicative inverses — e.g., in $\mathbb{Z}_6$, there is no x with $2x \equiv 1 \pmod{6}$, since 2 shares a common factor with 6. This is exactly why cryptographic schemes that need a field (rather than just a ring) are built with a prime modulus q. (Section 1.4.1 gave the deeper structural reason: composite n always produces zero divisors, and a ring with zero divisors can never be a field.)

1.6.1 Composite Moduli Aren't Fields — But Their Invertible Elements Still Form a Group

The remark above closes the door firmly: $\mathbb{Z}_n$ for composite n is never a field, no matter how you look at it — some nonzero elements simply have no multiplicative inverse, and that's disqualifying, permanently. But it's natural to ask a follow-up question: *which* elements of $\mathbb{Z}_n$ *do* have an inverse, and how many of them are there? That subset turns out to be extremely well-behaved — just not a field, and not all of $\mathbb{Z}_n$.

Definition: Euler's Totient Function

For a positive integer n, **Euler's totient function** $\varphi(n)$ counts how many integers in $\{1, 2, \ldots, n\}$ are **coprime** to n (i.e., $\gcd(a, n) = 1$). Equivalently — recalling the "Fact: When Does an Inverse Exist?" remark box in the Extended Euclidean Algorithm subsection just below, $a \in \mathbb{Z}_n$ has a multiplicative inverse iff $\gcd(a, n) = 1$ — $\varphi(n)$ is exactly **the number of elements of $\mathbb{Z}_n$ that have a multiplicative inverse**.

Definition: The Group of Units $\mathbb{Z}_n^*$

The set of invertible elements of $\mathbb{Z}_n$,

$$\mathbb{Z}_n^* \;=\; \{a \in \mathbb{Z}_n : \gcd(a, n) = 1\},$$

is called the **group of units** modulo n. Under multiplication mod n, $(\mathbb{Z}_n^*, \times)$ is an abelian group (Section 1.3): it's closed (the product of two units is a unit — if a, b each have inverses, so does ab, namely $b^{-1}a^{-1}$), it has identity 1, and by definition every element already has an inverse. By construction, $|\mathbb{Z}_n^*| = \varphi(n)$.

Remark: This Is *Not* a Field — Read the Claim Carefully

It's tempting to summarize this as "$\mathbb{Z}_n$ is secretly a field with only $\varphi(n)$ elements," but that phrasing is not correct, and it's worth being precise about why. A field needs *every* nonzero element invertible, checked against the field's own addition (Section 1.5, axiom F1: $(F, +)$ a group over *all* of F). $\mathbb{Z}_n^*$ is not closed under $+$ at all — e.g., in $\mathbb{Z}_{12}^* = \{1, 5, 7, 11\}$, $5{+}7 = 12 \equiv 0 \pmod{12}$, which isn't even in $\mathbb{Z}_{12}^*$ (it's not coprime to 12). So $\mathbb{Z}_n^*$ is a perfectly good *group under multiplication alone* — it is **not** a smaller field sitting inside $\mathbb{Z}_n$; it doesn't have an addition operation of its own at all. The correct statement is: composite $\mathbb{Z}_n$ is a ring, not a field, but its invertible elements form a multiplicative group of order $\varphi(n)$.

Example: Computing $\varphi(12)$ and $\mathbb{Z}_{12}^*$ by Hand

List 1 through 12 and check gcd with $12 = 2^2 \times 3$ for each:

a	$\gcd(a, 12)$	coprime to 12?
1	1	yes
2	2	no
3	3	no
4	4	no
5	1	yes
6	6	no
7	1	yes
8	4	no
9	3	no
10	2	no
11	1	yes
12	12	no (this is 0 in $\mathbb{Z}_{12}$)

So $\mathbb{Z}_{12}^* = \{1, 5, 7, 11\}$ and $\varphi(12) = 4$. Check closure under multiplication mod 12: $5 \times 7 = 35 \equiv 11$, $5 \times 11 = 55 \equiv 7$, $7 \times 11 = 77 \equiv 5$, $5 \times 5 = 25 \equiv 1$, $7 \times 7 = 49 \equiv 1$, $11 \times 11 = 121 \equiv 1$ — every product of two elements of $\{1, 5, 7, 11\}$ lands back in $\{1, 5, 7, 11\}$, exactly as the "group of units" defbox above promises. Notice $\varphi(12) = 4 < 12$: only 4 of the 12 elements of $\mathbb{Z}_{12}$ are invertible, and the other 8 (including all the zero divisors found in the Section 1.4.1 example, 2 and 3) are permanently excluded from $\mathbb{Z}_{12}^*$.

Remark: A Formula, and the Prime Sanity Check

For a prime p, every one of $1, \ldots, p-1$ is automatically coprime to p (since p's only divisors are 1 and p itself), so $\varphi(p) = p - 1$ — matching exactly the earlier fact that *all* nonzero elements of $\mathbb{Z}_p$ are invertible when p is prime. In other words, $\mathbb{Z}_p^* = \mathbb{Z}_p \setminus \{0\}$ precisely when p is prime — the group of units swallows the entire nonzero ring, which is exactly the extra guarantee that makes $\mathbb{Z}_p$ a field (Section 1.5) and $\mathbb{Z}_{12}$ merely a ring. For composite n, $\varphi(n)$ is computed from the prime factorization of n (e.g., $\varphi(12) = \varphi(2^2)\varphi(3) = 2 \times 2 = 4$, matching the hand count above), but the formula itself isn't needed for this course — what matters is the concept: $\varphi(n)$ measures exactly how far $\mathbb{Z}_n$ falls short of being a field.

Looking Ahead: Why This Matters Later: RSA

Euler's totient function is the load-bearing quantity behind RSA (the classical, non-lattice-based scheme this thesis's cryptography ultimately aims to replace): RSA's security and correctness both hinge on $\varphi(n)$ for a composite $n = pq$ (product of two large primes), computed via $\varphi(pq) = (p-1)(q-1)$, and Euler's theorem, $a^{\varphi(n)} \equiv 1 \pmod{n}$ for $a \in \mathbb{Z}_n^*$, which generalizes Fermat's Little Theorem from prime moduli to composite ones. This isn't needed anywhere else in this lecture series, but it's worth recognizing the totient function on sight — it's one of the few classical-cryptography ideas that resurfaces constantly in the literature this thesis will cite.

Exercise: Practice

Compute $\varphi(10)$ by listing $\mathbb{Z}_{10}^*$ directly (check $\gcd(a, 10) = 1$ for $a = 1, \ldots, 10$). Then verify closure under multiplication mod 10 for at least two pairs of elements from your list, the way the $\varphi(12)$ example above did.

1.6.2 Finding Inverses Without Guessing: The Extended Euclidean Algorithm

So far, the only way we've found a multiplicative inverse mod n is trial and error (Exercise: "find x with $3x \equiv 1 \pmod{7}$" — you could just try $x = 1, 2, 3, \ldots$ until one works). That's fine for small numbers, but real cryptographic moduli are enormous (Kyber's modulus is $q = 3329$; RSA moduli have hundreds of digits) — trial and error is completely impractical. We need an actual algorithm.

Definition: Greatest Common Divisor (GCD)

For integers a, b (not both zero), $\gcd(a, b)$ is the largest positive integer that divides both a and b. Two integers are **coprime** (or relatively prime) if $\gcd(a, b) = 1$.

Remark: Fact: When Does an Inverse Exist?

An element $a \in \mathbb{Z}_n$ has a multiplicative inverse if and only if $\gcd(a, n) = 1$. This is why every nonzero element of $\mathbb{Z}_p$ (for prime p) has an inverse: any a with $1 \leq a \leq p - 1$ automatically satisfies $\gcd(a, p) = 1$, since p's only divisors are 1 and p itself.

The classical **Euclidean Algorithm** computes $\gcd(a, b)$ by repeated division with remainder: $\gcd(a, b) = \gcd(b, a \bmod b)$, stopping when the remainder hits 0. The **Extended** Euclidean Algorithm does this same process but also tracks, at *every single step*, how the current remainder can be written as a combination $a \cdot x + b \cdot y$ (integers x, y) — and when the algorithm finishes with $\gcd(a, b) = 1$, the row where the remainder equals 1 hands you the inverse directly. The cleanest way to track this — much easier to follow than plain back-substitution — is a running **table** with one row per step.

Definition: The Extended Euclidean Algorithm, as a Table

To find $\gcd(a, b)$ together with integers x, y satisfying $a \cdot x + b \cdot y = \gcd(a, b)$, build a table with columns q (quotient), r (remainder), x, y, starting with two fixed initial rows and then one new row per division step:

step	q	r	x	y
-1	—	a	1	0
0	—	b	0	1
1	$q_1 = \lfloor r_{-1}/r_0 \rfloor$	$r_1 = r_{-1} - q_1 r_0$	$x_1 = x_{-1} - q_1 x_0$	$y_1 = y_{-1} - q_1 y_0$
2	$q_2 = \lfloor r_0/r_1 \rfloor$	$r_2 = r_0 - q_2 r_1$	$x_2 = x_0 - q_2 x_1$	$y_2 = y_0 - q_2 y_1$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Every single row satisfies the **invariant** $r_i = a \cdot x_i + b \cdot y_i$ by construction — this is the whole mechanism, and it's worth checking explicitly on at least one row the first time you use this table, to convince yourself it isn't magic. Stop once a row produces $r_i = 0$; the *previous* row's r is $\gcd(a, b)$, and that row's x, y are exactly the combination you want.

Example: Table Method, Short Version: $\gcd(7, 3)$ and $3^{-1} \bmod 7$

step	q	r	x	y	check: $7x + 3y$
-1	—	7	1	0	$7(1) + 3(0) = 7$ ✓
0	—	3	0	1	$7(0) + 3(1) = 3$ ✓
1	$\lfloor 7/3 \rfloor = 2$	$7 - 2(3) = 1$	$1 - 2(0) = 1$	$0 - 2(1) = -2$	$7(1) + 3(-2) = 1$ ✓
2	$\lfloor 3/1 \rfloor = 3$	$3 - 3(1) = 0$	$0 - 3(1) = -3$	$1 - 3(-2) = 7$	(stop: $r = 0$)

The row *before* r hit 0 is step 1: $r_1 = 1 = \gcd(7, 3)$, with $x_1 = 1, y_1 = -2$. So $7(1)+3(-2) = 1$, i.e., $3 \times (-2) \equiv 1 \pmod 7$, giving $3^{-1} \equiv -2 \equiv 5 \pmod 7$ — matching what trial and error would also find, but now via a method with a clear, mechanical stopping point.

Example: Table Method, Longer Version: $15^{-1} \bmod 26$

Here the table earns its keep — more steps than the tiny example above, but the process is identical, row by row.

step	q	r	x	y
-1	—	26	1	0
0	—	15	0	1
1	$\lfloor 26/15 \rfloor = 1$	$26 - 1(15) = 11$	$1 - 1(0) = 1$	$0 - 1(1) = -1$
2	$\lfloor 15/11 \rfloor = 1$	$15 - 1(11) = 4$	$0 - 1(1) = -1$	$1 - 1(-1) = 2$
3	$\lfloor 11/4 \rfloor = 2$	$11 - 2(4) = 3$	$1 - 2(-1) = 3$	$-1 - 2(2) = -5$
4	$\lfloor 4/3 \rfloor = 1$	$4 - 1(3) = 1$	$-1 - 1(3) = -4$	$2 - 1(-5) = 7$
5	$\lfloor 3/1 \rfloor = 3$	$3 - 3(1) = 0$	$3 - 3(-4) = 15$	$-5 - 3(7) = -26$

The row before r hit 0 is step 4: $r_4 = 1 = \gcd(26, 15)$, with $x_4 = -4, y_4 = 7$. **Check the invariant directly:** $26(-4) + 15(7) = -104 + 105 = 1$ ✓. Reading this modulo 26: $15 \times 7 \equiv 1 \pmod{26}$, so $15^{-1} \equiv 7 \pmod{26}$. **Final check:** $15 \times 7 = 105 = 4 \times 26 + 1 \equiv 1 \pmod{26}$. Correct.

Exercise: Practice

Use the table method above (not back-substitution, and not trial and error) to find $5^{-1} \bmod 11$. Build the full table — steps $-1, 0, 1, 2, \ldots$ — and stop at the correct row. Then check your answer by direct multiplication.

Looking Ahead: Why Cryptography Loves the Modulus

Modular arithmetic gives us two things cryptography needs badly: (1) a **finite** number system (so keys and computations have a fixed, bounded size, no matter how much you compute), and (2) built-in **one-wayness** in places — e.g., given only $a \bmod n$, you cannot tell which of infinitely many integers a actually was. The Learning-With-Errors (LWE) problem from the proposal document hides its secret inside equations that live in exactly this kind of finite, modular number system (there, the modulus is usually called q).

Exercise: Practice

(a) Compute $23 \times 15 \bmod 7$. (b) List all elements of $\mathbb{Z}_8$ that *do not* have a multiplicative inverse, and explain why (hint: compare each element to 8). (c) Confirm $\mathbb{Z}_7$ is a field by checking that 3 has a multiplicative inverse mod 7 (find x such that $3x \equiv 1 \pmod 7$).

1.7 Vector Spaces, Basis, and Dimension

Definition: Vector Space

A **vector space** over a field F is a set V (whose elements are called **vectors**) equipped with an addition operation $V \times V \to V$ and a scalar multiplication operation $F \times V \to V$, satisfying, for all $u, v, w \in V$ and all $a, b \in F$:

(V1) **Closure of addition:** $u + v \in V$.
(V2) **Associativity of addition:** $(u + v) + w = u + (v + w)$.
(V3) **Commutativity of addition:** $u + v = v + u$.
(V4) **Additive identity:** there exists $\vec{0} \in V$ with $v + \vec{0} = v$ for all v.
(V5) **Additive inverses:** for every $v \in V$, there exists $-v \in V$ with $v + (-v) = \vec{0}$.
(V6) **Compatibility of scalar multiplication with field multiplication:** $a(bv) = (ab)v$.
(V7) **Identity element of scalar multiplication:** $1v = v$, where 1 is the multiplicative identity of F.
(V8) **Distributivity, two directions:** $a(u + v) = au + av$ *and* $(a + b)v = av + bv$.

Axioms (V1)–(V5) say exactly "$(V, +)$ is an abelian group" (Section 1.3) — nothing new there. Axioms (V6)–(V8) are the genuinely new content: they say scalar multiplication by field elements behaves consistently with the field's own addition and multiplication (Sections 4–5). The most important example for us: $\mathbb{R}^n$, the set of all n-tuples of real numbers, is a vector space over $\mathbb{R}$ — and it's a good exercise to mentally check each of (V1)–(V8) holds there before moving on.

Definition: Linear Combination, Span, Linear Independence

Given vectors $v_1, \ldots, v_k \in V$ and scalars $c_1, \ldots, c_k \in F$, the vector $c_1v_1 + c_2v_2 + \cdots + c_kv_k$ is called a **linear combination** of $v_1, \ldots, v_k$. The set of *all* linear combinations of a set of vectors is called their **span**. The vectors $v_1, \ldots, v_k$ are **linearly independent** if the only way to write $0 = c_1v_1 + \cdots + c_kv_k$ is with every $c_i = 0$ — informally, no vector in the set is "redundant" (expressible using the others).

1.7.1 Testing Linear Independence: Row Operations and Row Echelon Form

The definition above is a yes/no test stated abstractly in terms of the c_i's. For concrete vectors in $\mathbb{R}^n$, it is also a completely mechanical computation — and the standard tool for carrying it out, **Gaussian elimination**, is worth setting up properly here, since we will lean on it again once bases and lattices enter the picture.

Definition: Elementary Row Operations

Given a matrix, the following three operations on its rows are called **elementary row operations**:

(E1) **Row swap:** interchange two rows, $R_i \leftrightarrow R_j$.

(E2) **Row scaling:** multiply a row by a nonzero scalar, $R_i \to c\,R_i$ for some $c \neq 0$.

(E3) **Row addition (replacement):** add a scalar multiple of one row to another, $R_i \to R_i + c\,R_j$ (for $i \neq j$).

Each operation is **reversible** by another operation of the same type: a swap undoes itself; scaling by c is undone by scaling by $1/c$; adding cR_j is undone by adding $-cR_j$. This reversibility is exactly why row operations never destroy information — every row of the new matrix is some combination of the old rows, and (because the operation can be undone) every old row is likewise recoverable as a combination of the new ones.

Fact: Row Operations Preserve the Span — and Hence Independence — of the Rows

Let $v_1, \ldots, v_k$ be the rows of a matrix A. Applying any sequence of elementary row operations produces a new matrix A' whose rows $v_1', \ldots, v_k'$ **span exactly the same space** as $v_1, \ldots, v_k$ did — because, by the remark above, each new row is some linear combination of the old rows, and each old row is recoverable as a combination of the new ones. In particular: $v_1, \ldots, v_k$ are linearly independent **if and only if** $v_1', \ldots, v_k'$ are. Row operations are therefore "free" for this purpose — we can simplify a set of vectors as aggressively as we like without ever changing the yes/no answer to the independence question, exactly the same way row operations leave $\gcd$, solution sets, and (as we'll see in Section 1.8.5) a lattice's determinant unchanged under the right kind of transformation.

Definition: Row Echelon Form

A matrix is in **row echelon form (REF)** if:

(1) Every all-zero row (if any) lies below every nonzero row.

(2) In each nonzero row, its first nonzero entry — the **leading entry** or **pivot** — lies strictly to the right of the leading entry of the row above it (so the pivots form a descending "staircase").

Any matrix can be brought to row echelon form using finitely many elementary row operations — systematically eliminating entries below each pivot, top to bottom, is exactly **Gaussian elimination**. The number of nonzero rows in the resulting echelon form is called the **rank** of the matrix; by the Fact above, this number doesn't depend on which particular sequence of row operations you used to get there.

Remark: Turning This Into an Independence Test

Stack $v_1, \ldots, v_k \in \mathbb{R}^n$ as the k rows of a $k \times n$ matrix A, and row-reduce A to echelon form A'. Since row operations never change independence, $v_1, \ldots, v_k$ are linearly independent **exactly when A' has no zero row** — equivalently, when $\operatorname{rank}(A) = k$ (every row produced its own pivot). If instead a zero row appears, the original vectors are dependent — and the specific combination of row operations that produced that zero row *is* a witness: it records exactly which combination of the original rows sums to $\vec{0}$. The worked example below shows a clean way to read that combination off directly.

Example: Row Reduction in Action: Testing $v_1 = (1, 2, 3)$, $v_2 = (2, 3, 4)$, $v_3 = (1, 1, 1)$

Stack the vectors as rows, and — to make the eventual dependency relation easy to read off — augment the matrix on the right with the 3×3 identity, one column per original vector:

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 2 & 3 & 4 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 \end{array}\right] \begin{array}{l} \leftarrow v_1 \\ \leftarrow v_2 \\ \leftarrow v_3 \end{array}$$

The idea: whatever combination of rows we apply to the left block, we apply *identically* to the right block — so the right block always records "this row currently equals (this combination) of v_1, v_2, v_3."

Step 1: eliminate below the first pivot. Use R_1 (pivot in column 1) to clear column 1 in R_2 and R_3:

$$R_2 \to R_2 - 2R_1, \qquad R_3 \to R_3 - R_1.$$

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & -1 & -2 & -2 & 1 & 0 \\ 0 & -1 & -2 & -1 & 0 & 1 \end{array}\right]$$

Step 2: eliminate below the second pivot. R_2's leading entry is now in column 2; use it to clear column 2 in R_3:

$$R_3 \to R_3 - R_2.$$

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & -1 & -2 & -2 & 1 & 0 \\ 0 & 0 & 0 & 1 & -1 & 1 \end{array}\right]$$

This is row echelon form: two pivots (row 1 in column 1, row 2 in column 2), and one zero row — so $\text{rank} = 2 < 3 = k$, meaning v_1, v_2, v_3 **are linearly dependent**.

Reading off the dependency. Row 3's right-hand block, $(1, -1, 1)$, records exactly which combination of the *original* rows produced that zero row: $1 \cdot v_1 + (-1) \cdot v_2 + 1 \cdot v_3$. Check directly: $v_1 - v_2 + v_3 = (1, 2, 3) - (2, 3, 4) + (1, 1, 1) = (0, 0, 0)$ ✓. So $c_1 = 1,\ c_2 = -1,\ c_3 = 1$ is a non-trivial solution — equivalently, $v_2 = v_1 + v_3$, which you can also confirm directly.

Remark: Why More Vectors Than Dimensions Forces Dependence

If $k > n$ (more vectors than the ambient dimension), the vectors are *automatically* dependent — no computation needed. Setting up $c_1 v_1 + \cdots + c_k v_k = \vec{0}$ gives n equations (one per coordinate) in $k > n$ unknowns $c_1, \ldots, c_k$; row-reducing the coefficient matrix can produce at most n pivots (one per row), so at least $k - n$ of the unknowns are left as **free variables** — and a homogeneous system with a free variable always has a nontrivial solution (just set the free variable to 1 and solve for the rest). We will meet this exact fact again, in geometric disguise, once we reach lattices in Section 1.8.

Definition: Basis and Dimension

A set of vectors $B = \{b_1, \ldots, b_k\}$ is a **basis** of a vector space V if (1) B is linearly independent, and (2) B **spans** V (every vector in V can be written as a linear combination of B). Every basis of a given vector space has the same number of elements k, called the **dimension** of V.

Example: Basis of $\mathbb{R}^2$

The vectors $e_1 = (1, 0)$ and $e_2 = (0, 1)$ form the "standard" basis of $\mathbb{R}^2$: any vector (a, b) is exactly $a \cdot e_1 + b \cdot e_2$. But this is far from the *only* basis — $\{(1, 1), (1, -1)\}$ is also a valid basis of $\mathbb{R}^2$ (check: they're linearly independent, and any (a, b) can be written as a combination of them). A vector space almost always has infinitely many possible bases; some are more convenient to work with than others. **Keep this fact in mind — it becomes very important in Section 1.8.**

Remark: A Quick Shortcut When $k = n$

When there are exactly as many vectors as dimensions, there's a fast special-case test that avoids row reduction entirely: stack $v_1, \ldots, v_n$ as the columns (or rows) of a square matrix A and compute $\det(A)$. Nonzero determinant $\Rightarrow$ independent; zero determinant $\Rightarrow$ dependent. This is a quick yes/no check, but — unlike the row-reduction method above — it doesn't by itself hand you the explicit nontrivial combination when the answer is "dependent"; for that, fall back on row reduction (optionally with the identity-augmentation trick above).

Exercise: Practice: Testing Linear Independence

For each set of vectors in $\mathbb{R}^3$ below, determine whether the set is linearly independent. If a set turns out to be **dependent**, don't just say so — exhibit an explicit **non-trivial** solution (scalars c_1, c_2, c_3, not all zero) to $c_1v_1 + c_2v_2 + c_3v_3 = \vec{0}$, and check it by substitution. Use row reduction (with the identity-augmentation trick from the worked example above) at least once, even where the answer is also visible by inspection.

(a) $v_1 = (2, 4, 6),\ v_2 = (1, 2, 3),\ v_3 = (0, 1, 0)$.
(b) $v_1 = (1, 0, 0),\ v_2 = (1, 1, 0),\ v_3 = (1, 1, 1)$.
(c) $v_1 = (1, 2, 1),\ v_2 = (2, 1, 3),\ v_3 = (4, 5, 5)$.

(Hint for part (c): unlike part (a), the dependency here isn't visible by inspection — row-reduce the augmented matrix $[\, v_1; v_2; v_3 \mid I_3 \,]$ exactly as in the worked example above.)

1.7.2 Inner Products, Norms, and Orthogonality

To make "good basis" (short, nearly-perpendicular vectors) versus "bad basis" (long, nearly-parallel vectors) precise rather than just intuitive, we need a way to measure length and angle.

Definition: Inner Product and Norm

For $x = (x_1, \ldots, x_n), y = (y_1, \ldots, y_n) \in \mathbb{R}^n$, the (standard) **inner product** is $\langle x, y \rangle = \sum_{i=1}^{n} x_i y_i$. The **norm** (length) of x is $\|x\| = \sqrt{\langle x, x \rangle}$. Two vectors are **orthogonal** if $\langle x, y \rangle = 0$ (the angle between them is $90°$). A basis $\{b_1, \ldots, b_k\}$ is an **orthogonal basis** if every pair b_i, b_j $(i \neq j)$ is orthogonal.

Example: Computing Norms and Checking Orthogonality

For $x = (3, 4)$: $\|x\| = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$. For $x = (1, 1)$ and $y = (1, -1)$: $\langle x, y \rangle = (1)(1) + (1)(-1) = 0$, so these two vectors (the alternate basis of $\mathbb{R}^2$ from the previous example) are orthogonal — a nicer basis, geometrically, than it might have looked at first glance.

Remark: Extending the Inner Product to Complex Vectors

The formula $\langle x, y\rangle = \sum_i x_i y_i$ above is only correct for *real* vectors, $x, y \in \mathbb{R}^n$. It breaks down over $\mathbb{C}^n$: take a single complex coordinate, $x = (i)$. The formula gives $\langle x, x\rangle = i \cdot i = i^2 = -1$, so $\|x\| = \sqrt{-1}$ — not a real number at all, let alone a sensible "length." A norm must be a non-negative real number, so this naive formula simply stops being a norm once complex entries are allowed.

The standard fix is to place a **complex conjugate** on one argument,

$$\langle x, y\rangle = \sum_{i=1}^{n} x_i \overline{y_i} \qquad \text{(the } \textit{Hermitian} \text{ inner product)},$$

where $\overline{a + bi} = a - bi$. Now $\langle x, x\rangle = \sum_i x_i \overline{x_i} = \sum_i |x_i|^2$ — a sum of non-negative real numbers, always real and always ≥ 0, exactly what a norm needs. And whenever every entry happens to be real, $\overline{y_i} = y_i$, so this formula collapses back *exactly* to the plain dot product in the defbox above — the conjugate only ever does work when the entries are genuinely complex, and is invisible (a no-op) on real vectors. This is why the definition above was stated over $\mathbb{R}^n$ specifically: every lattice in Lectures 1–4 lives in $\mathbb{R}^n$ or $\mathbb{Z}^n$, so the plain real formula is all we need for now — the moment complex vectors enter the picture, however, conjugating becomes required, not optional.

Looking Ahead: Where Complex Vectors Actually Show Up (Preview of Falcon, Lecture 4)

This isn't a hypothetical fix-it-just-in-case. Falcon's signing step ("fast Fourier sampling," Lecture 4) embeds a ring element of $R_q = \mathbb{Z}_q[x]/(x^n+1)$ into $\mathbb{C}^n$ using the n complex roots of x^n+1 — the **canonical embedding** — because in that embedding, the otherwise-awkward polynomial multiplication in R_q becomes simple coordinate-wise multiplication of complex numbers, which is exactly what makes fast Fourier sampling fast. Every norm and inner product computed inside that embedding *is* the Hermitian version defined above, not the plain real dot product. Keep this box in mind when Falcon's sampler resurfaces in Lesson 6: part of why its floating-point implementation is so delicate to secure is that it's performing real, finite-precision arithmetic on quantities that are conceptually complex-number computations underneath.

Looking Ahead: Quantifying "Good" vs. "Bad" Bases

With norms in hand, we can finally give a real, measurable meaning to the "good basis / bad basis" idea flagged repeatedly in this lecture: a good basis has vectors with small norm $\|b_i\|$ that are close to orthogonal; a bad basis for the *same* lattice can have arbitrarily large norms and near-parallel vectors, despite generating an identical set of points. (Here L denotes the **lattice** generated by the basis $\{b_1, \ldots, b_n\}$ — all integer combinations of the b_i's, i.e. the grid of points they produce; we've been using this good-basis/bad-basis idea informally for a few pages now, and Section 1.8 below is where L finally gets a full, formal definition. For now, just read L as "the lattice this basis describes.") One common numerical measure of "how good" a basis is is its **orthogonality defect**, $\dfrac{\prod_i \|b_i\|}{\det(L)}$, where $\det(L)$ is a single positive number — the lattice's fundamental-cell volume — that depends only on L itself, not on which basis is used to describe it (defined precisely in Section 1.8.5). A good basis has vectors close in length to $\det(L)^{1/n}$ and close to orthogonal, which drives this ratio down toward its minimum possible value of 1; a bad basis inflates the numerator $\prod_i \|b_i\|$ without changing $\det(L)$ at all, so the ratio grows the more skewed the basis is.

1.7.3 Gram–Schmidt Orthogonalization

Given *any* basis, can we systematically produce an orthogonal one (spanning the same space, though generally *not* the same lattice — more on that distinction below)? Yes — and the entire idea rests on one simple geometric move: **taking a vector and subtracting off however much of it points along some other direction**, leaving only the part that's genuinely new. Let's make that move precise before writing down the full process.

Definition: Vector Projection

The **projection** of a vector v onto a (nonzero) vector u is

$$\mathrm{proj}_u(v) \;=\; \frac{\langle v, u\rangle}{\langle u, u\rangle}\, u.$$

Geometrically, $\mathrm{proj}_u(v)$ is the "shadow" v casts on the line through u — the closest point to v that lies on that line, i.e. the component of v pointing in the u direction. The leftover piece, $v - \mathrm{proj}_u(v)$, is exactly the component of v perpendicular to u: check that $\langle v - \mathrm{proj}_u(v),\, u\rangle = \langle v, u\rangle - \frac{\langle v,u\rangle}{\langle u,u\rangle}\langle u, u\rangle = 0$, confirming orthogonality directly from the definition.

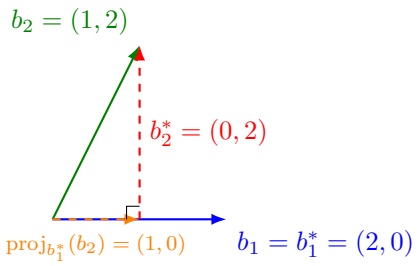


Figure 1.2: The geometry behind one Gram–Schmidt step (numbers match the worked example below). The green vector b_2 decomposes into an orange piece — its projection onto b_1^*, i.e. "how much of b_2 points along b_1^*" — and a red perpendicular leftover, $b_2^* = b_2 - \mathrm{proj}_{b_1^*}(b_2)$. That red leftover, translated back to the origin, *is* the new orthogonal basis vector. Every step of Gram–Schmidt is this same picture, repeated once per previous direction.

Definition: Gram–Schmidt Process

Given linearly independent $b_1, \ldots, b_k$, define $b_1^* = b_1$, and for $i = 2, \ldots, k$:

$$b_i^* = b_i - \sum_{j=1}^{i-1} \mu_{i,j}\, b_j^*, \qquad \text{where } \mu_{i,j} = \frac{\langle b_i, b_j^*\rangle}{\langle b_j^*, b_j^*\rangle}.$$

Each term $\mu_{i,j}\, b_j^*$ is precisely $\mathrm{proj}_{b_j^*}(b_i)$ from the defbox above — so this formula is nothing more than **"take b_i, and subtract off its projection onto every previously-built orthogonal direction, one at a time."** Each b_i^* is b_i with the "shadow" it casts on all previous b_j^* subtracted off, leaving only the genuinely new, orthogonal direction. The result $\{b_1^*, \ldots, b_k^*\}$ is an orthogonal basis of the same vector space (though, importantly, its integer combinations generally do *not* produce the same lattice as the original b_i's — Gram–Schmidt vectors are a vector-space tool, not automatically a lattice basis).

Example: Gram–Schmidt on a 2×2 Example

Let $b_1 = (2, 0)$, $b_2 = (1, 2)$ (the same vectors as Figure 1.2). Then $b_1^* = b_1 = (2, 0)$. Compute $\mu_{2,1} = \frac{\langle b_2, b_1^* \rangle}{\langle b_1^*, b_1^* \rangle} = \frac{(1)(2)+(2)(0)}{4} = \frac{2}{4} = 0.5$ — this is exactly $\text{proj}_{b_1^*}(b_2) = 0.5\,(2, 0) = (1, 0)$, the orange vector in the figure. So $b_2^* = b_2 - \text{proj}_{b_1^*}(b_2) = (1, 2) - (1, 0) = (0, 2)$ — the red leftover. Check: $\langle b_1^*, b_2^* \rangle = (2)(0) + (0)(2) = 0$ — orthogonal, as required.

Looking Ahead: Where Gram–Schmidt Leads

This process reappears, in a modified "integer-preserving" form, inside the **LLL algorithm** — the classic algorithm for turning a bad lattice basis into a reasonably good one, and a core tool used both to *attack* weak lattice-based schemes and to reason about the security of well-parametrized ones. We'll meet it properly in a later lecture; for now, just remember that Gram–Schmidt is the geometric heart of it.

Exercise: Practice

Run Gram–Schmidt on $b_1 = (1, 1)$, $b_2 = (0, 2)$: compute b_1^*, $\mu_{2,1}$, and b_2^*, and verify $\langle b_1^*, b_2^* \rangle = 0$.

1.8 Lattices: A Formal Definition

Everything so far has allowed *any* scalar from the field F (e.g., any real number) when forming linear combinations. Informally, a lattice is what happens when we impose one dramatic restriction: **only integer combinations are allowed**, which turns a solid, continuous vector space into a discrete grid of isolated points. That informal picture (Figure 1.4 below) is correct and worth keeping in your head. But the definition we will actually use — the one used in serious treatments of the subject, such as Chris Peikert's graduate lecture notes *Lattices in Cryptography*[1] — starts from the **subgroup** idea from Section 1.3.1, not from a basis. We will show the two views agree.

1.8.1 Discrete Sets

Definition: Discrete Subset

A subset $S \subseteq \mathbb{R}^n$ is **discrete** if every point of S is *isolated*: for every $x \in S$, there exists some $\varepsilon > 0$ such that the open ball $B(x, \varepsilon)$ (all points within distance ε of x) contains *no* point of S other than x itself. Informally: you can always draw a small enough circle around any point of S that catches nothing else from S.

Remark: A Shortcut for Subgroups

Checking every single point of a set for isolation sounds like a lot of work. But if H is a *subgroup* of $(\mathbb{R}^n, +)$ (Section 1.3.1), there's a shortcut: it suffices to check isolation *at the origin only* — H is discrete if and only if there is some $\varepsilon > 0$ with $B(\vec{0}, \varepsilon) \cap H = \{\vec{0}\}$. (This is a statement about how much *checking* is needed, not about how many points end up isolated: as the argument below shows, once the origin passes this test, *every* point of H turns out to be isolated too — not just the origin.) *Why this works:* suppose $\vec{0}$ is isolated with radius ε, and suppose some other point $y \in H$ were within ε of some $x \in H$. Since H is a subgroup, $y - x \in H$ as well, and $y - x$ is within ε of $\vec{0}$ — so by our assumption, $y - x = \vec{0}$, i.e., $y = x$. So checking one point (the origin) is enough to certify the whole subgroup is discrete, thanks to closure under subtraction.

[1] C. Peikert, *Lattices in Cryptography*, lecture notes, https://github.com/cpeikert/LatticesInCryptography.

1.8.2 The Definition

Definition: Lattice

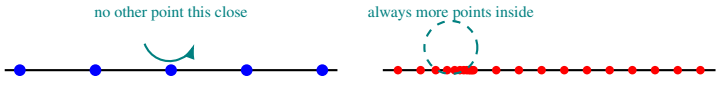
A **lattice** is a discrete additive subgroup L of $\mathbb{R}^n$. That is, $L \subseteq \mathbb{R}^n$ is a lattice if:

(L1) L is a subgroup of $(\mathbb{R}^n, +)$ (Section 1.3.1): it contains $\vec{0}$, and is closed under addition and negation.

(L2) L is discrete: by the shortcut above, there exists $\varepsilon > 0$ such that $B(\vec{0}, \varepsilon) \cap L = \{\vec{0}\}$.

Example: $\mathbb{Q}$ Is a Subgroup, But *Not* a Lattice

Consider $\mathbb{Q} \subseteq \mathbb{R}$ (the rational numbers) under addition. This *is* a subgroup: it contains 0, the sum of two rationals is rational, and $-a$ is rational whenever a is. So condition (L1) holds. But condition (L2) fails badly: the rationals are **dense** in $\mathbb{R}$, meaning every open ball around 0, no matter how tiny, contains infinitely many other rational numbers. There is no $\varepsilon > 0$ that isolates 0. So $\mathbb{Q}$ is a perfectly good subgroup that is **not** a lattice — this is exactly why condition (L2) has to be part of the definition; subgroup alone is not a strong enough requirement.



$\mathbb{Z}$ — discrete: points have gaps $\mathbb{Q}$ — dense: no gaps, ever

Figure 1.3: Discreteness, visualized. Left: in $\mathbb{Z}$, every point has a gap of size 1 around it with nothing else nearby — discrete. Right: in $\mathbb{Q}$, no matter how small a circle you draw around a point, there are always infinitely many more rational points crammed inside it — dense, and therefore *not* discrete.

Example: $\mathbb{Z}$ and $\mathbb{Z}^n$ Are Lattices

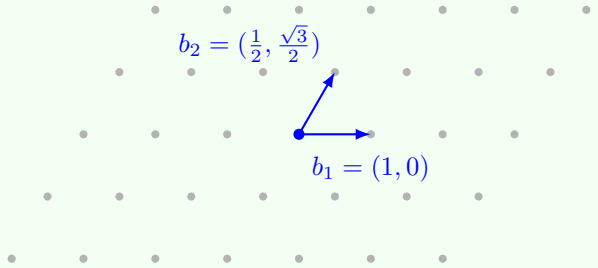
$\mathbb{Z} \subseteq \mathbb{R}$ is a subgroup (trivially — and recall from Section 1.3.1 that $n\mathbb{Z}$ is a subgroup of $\mathbb{Z}$ for any n; here we're just looking at $\mathbb{Z}$ itself sitting inside $\mathbb{R}$), and it is discrete: taking $\varepsilon = 0.5$, $B(0, 0.5) \cap \mathbb{Z} = \{0\}$, since the nearest other integers (± 1) are a full unit away. So $\mathbb{Z}$ is a lattice in $\mathbb{R}$ — the simplest possible example. The same argument, coordinate by coordinate, shows $\mathbb{Z}^n \subseteq \mathbb{R}^n$ (all integer-coordinate points) is a lattice in $\mathbb{R}^n$; it is often called **the integer lattice**.

Fact: Lattices Beyond Cryptography

Every example so far has been a rectangular grid, which can make "lattice" feel like a synonym for graph paper. It isn't — a lattice is any discrete, evenly-repeating set of points, and they show up constantly outside cryptography. A crystal (table salt, diamond, quartz) is, physically, atoms sitting at the points of a lattice in $\mathbb{R}^3$; the specific lattice shape determines the crystal's physical properties. The densest possible way to pack circles in the plane, or spheres in 8 and 24 dimensions (the famous E_8 and Leech lattices), is a lattice-packing question. Error-correcting codes used in deep-space communication have been built directly from lattices. None of this is needed for the rest of the course — it's just worth knowing that "lattice" is a genuinely fundamental geometric object, not a structure cryptographers invented for their own purposes.

Example: The Hexagonal Lattice: a Genuinely Non-Rectangular Example

Every lattice so far has had a basis of perpendicular (or near-perpendicular) vectors. That's not required. Let $b_1 = (1, 0)$ and $b_2 = \left(\frac{1}{2}, \frac{\sqrt{3}}{2}\right)$ in $\mathbb{R}^2$ — both length exactly 1, but at a $60°$ angle to each other, not $90°$. The resulting lattice $\mathcal{L}(b_1, b_2)$ tiles the plane with equilateral triangles instead of squares:



This is the **hexagonal (or triangular) lattice** — the densest possible way to pack circles in the plane, and the pattern honeycombs and hexagonal close-packed crystals actually use. Its shortest vector is easy to name by eye here: $\lambda_1(L) = 1$, achieved by b_1, b_2, and four other neighbors of the origin (every point has exactly 6 nearest neighbors, all at distance 1 — try spotting them in the picture). Its determinant is $\det(L) = |\det \begin{pmatrix} 1 & 1/2 \\ 0 & \sqrt{3}/2 \end{pmatrix}| = \frac{\sqrt{3}}{2} \approx 0.866$. Compare this to $\mathbb{Z}^2$: same $\lambda_1 = 1$, but $\det(\mathbb{Z}^2) = 1$ — a strictly *larger* determinant for the *same* shortest-vector length. That's not a coincidence: packing points as densely as the geometry allows (small determinant) while keeping them as far apart as possible (large λ_1) is exactly what makes the hexagonal lattice optimal, and it's the same tension Minkowski's theorem (Section 1.8.7) quantifies in general.

1.8.3 The Shortest Vector and the Minimum Distance $\lambda_1(L)$

Before going further, it's worth naming one basic quantity attached to every lattice: how close together its points actually are. We will use this constantly for the rest of the lecture (and the rest of the course), so it earns a formal definition rather than staying an informal aside.

Definition: Shortest Vector and Minimum Distance $\lambda_1(L)$

Let $L \subseteq \mathbb{R}^n$ be a lattice. A **shortest vector** of L is a nonzero vector $v \in L$ whose norm is minimal among all nonzero lattice vectors. The length of a shortest vector is called the lattice's **minimum distance**, or **first successive minimum**, and is defined by

$$\lambda_1(L) = \min_{v \in L \setminus \{\vec{0}\}} \|v\|.$$

The subscript "1" is intentional. More generally, the **successive minima**

$$\lambda_1(L) \leq \lambda_2(L) \leq \cdots \leq \lambda_n(L)$$

describe the minimum lengths needed to obtain $1, 2, \ldots, n$ linearly independent lattice vectors, respectively. In particular, $\lambda_i(L)$ is the smallest radius r such that the ball

$$B_n(\vec{0}, r) = \{x \in \mathbb{R}^n : \|x\| \leq r\}$$

contains i linearly independent vectors of L.
Thus, $\lambda_1(L)$ is simply the length of a shortest nonzero lattice vector. In this course, we will only need $\lambda_1(L)$ until the optional **Minkowski's Second Theorem** box near the end of this section.

Remark: Why the Minimum Is Actually Attained (Not Just an Infimum)

For a general infinite set, "the smallest nonzero norm" might not literally be achieved by any single element (imagine values getting closer and closer to some limit without ever reaching it). This cannot happen for a lattice, precisely *because* L is discrete (Section 1.8.1): discreteness guarantees a gap $\varepsilon > 0$ around the origin with no other lattice points inside it, so only *finitely many* lattice points can lie in any bounded region — and the minimum of a norm over a finite set is always achieved. This is also exactly the first step of the "harder direction" proof sketch below (Section 1.8.4): $\lambda_1(L)$ isn't just a number we can talk about, it's a specific vector we can (in principle) point to.

Remark: Why $\lambda_1(L)$ Matters

Computing $\lambda_1(L)$ exactly — or even just estimating it well — for a lattice given only a (possibly bad) basis is called the **Shortest Vector Problem (SVP)**, and it is believed to be extremely hard once the dimension n is large. This hardness is one of the two or three core hard problems that lattice-based cryptography's security ultimately rests on; we will state SVP formally, and meet its hardness in practice, once we reach the LWE problem in the next lecture.

1.8.4 Recovering the Basis Picture

The subgroup definition is more general and more standard, but the "grid generated by a basis" picture you may already have in your head is not wrong — it is a theorem, not a definition, and it's worth seeing why.

Fact: Basis Representation of Lattices

Let $L \subseteq \mathbb{R}^n$. Then L is a lattice (in the sense of the subgroup definition above, i.e., a discrete additive subgroup) **if and only if** there exist linearly independent vectors $b_1, \ldots, b_k \in \mathbb{R}^n$ (with $k \leq n$) such that

$$L = \mathcal{L}(b_1, \ldots, b_k) = \left\{ \sum_{i=1}^{k} a_i b_i \;\middle|\; a_i \in \mathbb{Z} \right\}.$$

The vectors $b_1, \ldots, b_k$ are called a **basis** of L; the number k is the **rank** of L. If $k = n$ (the basis spans all of $\mathbb{R}^n$), L is called **full-rank** — this is the common case in cryptography.

Remark: Proof Idea (Sketch Only)

Easy direction ($\Leftarrow$): given linearly independent $b_1, \ldots, b_k$, the set of integer combinations is clearly a subgroup (adding two integer combinations, or negating one, gives another integer combination). Discreteness holds because the b_i point in genuinely independent directions: you cannot make $\sum a_i b_i$ arbitrarily close to $\vec{0}$ without every a_i being exactly 0 — linear independence rules out any "near-cancellation." (Making this fully rigorous uses the fact that the linear map $(a_1, \ldots, a_k) \mapsto \sum a_i b_i$ has a bounded inverse on its image.)
Harder direction ($\Rightarrow$): given only that L is a discrete subgroup, we must *construct* a basis. The idea is greedy and inductive: pick b_1 to be a shortest nonzero vector in L — exactly $\lambda_1(L)$ from the defbox above, which exists precisely *because* L is discrete (in a non-discrete set like $\mathbb{Q}$, there is no shortest nonzero element, since you can always find a smaller one). Then look at the part of L not lying on the line through b_1, and pick b_2 to be a shortest vector there, and so on. A careful argument (using discreteness at each step, and projecting onto the space orthogonal to the vectors already chosen) shows this process terminates with a valid basis after $k \leq n$ steps. The full details are carried out rigorously in Peikert's notes; we take the result as given here.

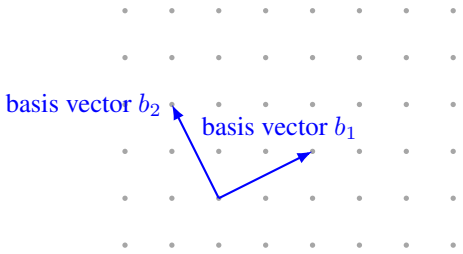


Figure 1.4: A 2-dimensional, full-rank lattice generated by a basis $\{b_1, b_2\}$: every grid dot is reachable by some *integer* combination $a_1 b_1 + a_2 b_2$ (with $a_1, a_2 \in \mathbb{Z}$), starting from any chosen origin dot. This is the same object as the subgroup definition above — just viewed through its basis instead of as an abstract subgroup.

Remark: Same Lattice, Different Basis

A single lattice has *many* different valid bases (just as a vector space does — recall the "Basis of $\mathbb{R}^2$" example in Section 1.7). Two bases generate the exact same set of points if and only if one can be converted to the other using an integer matrix with determinant ± 1 (a **unimodular** transformation) — we won't need the details of this fact yet, but the consequence is critical: some bases consist of short, nearly-perpendicular vectors (a "good" basis, easy to compute with), while other, equally valid bases for the *same* lattice consist of long, nearly-parallel vectors (a "bad" basis, hard to compute with). **This gap between good and bad bases for the same lattice is the single most important idea in lattice-based cryptography**: the secret key is typically a good (short) basis, while the public key describes the same lattice using a bad (long, scrambled-looking) basis. Recovering the good basis from the bad one is believed to be extremely hard once the dimension n is large enough.

Example: A Tiny Lattice by Hand

Let $b_1 = (2, 0)$ and $b_2 = (0, 3)$ in $\mathbb{R}^2$. The lattice $\mathcal{L}(b_1, b_2)$ consists of every point $(2a_1, 3a_2)$ for integers a_1, a_2 — so it includes $(0, 0)$, $(2, 0)$, $(-2, 0)$, $(0, 3)$, $(2, 3)$, $(4, -3)$, and so on, but *not* $(1, 1)$ or $(1, 0)$, since those cannot be written as $(2a_1, 3a_2)$ for integer a_1, a_2. As a sanity check against the subgroup definition directly: is this set a subgroup? Yes — closed under addition and negation, contains $(0, 0)$. Is it discrete? Yes — the nearest point to the origin is at distance 2, so any $\varepsilon < 2$ isolates $\vec{0}$. (Equivalently: $\lambda_1(L) = 2$, achieved by b_1 itself.)

Example: A Lattice With a Deliberately Bad Basis (the Running Example for This Section)

The lattice above had an obviously good basis — λ_1 was just sitting there, equal to one of the given vectors. That's not always true, and it's worth seeing a case where it fails, since "the shortest vector is hiding behind a combination of your basis vectors, not equal to any one of them" is precisely the situation real cryptographic bases are designed to create. Let $b'_1 = (2, 1)$ and $b'_2 = (5, 3)$. Check $|\det(B')| = |2 \times 3 - 5 \times 1| = 1$, so by the unimodular test (Section 1.8.6) this basis generates *exactly* $\mathbb{Z}^2$ — the same lattice as the standard basis $\{(1, 0), (0, 1)\}$, with $\lambda_1 = 1$. But look only at b'_1 and b'_2 themselves: $\|b'_1\| = \sqrt{5} \approx 2.24$ and $\|b'_2\| = \sqrt{34} \approx 5.83$ — neither one is anywhere near length 1, and a glance at this basis alone gives no hint that $\lambda_1(L) = 1$. The short vectors are still there, but only reachable through a non-obvious combination: direct substitution confirms

$$3b'_1 - b'_2 = 3(2, 1) - (5, 3) = (1, 0), \qquad -5b'_1 + 2b'_2 = -5(2, 1) + 2(5, 3) = (0, 1),$$

recovering the standard basis — and $\lambda_1 = 1$ — but only after combining the given vectors with coefficients as large as 5. **This is exactly what "bad basis" means**: not that the lattice is different or harder, only that its short vectors are algebraically disguised. Section 1.8.8 picks this exact example back up and shows an algorithm that undisguises it automatically.

1.8.5 The Determinant (Volume) of a Lattice

One more piece of structure will matter a great deal in later lectures: every full-rank lattice has a well-defined "volume," independent of which basis is used to describe it.

Definition: Fundamental Domain and Determinant

Let $L \subseteq \mathbb{R}^n$ be a full-rank lattice with basis $b_1, \ldots, b_n$, and let B be the matrix whose columns are $b_1, \ldots, b_n$. The **fundamental parallelepiped** associated with this basis is

$$\mathcal{P}(B) = \left\{ \sum_{i=1}^{n} t_i b_i \;\middle|\; 0 \le t_i < 1 \right\}.$$

Here, each coefficient t_i can be *any real number* in the interval $[0, 1)$. Thus, $\mathcal{P}(B)$ consists of all fractional linear combinations of the basis vectors, not merely combinations using coefficients 0 or 1. The **determinant** of the lattice L, denoted by $\det(L)$, is the volume of its fundamental parallelepiped:

$$\det(L) = \mathrm{vol}(\mathcal{P}(B)) = |\det(B)|.$$

The fundamental parallelepiped can be viewed as a basic "tile" for the entire space. Copies of $\mathcal{P}(B)$, shifted by every lattice point $\ell \in L$, tile all of $\mathbb{R}^n$ *exactly*: every point of $\mathbb{R}^n$ lies in *one and only one* translated copy, with no overlap at all. (This is precisely why the definition uses the half-open condition $0 \le t_i < 1$ rather than $0 \le t_i \le 1$: it guarantees each point has a unique representative $p \in \mathcal{P}(B)$, avoiding double-counting along shared faces.) Formally,

$$\mathbb{R}^n = \bigsqcup_{\ell \in L} (\ell + \mathcal{P}(B)),$$

where $\bigsqcup$ denotes a disjoint union. Here, **shifting** (or **translating**) $\mathcal{P}(B)$ by a lattice point ℓ simply means adding ℓ to every point in the parallelepiped:

$$\ell + \mathcal{P}(B) = \{\ell + x \mid x \in \mathcal{P}(B)\}.$$

The shape and volume do not change; only the location changes.

Example: A non-standard lattice in $\mathbb{R}^2$. Consider the lattice

$$L = \{mb_1 + nb_2 \mid m, n \in \mathbb{Z}\}$$

with basis

$$b_1 = (1, 0), \qquad b_2 = \left(\tfrac{1}{2}, \tfrac{\sqrt{3}}{2}\right).$$

These vectors have length 1 and are separated by $60°$, so they generate the familiar *triangular* (or hexagonal) lattice.

Step 1: Write the definition. The fundamental parallelepiped for this basis is

$$\mathcal{P}(B) = \{t_1 b_1 + t_2 b_2 \mid 0 \le t_1 < 1,\ 0 \le t_2 < 1\}.$$

Here t_1, t_2 are real numbers in $[0, 1)$.

Step 2: Substitute the basis vectors. Substituting $b_1 = (1, 0)$ and $b_2 = \left(\frac{1}{2}, \frac{\sqrt{3}}{2}\right)$ gives

$$\begin{aligned} t_1 b_1 + t_2 b_2 &= t_1(1, 0) + t_2 \left(\tfrac{1}{2}, \tfrac{\sqrt{3}}{2}\right) \\ &= (t_1, 0) + \left(\tfrac{t_2}{2}, \tfrac{\sqrt{3}\, t_2}{2}\right) \\ &= \left(t_1 + \tfrac{t_2}{2},\ \tfrac{\sqrt{3}\, t_2}{2}\right). \end{aligned}$$

Therefore,

$$\mathcal{P}(B) = \left\{ \left(t_1 + \tfrac{t_2}{2},\ \tfrac{\sqrt{3}\, t_2}{2}\right) \;\middle|\; 0 \le t_1 < 1,\ 0 \le t_2 < 1 \right\}.$$

Step 3: Understand the shape geometrically. The set $\mathcal{P}(B)$ is a *parallelogram* with vertices at:

$$\begin{aligned} &t_1 = 0,\ t_2 = 0 \mapsto (0, 0), \\ &t_1 = 1,\ t_2 = 0 \mapsto (1, 0), \\ &t_1 = 0,\ t_2 = 1 \mapsto \left(\tfrac{1}{2}, \tfrac{\sqrt{3}}{2}\right), \\ &t_1 = 1,\ t_2 = 1 \mapsto \left(\tfrac{3}{2}, \tfrac{\sqrt{3}}{2}\right). \end{aligned}$$

So $\mathcal{P}(B)$ is the parallelogram spanned by b_1 and b_2, with the top edge excluded (because $t_2 = 1$ is not allowed) and the right edge excluded (because $t_1 = 1$ is not allowed).

Step 4: Check some example points inside. Since t_1, t_2 are real, we can pick non-integer values:

$$0.3b_1 + 0.7b_2 = 0.3(1, 0) + 0.7\left(\tfrac{1}{2}, \tfrac{\sqrt{3}}{2}\right) = (0.3 + 0.35,\ 0.35\sqrt{3}) = (0.65,\ 0.35\sqrt{3}).$$

This point lies strictly inside the parallelogram. Similarly,

$$0b_1 + 0b_2 = (0, 0) \in \mathcal{P}(B)$$

lies on the bottom-left corner (included), but

$$1b_1 + 0b_2 = (1, 0) \notin \mathcal{P}(B), \qquad 0b_1 + 1b_2 = \left(\tfrac{1}{2}, \tfrac{\sqrt{3}}{2}\right) \notin \mathcal{P}(B)$$

because $t_1 = 1$ or $t_2 = 1$ is not allowed.

Step 5: Translate by a lattice point. Take the lattice point $\ell = b_1 + b_2 = \left(\frac{3}{2}, \frac{\sqrt{3}}{2}\right)$. Then

$$\ell + \mathcal{P}(B) = \left(\tfrac{3}{2} + t_1 + \tfrac{t_2}{2},\ \tfrac{\sqrt{3}}{2} + \tfrac{\sqrt{3}t_2}{2}\right), \quad 0 \le t_1, t_2 < 1.$$

This is the parallelogram with vertices:

$$\left(\tfrac{3}{2}, \tfrac{\sqrt{3}}{2}\right),\ \left(\tfrac{5}{2}, \tfrac{\sqrt{3}}{2}\right),\ \left(2, \sqrt{3}\right),\ \left(3, \sqrt{3}\right).$$

So it is the original parallelogram shifted by $b_1 + b_2$.

For example, the point

$$0.3b_1 + 0.7b_2 = (0.65,\ 0.35\sqrt{3})$$

maps to

$$\ell + (0.3b_1 + 0.7b_2) = \left(\tfrac{3}{2} + 0.65,\ \tfrac{\sqrt{3}}{2} + 0.35\sqrt{3}\right) = (2.15,\ 0.85\sqrt{3}).$$

Step 6: Disjointness of translates. The original parallelogram $\mathcal{P}(B)$ and its translate by ℓ are disjoint: they share only boundary points (which are excluded by the half-open convention). In fact, the entire plane is tiled by these parallelograms:

$$\mathbb{R}^2 = \bigcup_{\ell \in L} (\ell + \mathcal{P}(B)),$$

with the union being disjoint.

To see this visually, imagine placing copies of this parallelogram so that their bottom-left corners are at every lattice point. They fit together perfectly because b_1 and b_2 are not parallel and their lengths/angles are chosen so that the parallelograms interlock without gaps.

Step 7: Unique decomposition. Every point $x \in \mathbb{R}^2$ can be written *uniquely* as

$$x = \ell + p, \qquad \ell \in L, \quad p \in \mathcal{P}(B).$$

For example, take $x = (2.15,\ 0.85\sqrt{3})$. Then:

$$x = \left(\tfrac{3}{2}, \tfrac{\sqrt{3}}{2}\right) + (0.65,\ 0.35\sqrt{3}),$$

so $\ell = b_1 + b_2$ and $p = 0.3b_1 + 0.7b_2$.

Step 8: Volume (area) and determinant. The area of the parallelogram spanned by b_1 and b_2 is

$$\text{Area}(\mathcal{P}(B)) = |\det(b_1, b_2)| = \left|\det \begin{pmatrix} 1 & \frac{1}{2} \\ 0 & \frac{\sqrt{3}}{2} \end{pmatrix}\right| = \left|1 \cdot \tfrac{\sqrt{3}}{2} - \tfrac{1}{2} \cdot 0\right| = \tfrac{\sqrt{3}}{2}.$$

This is the area of a rhombus with side length 1 and angle 60°.

The determinant of the lattice is therefore

$$\det(L) = |\det(B)| = \frac{\sqrt{3}}{2}.$$

Thus,

$$\boxed{\det(L) = |\det(B)| = \frac{\sqrt{3}}{2} = \text{Area}(\mathcal{P}(B)).}$$

Key takeaway: The fundamental parallelepiped is the parallelogram spanned by the basis vectors. Its shape and area come directly from the basis, and translating it by all lattice points tiles the entire plane without overlap. The determinant of the lattice is exactly the area of this fundamental cell.

Remark: Why $\det(L)$ Doesn't Depend on Which Basis You Pick

Recall from the "Same Lattice, Different Basis" remark above: any two bases of the same lattice are related by a unimodular matrix U (integer entries, $\det(U) = \pm 1$), i.e., $B' = UB$. Then $\det(B') = \det(U)\det(B) = \pm\det(B)$, so $|\det(B')| = |\det(B)|$. This confirms $\det(L)$ is a genuine property of the lattice itself, not an artifact of which basis happened to be used to compute it — exactly the kind of basis-independence you should expect from something called "the volume of the lattice."

Example: Computing $\det(L)$ for the Tiny Lattice

For $b_1 = (2, 0)$, $b_2 = (0, 3)$ from the example above, $B = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}$, so $\det(L) = |\det(B)| = |2 \times 3 - 0 \times 0| = 6$. Geometrically: the fundamental parallelepiped is just the 2×3 rectangle $[0, 2) \times [0, 3)$, and copies of this rectangle exactly tile the plane, anchored at every lattice point — matching the picture in Figure 1.4's style, just for this specific (axis-aligned) lattice.

1.8.6 Unimodular Equivalence: When Do Two Bases Give the Same Lattice?

Example: Same Lattice, or Different? A Worked Test Using the Unimodular Criterion

Let $B = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}$ (columns $b_1 = (2,1), b_2 = (1,1)$) and $B' = \begin{pmatrix} 1 & 3 \\ 0 & 1 \end{pmatrix}$ (columns $b_1' = (1,0), b_2' = (3,1)$) be two candidate bases in $\mathbb{R}^2$. Do they generate the same lattice?

Confirming case. Suppose instead $B' = \begin{pmatrix} 3 & 1 \\ 2 & 1 \end{pmatrix}$. Since B is invertible ($\det B = 2 - 1 = 1 \neq 0$), solve $U = B^{-1}B'$: $B^{-1} = \begin{pmatrix} 1 & -1 \\ -1 & 2 \end{pmatrix}$ (check: $BB^{-1} = I$). Then

$$U = B^{-1}B' = \begin{pmatrix} 1 & -1 \\ -1 & 2 \end{pmatrix} \begin{pmatrix} 3 & 1 \\ 2 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}.$$

U has integer entries and $\det(U) = 1 \times 1 - 0 \times 1 = 1$, so $|\det(U)| = 1$ — U **is unimodular**. Since $B' = BU$ with U unimodular, B and B' generate **the same lattice**.

Contrasting case. Now go back to the original $B' = \begin{pmatrix} 1 & 3 \\ 0 & 1 \end{pmatrix}$. Compute $U = B^{-1}B'$:

$$U = \begin{pmatrix} 1 & -1 \\ -1 & 2 \end{pmatrix} \begin{pmatrix} 1 & 3 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 2 \\ -1 & -1 \end{pmatrix}.$$

Still integer entries, but $\det(U) = (1)(-1) - (2)(-1) = -1 + 2 = 1$ — wait, this is also unimodular; let's instead deliberately pick a genuinely different lattice to illustrate the failing case: $B'' = \begin{pmatrix} 1 & 3 \\ 0 & 2 \end{pmatrix}$. Then

$$U = B^{-1}B'' = \begin{pmatrix} 1 & -1 \\ -1 & 2 \end{pmatrix} \begin{pmatrix} 1 & 3 \\ 0 & 2 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix},$$

with $\det(U) = (1)(1) - (1)(-1) = 2$. Since $|\det(U)| = 2 \neq 1$, U is **not** unimodular — B and B'' generate *different* lattices, and in fact $\mathcal{L}(B'')$ turns out to be a proper sublattice of $\mathcal{L}(B)$ of **index** 2 (every point of $\mathcal{L}(B'')$ is a point of $\mathcal{L}(B)$, but only half of $\mathcal{L}(B)$'s points are recovered this way) — a direct consequence of $|\det(U)|$ measuring exactly this kind of "index" between the two lattices, matching $\det(L) = |\det(B)|$ scaling by the same factor: $\det(\mathcal{L}(B'')) = |\det(U)| \cdot \det(\mathcal{L}(B)) = 2\det(\mathcal{L}(B))$.

Remark: Discrete Implies Countable, But Not Conversely

It's worth clarifying a subtle point about the discreteness condition (L2) in the Lattice definition: **every discrete subset of $\mathbb{R}^n$ is automatically countable**, but the converse is false — being countable does not force a set to be discrete. *Why discrete $\Rightarrow$ countable:* around each point of a discrete set, by definition, sits a small ball containing no other point of the set; these disjoint balls can be shrunk enough to have rational-coordinate centers and rational radii, and there are only countably many such balls in total (since $\mathbb{Q}^n$ is countable) — so there can be at most countably many points of the discrete set, one per ball. *Why the converse fails:* $\mathbb{Q} \subseteq \mathbb{R}$ is a textbook counterexample — it is countable (the rationals can be listed $q_1, q_2, q_3, \ldots$), yet, as the exbox above on "$\mathbb{Q}$ Is a Subgroup, But Not a Lattice" already showed, $\mathbb{Q}$ is *dense* in $\mathbb{R}$, not discrete: every open ball around any point contains infinitely many other rational points. So countability is a necessary, but nowhere near sufficient, condition for discreteness — exactly why the Lattice definition needs the stronger, explicitly geometric condition (L2), rather than settling for the weaker "countable" in its place.

1.8.7 Minkowski's First Theorem: Volume Forces a Short Vector to Exist

We now have the two basic numbers attached to any lattice: $\lambda_1(L)$ (how short its shortest vector is) and $\det(L)$ (how much volume, on average, surrounds each lattice point). Minkowski's First Theorem is the single most important fact connecting them — and it flips the usual question around. So far, "finding $\lambda_1(L)$" has meant: given a specific basis, go and search for a short vector. Minkowski's theorem asks instead: *without searching at all*, can we predict, from $\det(L)$ alone, an upper bound on how long $\lambda_1(L)$ is *allowed* to be? Surprisingly, yes — and, remarkably, the argument is pure geometry, with no algebra and no knowledge of any particular basis required.

The theorem is about regions $K \subseteq \mathbb{R}^n$ with two specific shape properties, **symmetric** and **convex**, both used informally already in this course but worth pinning down precisely before they appear inside a theorem statement.

Definition: Symmetric Region (Origin-Symmetric / Centrally Symmetric)

A region $K \subseteq \mathbb{R}^n$ is **symmetric** (short for *origin-symmetric*, sometimes called *centrally symmetric* or *point-symmetric*) if

$$K = -K, \qquad \text{i.e.,} \qquad x \in K \implies -x \in K.$$

In words: whenever a point x belongs to K, its reflection through the origin, $-x$, must belong to K too. Informally, K "looks the same" if you rotate it $180°$ about the origin — a disk or square *centered at the origin* is symmetric in this sense, but the same disk or square shifted off-center is not (its reflection through the origin lands somewhere else entirely).

Definition: Convex Region

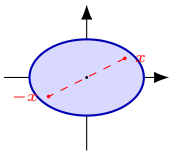
A region $K \subseteq \mathbb{R}^n$ is **convex** if, for every pair of points $x, y \in K$, the entire straight line segment joining them,

$$\{\, tx + (1-t)y \;:\; t \in [0,1] \,\},$$

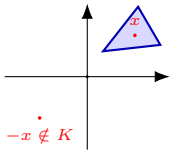
also lies inside K. Informally: a convex region has no "dents," "notches," or "holes" — you can never draw a straight line between two points of K that has to leave K and come back. Disks, squares, triangles, and ellipses are all convex; rings (annuli), crescents, and star shapes generally are not.

Remark: The Two Properties Are Independent

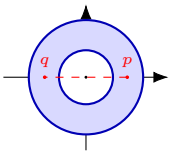
Symmetric and convex describe two *different* things about a region's shape — one is about "does it look the same reflected through the origin," the other is about "does it have any dents or holes" — so a region can have either property without the other, both, or neither. Minkowski's theorem needs *both at once*: Figure 1.5 below shows one example of each of the four possible combinations.



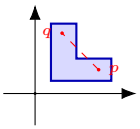
(a) Symmetric *and* convex — e.g. a disk or ellipse centered at the origin.



(b) Convex, but *not* symmetric — $x \in K$ but $-x \notin K$.



(c) Symmetric, but *not* convex — the ring is unchanged by $x \mapsto -x$, but the segment pq cuts through the empty hole.



(d) Neither symmetric nor convex — an off-center, notched ("L-shaped") region.

Figure 1.5: The four combinations of symmetric / not, convex / not. In (a), every point's reflection $-x$ is also in K, and every segment between two points of K stays inside K — this is the shape Minkowski's theorem needs. In (b), the triangle sits entirely in the upper-right, so a point $x \in K$ has its reflection $-x$ landing in the empty lower-left — symmetry fails, even though the triangle itself has no dents. In (c), the ring *is* symmetric (it looks the same rotated $180°$), but it fails convexity: the straight segment between the two marked points p, q passes through the hollow center, which is not part of the ring. In (d), the region is both off-center (so $x \mapsto -x$ maps points outside K) and has an inward notch (so some segments between points of K leave K), failing both properties at once.

Remark: The Idea in One Sentence, Before Any Symbols

If a symmetric, convex region around the origin is made large enough relative to $\det(L)$, it becomes *geometrically impossible* for that region to avoid every nonzero lattice point — so one must be hiding inside it, whether or not you know where.

Definition: Minkowski's First Theorem

Let $L \subseteq \mathbb{R}^n$ be a full-rank lattice, and let $K \subseteq \mathbb{R}^n$ be a **symmetric** ($K = -K$, i.e. $x \in K \Rightarrow -x \in K$) **convex** region. If

$$\mathrm{Vol}(K) \;>\; 2^n \det(L),$$

then K contains a nonzero lattice point, i.e. $K \cap (L \setminus \{\vec{0}\}) \neq \varnothing$.

Remark: Reading the Theorem in Plain Language, Term by Term

- $\det(L)$, as in Section 1.8.5, is "how much empty space, on average, surrounds each lattice point."
- $2^n \det(L)$ is a fixed volume threshold, scaled up from $\det(L)$ by a factor that grows with the dimension n (we will see exactly where this factor comes from in the proof below).
- **Symmetric and convex** are conditions on the *shape* of K — a disk or an ellipsoid centered at the origin both qualify; an oddly-shaped or off-center region generally does not.
- **The conclusion** is purely existential: once K's volume clears the threshold, K is *guaranteed* to contain some nonzero point of L — but the theorem does not say which point, or how to find it. That "existence without construction" flavor is typical of the deep facts underlying lattice cryptography, and it's worth getting comfortable with that style of guarantee now.

Remark: A Loose End First: Does K Always Contain the Origin?

Before turning to the proof, it's worth settling a question the theorem's statement doesn't address directly: could a symmetric convex region fail to contain the origin at all? No — and this is worth proving explicitly, because it removes a genuine ambiguity later.
Claim: any nonempty symmetric convex set K contains $\vec{0}$. Proof: since K is nonempty, pick any point $x \in K$. By symmetry ($K = -K$), $-x \in K$ too. By convexity, the line segment joining x and $-x$ lies entirely in K — in particular its midpoint, $\frac{1}{2}(x + (-x)) = \vec{0}$, is in K. ■
So $\vec{0} \in K$ is automatic, for *every* symmetric convex K, regardless of its size — it has nothing to do with the volume threshold $2^n \det(L)$ in the theorem. The origin is never at risk of being excluded, so the theorem's volume condition can't be about keeping the origin in; it must be doing something else. That something else is the theorem's real content: guaranteeing a *second*, *nonzero* lattice point, which is not automatic, and genuinely does depend on K being large enough.

A Fully Formal Proof, via Blichfeldt's Lemma

Remark: Proof Roadmap

The proof splits cleanly into two independent pieces, and it helps to see the two-step strategy before working through either one in detail:

1. **Blichfeldt's Lemma** (a pure volume/pigeonhole argument): *any* region with volume bigger than $\det(L)$ — symmetric, convex, or neither, it doesn't matter — must contain two distinct points whose difference is a nonzero lattice vector. This step never looks at the shape of the region at all.
2. **Bringing back symmetry and convexity**: apply Blichfeldt's Lemma not to K itself, but to the *half-sized* copy $S = \frac{1}{2}K$. The difference of the two points Blichfeldt hands us then turns out, *because* K is symmetric and convex, to land back inside K itself — which is exactly the nonzero lattice point in K that the theorem promises. This is also where the factor of 2^n in the theorem's statement comes from: shrinking K down to $S = \frac{1}{2}K$ divides its volume by 2^n, so S only clears Blichfeldt's threshold ($\mathrm{Vol}(S) > \det(L)$) once $\mathrm{Vol}(K) > 2^n \det(L)$ to begin with.

Fact: Blichfeldt's Lemma

Let $L \subseteq \mathbb{R}^n$ be a full-rank lattice with fundamental domain $\mathcal{P} = \mathcal{P}(B)$ (Section 1.8.5), so $\mathrm{Vol}(\mathcal{P}) = \det(L)$. Let $S \subseteq \mathbb{R}^n$ be any measurable set with $\mathrm{Vol}(S) > \det(L)$. Then there exist two **distinct** points $p, q \in S$ such that $p - q \in L \setminus \{\vec{0}\}$.

Remark: Proof of Blichfeldt's Lemma

Step 1: Cut S into pieces, one per lattice point, and fold every piece back to the origin. For each lattice point $v \in L$, consider the piece of S that lands in the copy of the fundamental domain sitting at v, i.e. $S \cap (v + \mathcal{P})$, and **fold it back** to the origin's copy by subtracting v:

$$S_v \;=\; \big(S \cap (v + \mathcal{P})\big) - v \;\subseteq\; \mathcal{P}.$$

Step 2: The folded pieces' volumes must add up to more than $\mathrm{Vol}(\mathcal{P})$. Because the translates $\{v + \mathcal{P} : v \in L\}$ tile $\mathbb{R}^n$ with no gaps or overlaps (Section 1.8.5), every point of S lies in *exactly one* $v + \mathcal{P}$, so the sets $S \cap (v + \mathcal{P})$, as v ranges over L, partition S exactly. Folding back (a rigid shift) doesn't change volume, so

$$\sum_{v \in L} \mathrm{Vol}(S_v) \;=\; \sum_{v \in L} \mathrm{Vol}\big(S \cap (v + \mathcal{P})\big) \;=\; \mathrm{Vol}(S) \;>\; \det(L) \;=\; \mathrm{Vol}(\mathcal{P}).$$

Step 3: Too much volume packed into one region forces an overlap (pigeonhole). All the S_v's live inside the single region $\mathcal{P}$. If they were all pairwise disjoint, the sum of their volumes could be at most $\mathrm{Vol}(\mathcal{P})$ — but Step 2 just showed that sum is *strictly greater* than $\mathrm{Vol}(\mathcal{P})$. Contradiction. So some two of them, S_{v_1} and S_{v_2} with $v_1 \neq v_2$, must overlap: there is a point $y \in S_{v_1} \cap S_{v_2}$.

Step 4: Unfold the overlap to recover two points of S whose difference is a lattice vector. By definition of S_v, $y = p - v_1$ for some $p \in S \cap (v_1 + \mathcal{P})$, and $y = q - v_2$ for some $q \in S \cap (v_2 + \mathcal{P})$. Both $p, q \in S$. Since $p - v_1 = q - v_2$, we get

$$p - q \;=\; v_1 - v_2 \;\in\; L \setminus \{\vec{0}\}$$

(nonzero because $v_1 \neq v_2$, and the difference of two lattice points is again a lattice point, by L's subgroup closure from Section 1.3.1). This is exactly the pair the lemma promised. ■

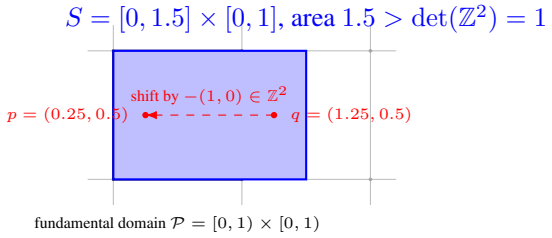


Figure 1.6: Blichfeldt's Lemma in action for $L = \mathbb{Z}^2$. S (blue) has area 1.5, exceeding $\det(L) = 1$, so it cannot fit inside one fundamental domain without "spilling over." The spillover — the sliver of S with $x \in [1, 1.5]$ — folds back (shift by $-(1, 0)$) into the same unit square as the rest of S, and lands on top of it: the two marked points $p, q \in S$ fold to the *same* location, and $p - q = (-1, 0)$ is a nonzero lattice vector, exactly as the lemma guarantees.

Example: Blichfeldt's Lemma, Checked by Hand

Take $L = \mathbb{Z}^2$ ($\det(L) = 1$) and $S = [0, 1.5] \times [0, 1]$, area $1.5 > 1$. Following the proof: the piece of S in the fundamental domain $\mathcal{P} = [0, 1) \times [0, 1)$ itself is all of $[0, 1) \times [0, 1)$ (area 1); the piece of S in the neighboring copy $v + \mathcal{P}$ for $v = (1, 0)$ is $[1, 1.5] \times [0, 1)$ (area 0.5), which folds back to $[0, 0.5) \times [0, 1)$ after subtracting v. These two folded pieces — area 1 and area 0.5, total $1.5 > 1 = \mathrm{Vol}(\mathcal{P})$ — cannot both fit disjointly inside $\mathcal{P}$, so they overlap on $[0, 0.5) \times [0, 1)$. Picking any point there, e.g. $(0.25, 0.5)$: this equals $p - (0, 0) = p$ for $p = (0.25, 0.5) \in S$, and also equals $q - (1, 0)$ for $q = (1.25, 0.5) \in S$. Indeed $p, q \in S$ (check: $0.25, 1.25 \in [0, 1.5]$, $0.5 \in [0, 1]$, both confirmed), and $p - q = (-1, 0) \in \mathbb{Z}^2 \setminus \{\vec{0}\}$ — exactly as guaranteed, and matching Figure 1.6.

Remark: Deriving Minkowski's Theorem from Blichfeldt's Lemma

Now bring back symmetry and convexity to finish the proof, following Step 2 of the roadmap above. Let K be symmetric and convex with $\mathrm{Vol}(K) > 2^n \det(L)$.

Set up the shrunk region. Define $S = \frac{1}{2}K = \{\frac{1}{2}x : x \in K\}$. Volume scales with the n-th power of a linear scale factor, so $\mathrm{Vol}(S) = \mathrm{Vol}(K)/2^n > \det(L)$ — exactly the hypothesis Blichfeldt's Lemma needs.

Apply Blichfeldt's Lemma to S. There exist distinct $p, q \in S$ with $p - q \in L \setminus \{\vec{0}\}$. Since $p, q \in S = \frac{1}{2}K$, write $p = \frac{1}{2}x$ and $q = \frac{1}{2}y$ for (unique) $x, y \in K$. Then

$$p - q \;=\; \tfrac{1}{2}(x - y) \;=\; \tfrac{1}{2}\big(x + (-y)\big).$$

Show this difference lands back inside K. Since K is symmetric, $-y \in K$. Since K is convex, the midpoint of x and $-y$ — which is exactly $\frac{1}{2}(x + (-y)) = p - q$ — lies in K. So $p - q$ is simultaneously:

- a nonzero lattice vector (from Blichfeldt's Lemma), and
- a point of K (from symmetry and convexity, just shown).

Therefore $p - q \in K \cap (L \setminus \{\vec{0}\})$ — a nonzero lattice point inside K, exactly as the theorem claims. ■

Remark: Tying the Two Halves Together

Notice how cleanly the labor divides. Blichfeldt's Lemma is the *engine*: pure volume-counting, no geometry of K's shape involved at all — it would work verbatim for *any* measurable S, symmetric or not, convex or not, as long as it's simply big enough. Symmetry and convexity only enter in the very last step, to upgrade the raw difference $p - q$ (which Blichfeldt hands you for free) into a point that's *also* guaranteed to lie inside K itself, not just somewhere in $\mathbb{R}^n$. This is worth remembering: it's the reason Minkowski's theorem needs *both* hypotheses (symmetric *and* convex) and not just "big enough" — drop either one, and the last step's midpoint trick breaks, even though Blichfeldt's Lemma itself still holds regardless.

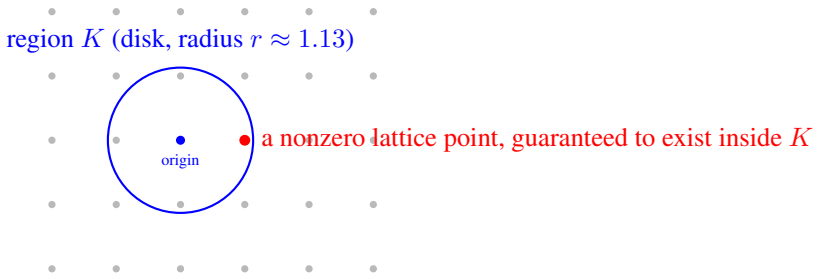


Figure 1.7: Minkowski's First Theorem for $L = \mathbb{Z}^2$ ($\det(L) = 1$): once the disk's area exceeds $2^2 \cdot \det(L) = 4$ (radius $r \gtrsim 1.13$), the theorem guarantees a nonzero lattice point lies inside it — and indeed $(1, 0)$ (shown here relative to the chosen origin) is inside, at distance exactly $1 < 1.13$.

Example: Minkowski's Bound on the Simplest Lattice, $L = \mathbb{Z}^2$

Take $L = \mathbb{Z}^2$, so $\det(L) = 1$ (Section 1.8.5). Let K be a disk of radius r centered at the origin — symmetric and convex, exactly as the theorem requires. The theorem applies once $\mathrm{Vol}(K) = \pi r^2 > 2^2 \cdot \det(L) = 4$, i.e. once

$$r > \frac{2}{\sqrt{\pi}} \approx 1.128.$$

So Minkowski's theorem *guarantees*, without any further work, that a disk of radius 1.13 around the origin must contain a nonzero lattice point. Checking directly: we already know $\lambda_1(\mathbb{Z}^2) = 1$ (the point $(1, 0)$), and indeed $1 < 1.128$ — consistent with, and in fact comfortably inside, the guarantee. Notice the theorem didn't need to know that $(1, 0)$ was the answer; it only needed the disk to be big enough.

Example: The Same Bound on a Very Different-Looking Lattice

Now take a stretched lattice with basis $b_1 = (5, 0)$, $b_2 = (0, 1)$, so $\det(L) = |5 \times 1 - 0 \times 0| = 5$ (Section 1.8.5). The disk threshold becomes $\pi r^2 > 4 \times 5 = 20$, i.e. $r > \sqrt{20/\pi} \approx 2.523$. Minkowski's theorem guarantees a nonzero lattice point within distance 2.523 of the origin. The actual shortest vector here is $(0, 1)$, of length 1 — again comfortably inside the guaranteed radius, but this time the guarantee is quite *loose*: the theorem promised "within 2.523" and reality delivered "within 1." This is expected and normal — Minkowski's theorem is a **worst-case guarantee**, not an exact formula; it never overpromises (the bound always holds), but it can certainly underdeliver on precision for any one specific lattice. Its real value is as a *universal* ceiling that holds for *every* lattice of a given determinant, not as a precise estimate for one.

Remark: From the Disk Bound to the General Formula

Redo the disk calculation above in n dimensions instead of 2: the volume of a radius-r ball in $\mathbb{R}^n$ shrinks (relative to r^n) as n grows, and pushing the same threshold argument through (we omit the n-dimensional volume formula itself, which involves the Gamma function) yields the general bound already stated in Lecture 1's introduction to this topic:

$$\lambda_1(L) \;\leq\; \sqrt{n} \cdot \det(L)^{1/n} \qquad \text{(up to a small constant factor).}$$

In plain terms: a lattice can never "hide" an unexpectedly long shortest vector behind a small determinant — volume and minimum distance are locked together by geometry alone, regardless of which basis you're handed to describe the lattice. This is part of *why* SVP is only believed hard in the specific regime cryptographers deliberately choose (large dimension n, carefully scaled $\det(L)$), rather than for arbitrary lattices — a preview of the parameter-selection discussion we'll return to once we look at how Kyber and Dilithium choose their concrete dimensions and moduli.

Exercise: Practice

Let $L = \mathbb{Z}^3$ (so $\det(L) = 1$). Using the volume of a ball in $\mathbb{R}^3$, $\text{Vol} = \frac{4}{3}\pi r^3$, find the radius r at which Minkowski's theorem first guarantees a nonzero lattice point inside the ball. Compare it to the true value $\lambda_1(\mathbb{Z}^3) = 1$ (e.g. the point $(1, 0, 0)$) — is the guarantee looser or tighter, relative to the truth, than it was for $\mathbb{Z}^2$ in the worked example above?

Exercise: Practice: Same Lattice, or Different?

For each pair of bases below (columns are the basis vectors), form $U = B^{-1}B'$, check whether U has integer entries with $|\det(U)| = 1$, and conclude whether the two bases generate the same lattice. Cross-check by also comparing $|\det(B)|$ and $|\det(B')|$ directly.

(a) $B = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$, $B' = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$.

(b) $B = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}$, $B' = \begin{pmatrix} 1 & 3 \\ 1 & 4 \end{pmatrix}$.

(c) $B = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$, $B' = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$.

1.8.8 The LLL Algorithm: Turning a Bad Basis Into a Good One

Remark: The Question You Should Be Asking

Section 1.8.4's bad-basis example needed an integer combination with a coefficient as large as 5 just to uncover a length-1 vector. A natural question follows immediately: if a *good* basis makes $\lambda_1(L)$ obvious, is there an efficient algorithm that takes a bad basis and hands back a good one — effectively solving SVP? The answer is yes and no, and the precise shape of that "yes and no" is one of the most important facts in this entire course.

Definition: The LLL Algorithm (Lenstra–Lenstra–Lovász, 1982)

Given any basis $b_1, \ldots, b_n$ of a lattice L, the LLL algorithm runs in time polynomial in n and the size of the input, and outputs a new basis $b'_1, \ldots, b'_n$ of the *same* lattice L that is **reduced**: informally, each b'_i is not too much longer than it needs to be, and the whole basis is not too far from orthogonal. The output's first vector satisfies a guarantee of the form

$$\|b'_1\| \;\leq\; 2^{(n-1)/2}\,\lambda_1(L).$$

Remark: Reading the Guarantee Two Ways

The good news: LLL runs in polynomial time and *always* finds *some* genuinely short vector, for *any* lattice, with no restriction at all — a remarkable, unconditional guarantee, and exactly why LLL is a standard tool in every computational algebra system. **The catch:** the approximation factor $2^{(n-1)/2}$ grows *exponentially* with the dimension n. For $n = 10$ that's already a factor of about 16; for $n = 100$ (nowhere near cryptographic scale) it's a number with about 15 digits. LLL is not solving SVP — it's solving a version of SVP with the approximation gap γ thrown wide open, and cryptographic hardness (the polynomial-γ regime Lecture 2 builds SIS and LWE on top of) lives nowhere near that wide-open end.

Example: LLL Reduction by Hand, on This Section's Running Example

Recall $b'_1 = (2,1), b'_2 = (5,3)$ from Section 1.8.4, generating $\mathbb{Z}^2$. LLL's core move, repeated until nothing changes: use Gram–Schmidt (Section 1.7.3) to measure the current basis, *shrink* each vector by rounding off an integer multiple of the previous one, and *swap* two vectors whenever that shrinks the basis. Watch it happen:

Step 1 — measure. $b_1^* = b'_1 = (2,1)$, $\|b_1^*\|^2 = 5$. $\mu_{2,1} = \dfrac{\langle b'_2, b_1^*\rangle}{\|b_1^*\|^2} = \dfrac{5(2)+3(1)}{5} = \dfrac{13}{5} = 2.6$.

Step 2 — shrink (size-reduce). Round 2.6 to 3 and subtract: $b''_2 = b'_2 - 3b'_1 = (5,3) - (6,3) = (-1,0)$. New basis: $\{(2,1), (-1,0)\}$.

Step 3 — swap. $\|(-1,0)\| = 1$ is far shorter than $\|(2,1)\| = \sqrt{5}$ — exactly the situation that makes LLL swap the two vectors to keep the basis ordered short-to-long. New basis: $\{(-1,0), (2,1)\}$.

Step 4 — measure and shrink again. $b_1^* = (-1,0)$, $\|b_1^*\|^2 = 1$. $\mu_{2,1} = \dfrac{(2)(-1)+(1)(0)}{1} = -2$. Subtract: $(2,1) - (-2)(-1,0) = (2,1) - (2,0) = (0,1)$.

Result. $\{(-1,0), (0,1)\}$ — length 1 each, perfectly orthogonal. Up to sign, this *is* the standard basis, and $\lambda_1 = 1$ is now unmissable. Four short, mechanical steps recovered exactly what Section 1.8.4 had to find by guessing an integer coefficient as large as 5.

Remark: Why the $n = 2$ Case Feels Like It Solves SVP — Because It Basically Does

The worked example above didn't just find *a* short vector; it found *the* shortest one, exactly. That's not a fluke of these particular numbers: in dimension $n = 2$, LLL (really a much older algorithm, Gauss–Lagrange reduction) is provably *exact* — it always outputs a genuine shortest vector, not merely an approximation. The reason traces directly to the reduction step's structure: LLL's swap-and-shrink rule only ever compares *adjacent* vectors, b_i against b_{i+1}. With only two vectors total, there is exactly one adjacent pair — so "locally reduced" (what LLL checks) and "globally shortest" (what SVP asks for) are the same statement. Once $n \geq 3$, this stops being true: LLL still only directly compares neighboring pairs, and a long chain of locally-fine relationships can still hide a globally short vector nowhere near where any single local check would find it. That gap, compounding across $n - 1$ pairwise comparisons, is exactly where the $2^{(n-1)/2}$ factor comes from.

Fact: BKZ: The One Knob LLL Doesn't Have

If LLL's weakness is that it only ever looks at pairs, the natural fix is to look at *more than pairs* — and that is exactly what **BKZ** (Blockwise Korkine–Zolotarev) does. BKZ takes a **block size** k as a parameter and, instead of reducing one vector against its immediate neighbor, solves SVP *exactly* within each sliding window of k consecutive basis vectors, then moves the window along. Two extremes make the whole picture click into place:

- $k = 2$**:** each window is just a pair — BKZ collapses to exactly LLL. Polynomial time, $2^{(n-1)/2}$-style approximation.
- $k = n$**:** the single window is the *whole* basis — BKZ becomes exhaustive enumeration, provably exact ($\gamma = 1$), at a cost that grows exponentially in n.

Every value of k in between is a genuine trade-off point: larger k buys a better approximation factor at the cost of more time per window, and real attacks against lattice cryptography are, in large part, a search for the largest block size k that stays computationally affordable against a given parameter set. **This is the direct answer to "how do you change LLL to get closer to the real shortest vector":** you don't change LLL's swap-and-shrink rule at all — you widen how many vectors it's allowed to look at during each local comparison, sliding along the entire spectrum from LLL's efficient-but-loose end to enumeration's exact-but-slow end, with no known algorithm reaching *both* efficient *and* exact at once, at cryptographic dimensions.

Looking Ahead: Optional / Advanced: Minkowski's Second Theorem

The First Theorem only talks about $\lambda_1(L)$, the *single* shortest vector. It says nothing about the second-shortest independent direction, the third, and so on. **Minkowski's Second Theorem** fills that gap: recall the **successive minima** $\lambda_1(L) \leq \lambda_2(L) \leq \cdots \leq \lambda_n(L)$ flagged in the defbox above, where $\lambda_i(L)$ is the smallest radius r such that the ball of radius r contains i *linearly independent* lattice vectors. The theorem bounds their *product*:

$$\frac{2^n}{n!}\det(L) \;\leq\; \lambda_1(L)\cdots\lambda_n(L) \;\leq\; 2^n \det(L).$$

This is not core material — everything in Lessons 1–6 (understanding LWE, the algorithms, and the physical attacks in Section 2 of the proposal) only ever leans on the First Theorem's intuition: "small determinant ⇒ short vector must exist." The Second Theorem becomes genuinely useful only once the research moves toward analyzing *trapdoor* lattices and Gaussian sampling in depth — which is exactly what underlies Falcon's signing procedure (flagged for Lesson 6). A trapdoor basis needs *all* of its vectors short, not just the shortest one, for Gaussian sampling to be both efficient and secure; that "all vectors short together" requirement is precisely what the successive minima $\lambda_1, \ldots, \lambda_n$ capture, and the Second Theorem is the tool that ties them back to $\det(L)$. This box is worth returning to *if and only if* the Phase 2 research direction (Section 4 of the proposal) ends up involving Falcon's sampler or trapdoor generation in detail — otherwise, the First Theorem alone is sufficient background.

Exercise: Practice

For the lattice generated by $b_1 = (1, 2)$, $b_2 = (3, 1)$ (from the earlier homework exercise): compute $\det(L)$ directly from the basis matrix. Then pick any *other* valid basis for the same lattice (e.g., $b_1' = b_1$, $b_2' = b_2 - b_1$) and confirm you get the same $\det(L)$.

Looking Ahead: Where This Leads Next Lecture

With Groups → Subgroups → Rings → Fields → Modular arithmetic → Vector spaces → Basis → Lattice now in place, the next lecture can finally state the **Learning With Errors (LWE)** problem precisely: a secret vector, hidden inside noisy linear equations over $\mathbb{Z}_q$ (a ring, Section 1.6), which is exactly the kind of "bad basis" hardness described above, dressed up in a specific, standardized cryptographic form. We will also unpack the ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ flagged in Section 1.4, since **module lattices** (lattices whose coordinates come from a ring like R_q instead of plain integers) are exactly what Kyber and Dilithium use in practice.

1.9 Summary Table

Structure	**Requires**	**Example**
Group	1 operation, inverses exist	$(\mathbb{Z}, +)$
Subgroup	A subset that is itself a group	$2\mathbb{Z} \subseteq \mathbb{Z}$
Coset / Quotient group	G sorted into shifted copies of H	$\mathbb{Z}/3\mathbb{Z} = \{0, 1, 2\}$
Ring	2 operations, no division needed	$(\mathbb{Z}, +, \times)$, $\mathbb{Z}[x]$
Integral domain	Ring, no zero divisors	$\mathbb{Z}$, $\mathbb{Z}_5$ (not $\mathbb{Z}_6$)
Field	Ring + division by nonzero elements	$\mathbb{Q}, \mathbb{R}, \mathbb{Z}_p$ (p prime)
Modulus	Wraparound arithmetic	$\mathbb{Z}_n = \{0, \ldots, n-1\}$
Group of units $\mathbb{Z}_n^*$	Invertible elements of $\mathbb{Z}_n$, size $\varphi(n)$	$\mathbb{Z}_{12}^* = \{1, 5, 7, 11\}$
Vector space	Field + vectors + scalar mult.	$\mathbb{R}^n$
Basis	Independent, spanning vector set	$\{(1, 0), (0, 1)\}$ in $\mathbb{R}^2$
Orthogonal basis	Basis, all pairs perpendicular	Gram–Schmidt output
Lattice	Discrete additive subgroup of $\mathbb{R}^n$	$\mathbb{Z}^n$; grid of points
$\lambda_1(L)$	Length of the shortest nonzero lattice vector	$\lambda_1(\mathbb{Z}^2) = 1$
$\det(L)$	Volume of the fundamental domain	$\lvert\det(B)\rvert$, any basis B
LLL	Poly-time basis reduction, exponential γ	Exact for $n = 2$ (Gauss–Lagrange)
BKZ	LLL generalized with block size k	$k = 2$: LLL; $k = n$: exact, exponential time

1.10 Full List of Exercises (Assign as Homework Before Lecture 2)

Exercise: Homework Set

1. Verify $(\{1, 2, 3, 4\}, \times \bmod 5)$ is a group by writing out its full multiplication table (Section 1.3 exercise, above).
2. Is $3\mathbb{Z} = \{\ldots, -6, -3, 0, 3, 6, \ldots\}$ a subgroup of $(\mathbb{Z}, +)$? Justify using the subgroup test (Section 1.3.1). Is $\{0, 3, 6, 9, \ldots\}$ (only the non-negative multiples of 3) a subgroup of $(\mathbb{Z}, +)$? Why or why not?
3. List all distinct cosets of $4\mathbb{Z}$ in $\mathbb{Z}$, confirm there are exactly 4, and identify which coset -5 belongs to (Section 1.3.2 exercise, above).
4. Find a pair of zero divisors in $\mathbb{Z}_{12}$, and use the zero-divisor fact (Section 1.4.1) to explain why $\mathbb{Z}_{12}$ cannot be a field, without computing any inverses.
5. Compute $23 \times 15 \bmod 7$; list the non-invertible elements of $\mathbb{Z}_8$; use the Extended Euclidean Algorithm (Section 1.6.1) to find the inverse of $3 \bmod 7$ — confirm it matches the worked example.
6. Use the Extended Euclidean Algorithm to find $5^{-1} \bmod 11$ (Section 1.6.1 exercise, above), showing all steps.
7. Is $\mathbb{Z}_9$ a field? Why or why not? Answer it two ways: (a) using $\gcd$ (Section 1.6), and (b) using zero divisors (Section 1.4.1).
8. Show that $\{(1, 1), (1, -1)\}$ is a valid basis of $\mathbb{R}^2$ by (a) checking linear independence and (b) writing the vector $(4, 2)$ as a linear combination of them.
9. Run Gram–Schmidt on $b_1 = (1, 1)$, $b_2 = (0, 2)$ (Section 1.7.3 exercise, above), and verify the result is orthogonal.
10. For the lattice generated by $b_1 = (1, 2)$ and $b_2 = (3, 1)$: (a) is the point $(7, 5)$ in the lattice? (b) compute $\det(L)$ from this basis; (c) pick a different valid basis for the same lattice and confirm $\det(L)$ is unchanged (Section 1.8.5 exercise, above).
11. Let $L = \{(x, y) \in \mathbb{Z}^2 : x + y \text{ is even}\}$. (a) Show L is a subgroup of $(\mathbb{R}^2, +)$. (b) Show L is discrete (find an ε that isolates $\vec{0}$). (c) Since L is therefore a lattice, find a basis $\{b_1, b_2\}$ for it, and compute $\det(L)$.
12. In your own words (3–4 sentences): why does the definition of a lattice need *both* "subgroup" and "discrete"? Use the $\mathbb{Q}$ example from Section 1.8.2 to explain what goes wrong if you drop the discreteness requirement.
13. In your own words (3–4 sentences), explain why a "good basis" and a "bad basis" for the same lattice can make the same mathematical object feel completely different to work with computationally, now that you can measure this with norms (Section 1.7.2).
14. Run the LLL reduction procedure by hand (Section 1.8.8) on $b_1 = (3, 1)$, $b_2 = (7, 2)$: compute $\mu_{2,1}$, size-reduce, check whether a swap is needed, and continue until the basis is fully reduced. Confirm your final basis matches the standard basis of $\mathbb{Z}^2$ (check $|\det(B)| = 1$ first to confirm this is the right lattice to expect that from).
15. In 3–4 sentences: why is LLL provably exact for $n = 2$ but only guaranteed within a factor of $2^{(n-1)/2}$ for larger n? Then, in 2–3 more sentences, explain what BKZ's block size k actually trades off, and why $k = n$ is not a practical algorithm even though it's an exact one.

Lesson 2: From Lattices to Learning With Errors

SVP, CVP, SIS, LWE, Ring-LWE, and Module-LWE

Purpose of this lecture. Lecture 1 built the algebraic toolkit — *Group* → *Subgroup* → *Ring* → *Field* → *Modular Arithmetic* → *Vector Space* → *Basis* → *Lattice* → *Determinant* — and ended by naming the Shortest Vector Problem (SVP) as one of the hard problems lattice cryptography rests on. This lecture finishes the job: we state SVP and CVP formally, meet the **Short Integer Solution (SIS)** and **Learning With Errors (LWE)** problems precisely, and then generalize LWE to **Ring-LWE** and **Module-LWE** — the exact hard problems underneath Kyber (ML-KEM) and Dilithium (ML-DSA). By the end, the public key of a real post-quantum scheme should look to you like "just an LWE sample," not a mysterious block of numbers.

2.1 Symmetric vs. Asymmetric Cryptography

Before any hard problem, any lattice, or any one-way function, cryptography splits into two families that solve two different problems. It's worth being precise about the split once, right at the start, since everything else in this course — and everything Kyber, Dilithium, and Falcon do — is building a piece of one specific family.

Definition: Symmetric-Key Cryptography

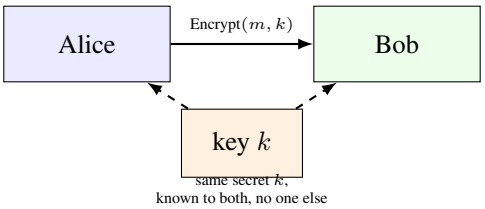
Both parties share the *same* secret key k, used both to encrypt and to decrypt (or to generate and to verify a message authentication code). Algorithms like AES and ChaCha20 are symmetric. Symmetric cryptography is fast — often gigabytes per second on ordinary hardware — but it has a bootstrapping problem built in: both parties need k *before* they can communicate securely, and getting k from one party to the other, over a channel an attacker might be watching, is exactly the problem symmetric cryptography alone cannot solve.

Definition: Asymmetric (Public-Key) Cryptography

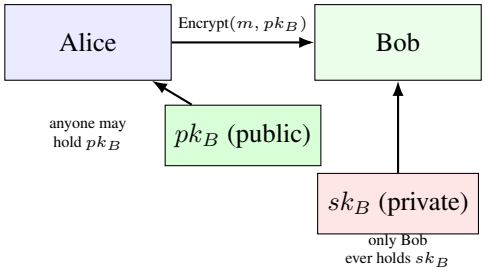
Each party generates a *pair* of mathematically linked keys: a **public key**, safe to hand to anyone, including an attacker, and a **private key**, kept secret forever. Anyone with Bob's public key can encrypt a message only Bob's private key can decrypt — or verify a signature only Bob's private key could have produced. No shared secret is ever needed in advance; the public key can be shouted across a room an eavesdropper is listening to, and security holds anyway. This is exactly the one-way-function machinery Section 2.3 builds next: the public key is the "easy forward direction," and only the matching private key makes the "hard backward direction" easy.

2.1.1 What Lattice-Based Crypto, RSA, and ECC Each Offer

All three of RSA, elliptic-curve cryptography (ECC), and lattice-based cryptography live on the asymmetric side — but "asymmetric cryptography" isn't one single service. It covers (at least) three distinct jobs: **encryption** (hide a message so only the private-key holder reads it), **key exchange** (two parties agree on a shared secret over a public channel, with no prior relationship), and **digital signatures** (prove a message came from the private-key holder, without hiding anything). Not every family offers all three the same way.



Symmetric: one shared key does both jobs.



Asymmetric: two different keys, only one ever secret.

Figure 2.1: The structural difference in one picture. Symmetric encryption needs the same secret on both ends before anything can happen. Asymmetric encryption lets Alice encrypt using a key that was never secret at all — only decryption needs the private half, and only Bob ever has it.

Family	**Encryption / KEM**	**Key exchange**	**Signatures**
RSA	RSA-OAEP	(via encryption)	RSA-PSS
ECC	ECIES (uncommon)	ECDH	ECDSA, EdDSA
Lattice-based	ML-KEM (Kyber)	via the KEM	ML-DSA, FN-DSA

Remark: One Scheme Doing Two Jobs, vs. Two Schemes Doing One Job Each

RSA is famous for a specific reason worth naming explicitly: the *exact same* trapdoor permutation underlies both RSA-OAEP encryption and RSA-PSS signing — one hard problem (factoring), one mathematical object, two uses. Lattice-based cryptography does not do this. ML-KEM (encryption) is built from Module-LWE with a trapdoor (Section 2.4.3 covers exactly why a trapdoor is optional, not universal); ML-DSA (signatures) is also built from Module-LWE, but via a completely different mechanism, Fiat–Shamir-with-Aborts, with no trapdoor at all; FN-DSA (signatures again) abandons Module-LWE entirely for a different lattice family and a GPV trapdoor. Three schemes, three distinct engineering answers, not one dual-purpose object. This isn't a weakness of lattice cryptography — it's a direct consequence of exactly the SIS-vs-LWE, trapdoor-vs-no-trapdoor split this lecture builds toward.

2.1.2 Why Real Protocols Use Asymmetric Crypto First, Then Switch to Symmetric

Given both families exist, an obvious question follows: why not just use asymmetric encryption for everything, and skip symmetric crypto altogether? Two practical reasons say no.

Remark: Speed and Size

Asymmetric encryption is computationally expensive — modular exponentiation (RSA), elliptic-curve point multiplication (ECC), or matrix-vector arithmetic over R_q (lattices) all cost far more per byte than symmetric ciphers like AES, which are often implemented directly in CPU hardware. Asymmetric schemes are also awkward at scale: RSA can only directly encrypt messages shorter than its modulus, and none of the three families are designed to efficiently churn through gigabytes of streaming data the way AES is. Symmetric crypto is fast and handles arbitrary volumes of data; asymmetric crypto is slow and, in various ways, size-limited.

Remark: The Standard Fix: Hybrid Encryption

Nearly every real protocol — TLS (the padlock in your browser), SSH, Signal — uses **both**, in a fixed order: run an asymmetric key exchange or KEM *once*, briefly, to establish a shared symmetric key between two parties who have never met; then switch to fast symmetric encryption (typically AES or ChaCha20) for the actual data, using that newly shared key. Asymmetric crypto solves the one problem symmetric crypto structurally cannot (agreeing on a secret with no prior relationship); symmetric crypto then does the actual heavy lifting, fast. This is precisely why ML-KEM is called a **Key Encapsulation Mechanism** rather than an encryption scheme in its own right — as you'll see in Lecture 3, its entire output is a short, fresh symmetric key, handed off to exactly this kind of hybrid handshake, not a general-purpose way to encrypt arbitrary messages.

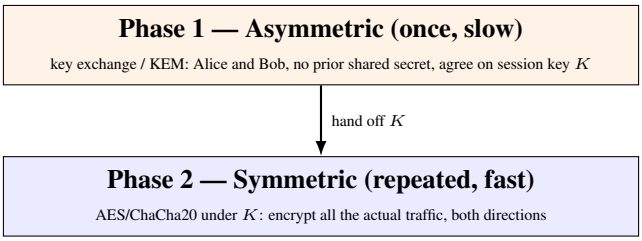


Figure 2.2: The hybrid pattern nearly every real protocol follows: a single, expensive asymmetric step establishes a shared key, then cheap symmetric encryption handles everything after. Kyber's role in ML-KEM is entirely Phase 1 — Lecture 3 picks this up in full.

2.2 Quick Recap of Lecture 1

Before building anything new, let's fix the vocabulary we'll be using constantly today.

Term	One-line meaning
Ring $(R, +, \times)$	Two operations; addition invertible, multiplication need not be
$\mathbb{Z}_q$	Integers $\{0, \ldots, q-1\}$, wraparound ("clock") arithmetic
Vector space, basis	Every point = a unique combination of basis directions
Lattice L	*Integer-only* combinations of a basis; a discrete grid in $\mathbb{R}^n$
$\det(L)$	Volume of the lattice's fundamental cell; basis-independent
$\lambda_1(L)$	Length of the shortest nonzero vector in L
Good basis vs. bad basis	Short, near-orthogonal vs. long, skewed — same lattice, different difficulty

Remark: The One Idea Everything Today Builds On

Lecture 1 ended with this sentence: *"the secret key is typically a good (short) basis, while the public key describes the same lattice using a bad basis."* Today we make that idea concrete and computational. Every hard problem in this lecture is, underneath, a question of the form: *"given a bad description of a lattice, recover something a good description would make trivial."*

2.3 One-Way Functions: The Idea Behind All of Cryptography

2.3.1 What Makes a Function "One-Way"?

Every cryptographic scheme — classical or post-quantum — is ultimately an engineering answer to the same question: is there a function that is **easy to compute in one direction and computationally infeasible to invert in the other**? Mixing two paint colors is easy; recovering the exact two starting

colors from the mixture is not, even though nothing about the mixing itself was complicated. A **one-way function** is the mathematical version of that asymmetry: given x, computing $f(x)$ takes a fraction of a second; given only $f(x)$, finding an x that produces it is believed to take longer than the lifetime of the universe, once the numbers involved are large enough.

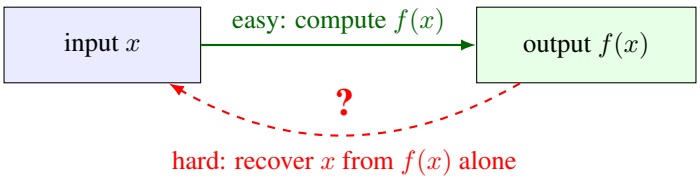


Figure 2.3: The shape every cryptographic hard problem shares: an easy forward direction, and a backward direction with no known efficient shortcut. Everything in this lecture — SVP, CVP, SIS, LWE — is one more attempt to build a trustworthy instance of this exact picture.

2.3.2 Two Classical Examples: Factoring and the Discrete Log Problem

Long before lattices entered cryptography, two number-theoretic one-way functions carried almost the entire field. Both are worth seeing now, briefly and without proof, purely so that the lattice-based version you're about to meet has something familiar to stand next to; a later lecture in this series covers RSA and elliptic-curve cryptography in full.

- **Integer factorization.** Multiplying two large primes p and q together to get $n = p \times q$ takes a computer a fraction of a second, no matter how large p and q are. Going backward — given only n, recovering the two primes p and q that were multiplied — has no known efficient algorithm once n is a few hundred digits long. This asymmetry is the entire foundation of **RSA**.
- **The discrete logarithm problem.** Fix a group (for instance, integers modulo a large prime p, under multiplication) and a fixed element g in it. Computing g^a for a given exponent a is fast (repeated squaring). Going backward — given g and g^a, recovering a — is again believed to have no efficient general algorithm. This asymmetry underlies **Diffie–Hellman key exchange**, **ElGamal encryption**, and, in a version that swaps the group for points on an elliptic curve, **elliptic-curve cryptography (ECC)**.

Remark: Why Both Are Now on Borrowed Time

Factoring and the discrete log problem are exactly the two problems a large-scale quantum computer, running **Shor's algorithm**, can solve efficiently — which is precisely why RSA and ECC are the schemes post-quantum cryptography needs to replace, and precisely why this course exists. Lattice problems are one of the leading replacement candidates because no known quantum algorithm solves them efficiently.

2.3.3 A Lattice-Flavored Preview

Lattice-based cryptography needs its own one-way function, built from its own hard problem, with the same easy-forward/hard-backward shape as factoring and the discrete log problem — just grown out of the geometry Lecture 1 built rather than out of number theory. The next section states that hard problem formally, in two versions.

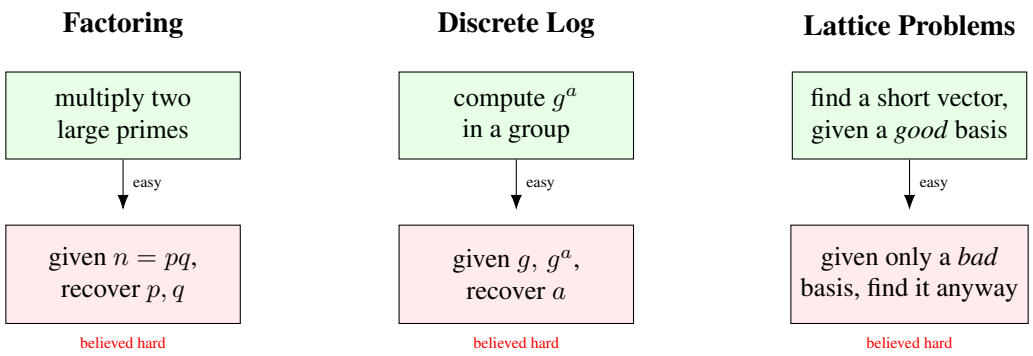


Figure 2.4: Three unrelated-looking sources of one-way hardness, side by side. Factoring underlies RSA; the discrete log problem underlies Diffie–Hellman and elliptic-curve cryptography. Lattice problems are this course's family — "good basis" and "bad basis" are exactly Lecture 1's terms, and the next section pins down precisely what "find it anyway" means.

2.4 Two Canonical Hard Problems on a Lattice

2.4.1 The Shortest Vector Problem (SVP)

Definition: Shortest Vector Problem (SVP)

Given a basis $B = \{b_1, \ldots, b_n\}$ for a lattice $L = \mathcal{L}(B) \subseteq \mathbb{R}^n$, find a nonzero vector $v \in L$ with $\|v\| = \lambda_1(L)$ (the minimum distance defined in Lecture 1). The **decision** version, GapSVP$_\gamma$, only asks: is $\lambda_1(L) \leq d$, or is $\lambda_1(L) > \gamma \cdot d$, for a given distance d and approximation factor $\gamma \geq 1$? (One of these two must hold; the problem promises no "messy middle" case.)

Remark: Why the "Approximation Factor" γ Matters

Finding the *exact* shortest vector ($\gamma = 1$) is extremely hard even for classical computers once n is a few hundred. But so is finding a vector even *polynomially longer* than the shortest one, as long as γ stays modest and n is large — and that weaker, "approximate" version is all cryptography actually needs to be hard. This is why you will see phrases like "SVP is hard to approximate within polynomial factors" rather than just "SVP is hard."

2.4.2 The Closest Vector Problem (CVP)

Definition: Closest Vector Problem (CVP)

Given a basis B for a lattice $L \subseteq \mathbb{R}^n$ and a **target point** $t \in \mathbb{R}^n$ (not necessarily in L), find the lattice point $v \in L$ minimizing $\|t - v\|$. The approximate decision version GapCVP$_\gamma$ again just distinguishes "some lattice point within distance d of t" from "every lattice point is farther than $\gamma \cdot d$ from t."

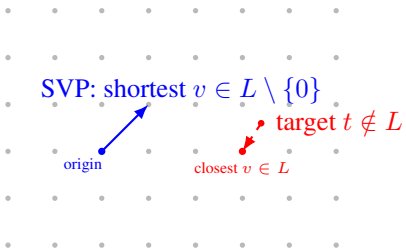


Figure 2.5: SVP (left, blue) asks for the shortest nonzero vector reachable inside the lattice itself. CVP (right, red) asks, for an arbitrary target point that is *not* on the grid, which grid point is nearest. Both are easy to state and easy to solve by eye in 2D — and famously hard in high dimensions with only a bad basis to work with.

Example: CVP by Eye vs. CVP by Formula

In Figure 2.5 you can *see* the closest grid point instantly. That's because your eye is effectively using a very good, orthogonal basis (the obvious horizontal/vertical grid directions). Algorithmically, CVP is solved by writing $t = \sum c_i b_i$ (allowing real, non-integer c_i) and **rounding each c_i to the nearest integer** — but this shortcut only gives the *true* closest point when the basis is good (short, near-orthogonal). Hand the exact same lattice to an algorithm with a bad basis instead, and naive rounding can miss the true closest point by a wide margin. This gap — correct-with-a-good-basis, unreliable-with-a-bad-basis — is precisely what CVP-based cryptography exploits.

2.4.3 The Big Picture: From Worst-Case Geometry to a Cryptographic Tool

Section 2.3 listed three things a raw hard problem needs before it becomes a usable one-way function: an efficiently **sampleable random instance**, a **proof** tying that instance's hardness back to the worst case, and an **easy forward direction**. SVP and CVP, exactly as just stated, don't hand you any of the three for free — "given a basis, find X" says nothing about how to generate a random hard instance, let alone prove anything about it.

SIS and LWE, met later in this lecture, are exactly this fix. Each one starts from a **uniformly random matrix** A — trivial to generate — and each comes with a theorem (Ajtai, 1996 for SIS; Regev, 2005 for LWE) tying average-case hardness of that random instance back to worst-case SVP/CVP-style hardness. In fact, SIS and LWE are not new ideas grafted onto SVP and CVP: **they are SVP and CVP**, restricted to one specific, algebraically-generated, easy-to-sample family of lattices built from A.

Remark: A Trapdoor Is a Fourth, Optional Ingredient — Not Part of the Three Requirements

The explicit **Trapdoor** box in Figure 2.6 sits only on LWE's path, and that placement is the point. The three requirements from Section 2.3 (a sampleable random instance, a worst-case reduction, an easy forward direction) are exactly what a plain one-way function needs, and that's already enough for **SIS-based hashing**: nobody — not even whoever generated A — can efficiently find a collision. There is no secret held by any party, and none is needed; that's the entire point of a hash function, which is why SIS's path skips the trapdoor box entirely. A **trapdoor** is a separate, optional fourth ingredient, needed only when some designated party must be able to invert efficiently while everyone else can't: exactly what **LWE-based encryption** needs (the key-owner decrypts; nobody else can), and what Falcon's GPV sampling (Lecture 4) needs on the signing side.

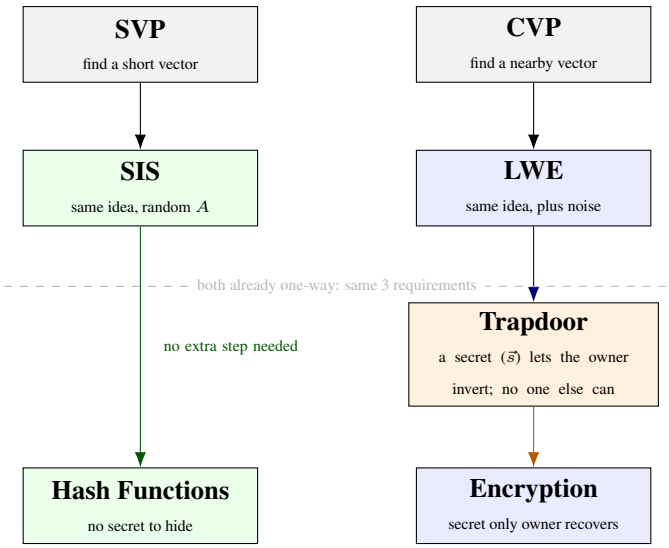


Figure 2.6: SIS and LWE are not a detour from SVP and CVP — they *are* SVP and CVP, restricted to a random, easy-to-sample family of lattices built from a matrix A. SIS lives on the kernel lattice $\Lambda^{\perp}(A)$ (Section 2.5) and keeps SVP's "find a short vector" shape; LWE lives on the image lattice $\Lambda_q(A)$ (Section 2.6.5) and keeps CVP's "find a nearby point" shape. Both already satisfy the three requirements above the dashed line — that alone makes each a plain one-way function. The paths then diverge: SIS's kernel/short-vector shape needs nothing further to give you "no two things collide" (hash functions — nothing to hide), while LWE's image/nearby-point shape only becomes *encryption* once a trapdoor is layered on top, giving one party (and only that party) an efficient inverse.

Remark: Where the Two Halves of This Figure Get Made Rigorous

Section 2.5's "Why This Is Secretly a Lattice Problem" box shows SIS's solutions living inside the kernel lattice $\Lambda^{\perp}(A)$ — literally an SVP instance. Section 2.6.5's "LWE Is Bounded-Distance Decoding" box shows LWE's target b sitting near the image lattice $\Lambda_q(A)$ — literally a (promise-bearing) CVP instance. Both boxes cash in the claim made by Figure 2.6 above.

2.5 The Short Integer Solution (SIS) Problem

Remark: Motivating SIS With a Question

Here's a question that sounds almost too easy: if I hand you a random matrix A (think: rows and columns of random numbers mod q), can you find a *short* nonzero vector x so that $Ax \equiv \vec{0} \pmod{q}$? If x could be any size at all, this would be trivial linear algebra — as long as A has more columns than rows, there are always *some* nonzero solutions (more unknowns than equations). The catch, and the entire reason this is a cryptographic problem rather than a homework exercise, is the word **short**: among the (many) solutions that exist, finding one whose entries are all small turns out to be hard once the matrix is big enough. This is SIS.

Definition: Short Integer Solution (SIS)

Fix positive integers n, $m > n$, a modulus q, and a bound β. Given a uniformly random matrix $A \in \mathbb{Z}_q^{n \times m}$, find a nonzero vector $x \in \mathbb{Z}^m$ with

$$Ax \equiv \vec{0} \pmod{q} \qquad \text{and} \qquad \|x\| \leq \beta.$$

Remark: Why This Is Secretly a Lattice Problem

Define $\Lambda^{\perp}(A) = \{\, x \in \mathbb{Z}^m : Ax \equiv \vec{0} \pmod{q} \,\}$. The name "$\perp$" is exactly what it looks like: $Ax \equiv \vec{0}$ means x pairs to zero (mod q) with *every* row of A, so $\Lambda^{\perp}(A)$ is the orthogonal complement of A's row space — the same idea as $V^{\perp}$ for ordinary vector spaces, just computed mod q instead of over $\mathbb{R}$.

A quick check confirms $\Lambda^{\perp}(A)$ is a subgroup of $\mathbb{Z}^m$ (closed under addition and negation, contains $\vec{0}$) and it is discrete (it's a subset of $\mathbb{Z}^m$) — so by Lecture 1's subgroup definition, $\Lambda^{\perp}(A)$ **is itself a lattice**. Solving SIS is exactly finding a short nonzero vector in $\Lambda^{\perp}(A)$ — i.e., **SIS is SVP**, restricted to this specific, algebraically-generated family of lattices. The reason cryptographers like this family is that generating a *random* instance is as easy as generating a random matrix A, yet the resulting lattice is (conjecturally) exactly as hard to solve SVP on as the worst-case lattices Lecture 1 discussed abstractly.

Example: A Tiny SIS Instance by Hand

Let $n = 1$, $m = 4$, $q = 17$, and $A = \begin{pmatrix} 3 & 5 & 8 & 1 \end{pmatrix} \in \mathbb{Z}_{17}^{1\times 4}$. We want a short, nonzero, *integer* vector $x = (x_1, x_2, x_3, x_4)$ with $3x_1+5x_2+8x_3+x_4 \equiv 0 \pmod{17}$. Try $x = (1, 1, -1, -16) \to$ too long. Instead try $x = (1, -1, 0, 2)$: $3(1) + 5(-1) + 8(0) + 1(2) = 3 - 5 + 0 + 2 = 0 \equiv 0 \pmod{17}$. Success, and $\|x\|$ is small (each entry is ≤ 1 in absolute value except the last, which is 2). This x is a valid, short solution — with $m = 4$ columns squeezed against only $n = 1$ modular equation, short solutions are guaranteed to exist by a pigeonhole argument (more columns than rows means lots of redundancy); the point of SIS being *hard* is that finding one is nontrivial once n, m are cryptographically large (hundreds to thousands), not that one fails to exist.

Remark: What SIS Is Used For

SIS is the workhorse behind lattice-based **hash functions** and was historically the basis of early lattice signature schemes. Section 2.5.1 below builds the hash-function connection explicitly, right now, while the SIS instance above is still fresh. You will meet SIS's signature-adjacent side again in Session 3 (Dilithium/ML-DSA) and again in Lesson 6, since Dilithium's fault-attack surface is closely tied to how it uses a SIS-flavored rejection-sampling step.

2.5.1 A Concrete Application: SIS-Based Hash Functions

Remark: What a Hash Function Needs

A cryptographic hash function takes an input of some size and **compresses** it down to a fixed, shorter output, with one crucial security property: **collision resistance** — it should be hard to find two *different* inputs $x_1 \neq x_2$ that hash to the *same* output. Everything below is aimed at exactly one goal: building a compressing function whose collision resistance follows directly, and provably, from SIS being hard — no separate, from-scratch security assumption required.

Definition: The Ajtai-Style SIS Hash Function

Fix a uniformly random matrix $A \in \mathbb{Z}_q^{n \times m}$ (exactly the SIS matrix above), with $m > n \log_2 q$ so the map below is genuinely compressing. Define $h_A : \{0,1\}^m \to \mathbb{Z}_q^n$ by

$$h_A(x) = Ax \bmod q.$$

The input x is an m-bit string (so there are 2^m possible inputs); the output lives in $\mathbb{Z}_q^n$ (only q^n possible outputs). Since $m > n \log_2 q$ forces $2^m > q^n$, the function is squeezing a strictly bigger set down into a strictly smaller one — exactly the compression a hash function needs, and exactly why some collision is guaranteed to exist by the pigeonhole principle alone (finding one is the hard part).

Remark: Why Collision Resistance *Is* SIS, Not Just *Like* SIS

Suppose an attacker finds a collision: two distinct inputs $x_1 \neq x_2 \in \{0,1\}^m$ with $h_A(x_1) = h_A(x_2)$, i.e. $Ax_1 \equiv Ax_2 \pmod q$. By linearity, $A(x_1 - x_2) \equiv \vec{0} \pmod q$. Set $x = x_1 - x_2$. Then:

- $x \neq \vec{0}$, since $x_1 \neq x_2$.
- $Ax \equiv \vec{0} \pmod q$ — exactly the SIS equation.
- x is automatically **short**: each coordinate of $x_1, x_2 \in \{0,1\}^m$ is 0 or 1, so each coordinate of $x = x_1 - x_2$ is in $\{-1, 0, 1\}$, giving $\|x\| \leq \sqrt{m}$ — comfortably within any reasonable SIS bound β.

So **a collision in h_A is, verbatim, a solution to the SIS instance defined by the same matrix A.** Turning this around: if SIS is hard for A, then finding a collision in h_A is exactly as hard — there is no weaker link elsewhere in the construction. This is the entire appeal of building a hash function this way: security reduces to a single, already-studied lattice problem, with nothing extra to trust.

Remark: Why the Domain Restriction Isn't Optional

It's worth being explicit about why h_A is defined on bit strings $\{0,1\}^m$ specifically, rather than on all of $\mathbb{Z}_q^m$. Without a shortness bound, $Ax \equiv \vec{0} \pmod q$ is *always* trivially solvable, for *any* matrix A at all: just take $x = q \cdot e_i$ (i.e. q in one coordinate, 0 elsewhere) for any i. Then $Ax = q \cdot (Ae_i) \equiv \vec{0} \pmod q$ automatically, no matter what A is — every entry is a multiple of q. So "find a nonzero x with $Ax \equiv \vec{0}$," with no bound on $\|x\|$, carries *zero* cryptographic content; this is exactly why the SIS defbox above required $\|x\| \leq \beta$ as part of the problem statement, not as an afterthought.

This is precisely what the domain restriction to $\{0,1\}^m$ is doing, and it's doing double duty: $m > n \log_2 q$ forces a compression (Section 2.5.1's defbox), *and*, because both inputs come from $\{0,1\}^m$, their difference is *automatically* short — $\|x\| \leq \sqrt{m}$ — no matter which collision an attacker happens to find. If the hash's domain were unrestricted instead, a "collision" could hand back the trivial $x = q \cdot e_i$ above, which satisfies the SIS equation but is not a hard instance of anything. The reduction only means something because the construction *forces* every possible collision, without exception, into the short/hard regime.

Example: A Collision, Found and Verified by Hand

Reuse $n = 1$, $m = 4$, $q = 17$, $A = \begin{pmatrix} 3 & 5 & 8 & 1 \end{pmatrix}$ from the SIS worked example above, and the short SIS solution already found there, $x = (1, -1, 0, 2)$. That x has entries outside $\{-1, 0, 1\}$ in one coordinate (the 2), so it isn't itself a difference of two bit-strings — but it illustrates the mechanism cleanly, and a genuine bit-string collision is built the same way. Take

$$x_1 = (1, 0, 0, 1), \qquad x_2 = (0, 1, 0, 0) \qquad \text{(both in } \{0,1\}^4\text{)}.$$

Compute both hashes mod 17:

$$h_A(x_1) = 3(1) + 5(0) + 8(0) + 1(1) = 4, \qquad h_A(x_2) = 3(0) + 5(1) + 8(0) + 1(0) = 5.$$

These don't collide ($4 \neq 5$) — so x_1, x_2 is *not* a collision, only a candidate pair, exactly as most randomly-chosen pairs will not collide. This is the expected, normal outcome: with $q = 17$ this toy instance barely compresses at all ($m = 4$ bits down to one symbol in $\{0, \ldots, 16\}$), so collisions are not hard to stumble on by search, but the *mechanism* — take any actual collision, subtract, get a short nonzero SIS solution — is exactly what the remark above proves in general, regardless of how easy or hard this particular toy-sized search happens to be.

Remark: Why This Only Works Because of Compression

It's worth being precise about where $m > n \log_2 q$ was used. Without that condition, h_A might be injective (no collisions could exist at all, even in principle), and the whole security question would be vacuous. With it, collisions are *guaranteed to exist* (pigeonhole, as noted in the def-box) — the hardness claim is never "no collision exists," only "no one can *find* one efficiently." This mirrors ordinary cryptographic hash functions like SHA-256 exactly: collisions must exist (infinite inputs, finite outputs), and security only ever means "hard to find," never "impossible."

Fact: $h_A(x) = Ax$ vs. SHA-2/SHA-3: Two Different Kinds of Confidence

- **SHA-2/SHA-3:** security is *empirical* — decades of public cryptanalysis have failed to find a collision. Fast, compact, no known quantum weakness worth designing around (Grover only gives a quadratic speedup). The default choice for hashing in practice.
- **h_A:** security is a *theorem* — a collision provably yields a solution to worst-case SIS/lattice problems, not just this one instance. The cost is size and speed: a large public matrix A and a matrix–vector multiply per hash, versus a few dozen fast bitwise operations.
- **Why bother, then:** h_A is *linear* in x, so it composes algebraically with other lattice constructions — homomorphic operations, commitments, zero-knowledge proofs — in ways SHA-2/SHA-3's deliberately nonlinear internals cannot. Its role is as a building block *inside* lattice-based protocols, not as a drop-in replacement for general-purpose hashing.

Exercise: Practice

Using the same $A = (3, 5, 8, 1) \in \mathbb{Z}_{17}^{1\times 4}$: (a) compute $h_A(x)$ for $x_1 = (1, 1, 0, 0)$ and $x_2 = (0, 0, 1, 1)$; do they collide? (b) Find, by inspection or trial, a genuine colliding pair $x_1 \neq x_2 \in \{0,1\}^4$ with $h_A(x_1) = h_A(x_2)$. (c) For your pair from part (b), compute $x = x_1 - x_2$ and confirm directly that $Ax \equiv 0 \pmod{17}$ — i.e., confirm your collision really does hand you a SIS solution, exactly as the argument above claims it must.

Remark: SIS vs. LWE: Why We Need Both

SIS and LWE are built from the same raw material — a random matrix A — but they are hard for different reasons and do different jobs. SIS's hardness is about *collisions*: it's hard to find a short x with $Ax \equiv \vec{0}$, which is exactly the guarantee **hash functions** (Section 2.5.1 above) and (lattice) **signatures** need — there's no secret to hide, only "no two short inputs collide." LWE's hardness is about *concealment*: without noise, $b = As$ is solved instantly by ordinary linear algebra (Section 2.6.1's box below); the noise e is precisely what breaks that shortcut, and it creates an asymmetry — whoever knows s can cancel/tolerate the noise, while anyone else just sees (A, b) that looks uniformly random. That asymmetry is what **encryption** needs, and SIS alone can't give it. The two problems are even dual views of the same A: SIS lives on the kernel lattice $\Lambda^{\perp}(A)$ (Section 2.5 above), LWE lives on the image lattice $\Lambda_q(A)$ (Section 2.6.5 below) — one random matrix, two lattices, two different cryptographic jobs.

2.6 Learning With Errors (LWE)

2.6.1 The Idea, in Plain Language

Imagine someone hands you a stack of linear equations in an unknown secret vector s — but each equation's right-hand side has been nudged by a small, random amount of noise before you see it. Solving *noise-free* linear equations is easy (that's just linear algebra — Gaussian elimination). Solving them once every equation is slightly wrong, and you don't know by how much, turns out to be extremely hard. That is the entire idea of LWE.

Remark: Why Not Just Solve It With Linear Algebra?

This is the single most common point of confusion about LWE, so it's worth working through a tiny concrete case. Suppose $n = 2$ and, with no noise at all, we're given:

$$1 \cdot s_1 + 2 \cdot s_2 = 8, \qquad 3 \cdot s_1 + 1 \cdot s_2 = 9.$$

Two equations, two unknowns — solve normally (e.g. elimination) and you get $s_1 = 2,\ s_2 = 3$ exactly, every time, no matter how large n gets, as long as you have n independent equations. **Linear algebra doesn't care how big the system is** — Gaussian elimination scales smoothly to thousands of unknowns. Now add noise: suppose the equations you're actually given are

$$1 \cdot s_1 + 2 \cdot s_2 = 8 + e_1, \qquad 3 \cdot s_1 + 1 \cdot s_2 = 9 + e_2,$$

for some small, unknown $e_1, e_2 \in \{-1, 0, 1\}$ that you are never told. Now elimination doesn't work: any attempt to "solve exactly" is solving for the *wrong* thing, since the right-hand sides are off by an unknown amount. You could try every combination of e_1, e_2 and re-solve each time — for two small error values that's manageable, but for hundreds of equations, each with its own small unknown error, the number of combinations to check explodes exponentially. That explosion, not any deep mystery, is where LWE's hardness lives: **noise breaks the one tool (linear algebra) that would otherwise make this trivial**, and no comparably efficient tool has been found to replace it.

2.6.2 Formal Definition

Definition: Learning With Errors (LWE)

Fix a secret dimension n, a number of samples m (typically $m \gg n$ in practice), a modulus q, and an error distribution χ over $\mathbb{Z}_q$ (Section 2.6.3 pins down the usual choice of χ). Exactly as with SIS (Section 2.5), the problem starts from a **uniformly random matrix** — fix $A \in \mathbb{Z}_q^{m \times n}$ chosen uniformly at random. Let $s \in \mathbb{Z}_q^n$ be a fixed, unknown **secret vector**, and let $e \in \mathbb{Z}_q^m$ be an **error vector** whose m entries are drawn independently from χ. Define

$$b \;=\; As + e \pmod{q} \;\in \mathbb{Z}_q^m.$$

- **Search-LWE:** given (A, b), find s.
- **Decision-LWE:** given (A, b), decide whether b was built this way, or b is instead a *uniformly random* vector in $\mathbb{Z}_q^m$, independent of A.

Row by row, this is exactly the "many samples" description. Write $a_i \in \mathbb{Z}_q^n$ for the i-th row of A, and b_i for the i-th entry of b. Then $b_i = \langle a_i, s \rangle + e_i \pmod{q}$ is one LWE sample, for $i = 1, \ldots, m$ — and A is nothing more than these m row-vectors $a_1, \ldots, a_m$ stacked on top of each other. "Many noisy inner-product samples" and "one noisy matrix-vector equation" are the *same object*, just described two ways; every worked example below moves freely between the two descriptions.

Remark: Yes — LWE Has a Matrix Too, Just Playing the Opposite Role From SIS

It's easy to walk away from Section 2.5 thinking "SIS has a matrix, LWE doesn't" — especially since LWE is usually introduced sample-by-sample first. That impression is wrong: **both problems start from the exact same raw ingredient, a uniformly random matrix over $\mathbb{Z}_q$.** What differs is the *shape* of the matrix and what you're asked to do with it:

- **SIS:** $A \in \mathbb{Z}_q^{n \times m}$ is *wide* (more columns than rows, $m > n$). You search for a short nonzero x in A's *kernel*: $Ax \equiv \vec{0}$. Nothing is hidden — the matrix is entirely public, and the whole difficulty is finding one particular short vector among the many that satisfy the equation.
- **LWE:** $A \in \mathbb{Z}_q^{m \times n}$ is *tall* (at least as many rows as columns, $m \geq n$, usually $m \gg n$). You're handed a point b that sits *near* A's *image* (near some As), nudged off by small noise e, and asked to recover which s produced it, or even just to tell that b isn't uniform.

Same raw material, opposite structural question: SIS asks "what short vector does A kill?"; LWE asks "what secret does A almost reveal?" This is also exactly why Lecture 1's subgroup/lattice machinery applies to *both*: SIS's matrix generates the kernel lattice $\Lambda^{\perp}(A)$ (Section 2.5), while LWE's matrix generates an image lattice $\Lambda_q(A)$ (Section 2.6.5 below) — two different lattices built from the same kind of object.

A notational warning. Some sources — including Chris Peikert's *Lattices in Cryptography* lecture notes (Lecture 13, "Learning With Errors"), which Lecture 1's bibliography already points to — state LWE with the *transposed* convention: a *wide* matrix $A \in \mathbb{Z}_q^{n \times m}$ (columns, not rows, are the samples), a secret *row* vector s, and $b^T = s^T A + e^T$. This is mathematically identical to the defbox above — just transpose everything, and "columns of A are samples" becomes "rows of A are samples." We fix the row-convention above throughout this course because it matches how Lecture 3 writes Kyber's $\vec{t} = A\vec{s} + \vec{e}$ and Lecture 4's ML-DSA $\vec{t} = A\vec{s}_1 + \vec{s}_2$; just be aware that flipping between sources' conventions is normal in this literature, and always check which axis a given source calls "n" and which it calls "m" before comparing formulas.

2.6.3 What the Error Distribution χ Actually Is

We've been saying "a small error" without pinning down what "small" means or where e_i actually comes from. Time to fix that.

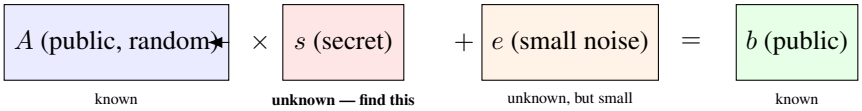
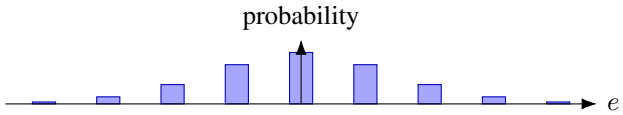


Figure 2.7: The anatomy of one LWE equation $b = As + e \pmod{q}$. An attacker sees A and b (both public/known) and must recover s, without ever being told e — only that e is small. Without the noise term e, this would be ordinary linear algebra and trivial to invert; the noise is precisely what makes it hard.

Definition: Discrete Gaussian Distribution (the usual choice of χ)

For a parameter $\sigma > 0$ (roughly, how spread out the noise is), the discrete Gaussian distribution over $\mathbb{Z}$ assigns to each integer x a probability proportional to $\exp(-x^2/2\sigma^2)$ — the same bell-curve shape as the ordinary (continuous) Gaussian, just evaluated only at integers and re-normalized so the probabilities sum to 1. In practice this means: $e = 0$ is the single most likely outcome, $e = \pm 1$ is somewhat less likely, $e = \pm 2$ less likely still, and values far from 0 (say $|e| > 6\sigma$) are so improbable they essentially never occur. χ is this distribution, and each e_i in an LWE sample is drawn independently from it.



Remark: Why a Bell Curve, and Not (Say) a Coin Flip $\{-1, 0, 1\}$?

Our worked examples below use a crude stand-in ($e \in \{-1, 0, 1\}$, each roughly equally likely) purely because it's easy to compute by hand. Real schemes use something bell-curve-shaped (a true discrete Gaussian, or in Kyber's case a closely related and cheaper-to-sample *centered binomial distribution*) for a precise mathematical reason: the hardness proofs connecting LWE back to worst-case lattice problems (Regev's reduction, Section 2.6.5) require the noise to behave like a Gaussian. Swap in a different-shaped distribution and those proofs may simply no longer apply — "small errors" alone isn't enough; their *shape* matters too. This is a recurring theme you'll meet again in Lessons 5 and 6: real implementations must sample from exactly the right distribution, and subtle sampling mistakes (or side-channel leakage *during* sampling) can quietly undermine security guarantees that look airtight on paper.

2.6.4 A Fully Worked Numeric Example

Example: Search-LWE by Hand, $n = 3$, $q = 17$

Let the (unknown, to an attacker) secret be $s = (2, 9, 1) \in \mathbb{Z}_{17}^3$, and let the error distribution χ produce values in $\{-1, 0, 1\}$ only (a toy stand-in for a discrete Gaussian). Suppose four samples are generated:

a_i	$\langle a_i, s \rangle \bmod 17$	e_i	$b_i = \langle a_i, s \rangle + e_i \bmod 17$
$(1, 0, 0)$	2	$+0$	2
$(0, 1, 0)$	9	-1	8
$(1, 1, 1)$	12	$+1$	13
$(3, 2, 1)$	$6 + 18 + 1 = 25 \equiv 8$	0	8

An attacker only ever sees the a_i (left column) and b_i (right column) — *not* the middle two columns. Notice that sample 1 alone, $b_1 = 2$, looks like it might reveal $s_1 = 2$ directly — and here it does, since e_1 happened to be 0. But sample 2 gives $b_2 = 8$, while the true $s_2 = 9$: the noise $e_2 = -1$ has already shifted the visible answer away from the truth by one full step, with no way for an outside observer to know whether the "off-by-one" is coming from s_2 or from e_2. Multiply that ambiguity across hundreds of dimensions and thousands of samples, and brute-force guessing collapses — this is the essence of why LWE is hard even though each individual equation looks almost solvable.

Remark: The Exact Same Example, Written as One Matrix Equation

Stack the four a_i rows from the table above into a matrix, exactly as the defbox's "row by row" remark describes — this is the $A \in \mathbb{Z}_{17}^{4\times 3}$ of the formal definition, with $m = 4$ samples and secret dimension $n = 3$:

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 1 & 1 \\ 3 & 2 & 1 \end{pmatrix}, \qquad s = \begin{pmatrix} 2 \\ 9 \\ 1 \end{pmatrix}, \qquad e = \begin{pmatrix} 0 \\ -1 \\ 1 \\ 0 \end{pmatrix}, \qquad b = As + e \bmod 17 = \begin{pmatrix} 2 \\ 8 \\ 13 \\ 8 \end{pmatrix}.$$

Multiplying out As row by row reproduces exactly the middle column of the table above $(2, 9, 12, 8 \bmod 17)$, and adding e then reduces mod 17 reproduces the last column $(2, 8, 13, 8)$ — the two views really are the same computation. An attacker in the search-LWE game is handed only A and b (the matrix and the right-hand column) and must recover s; in the decision-LWE game, they're handed A and *either* this b *or* four independent uniform values in $\mathbb{Z}_{17}$, and must say which.

Exercise: Try This Before Next Lecture

Using the same A rows as above but a *different* secret $s' = (2, 8, 1)$ (differs from s only in the middle coordinate), recompute $b_1, \ldots, b_4$ with the same errors e_i. Compare the resulting b_i values to the table above. How many of the four samples end up identical to the original table, purely by coincidence of the noise? What does this tell you about how many samples an attacker needs before guessing becomes hopeless?

Remark: Making "Brute Force Collapses" Concrete

In the toy example above, each error e_i took one of only 3 values, and there were 4 samples. A brute-force attacker who doesn't know the errors could, in principle, try every combination of errors, undo them, and check whether the resulting system solves consistently. The number of combinations to try is $(\text{number of error values})^{(\text{number of samples})}$. The table below shows how fast this explodes as the problem scales toward realistic sizes (real schemes use hundreds of samples, and a Gaussian χ with far more than 3 realistic values — this table deliberately keeps 3 values fixed just to isolate the effect of *sample count* growing):

Samples	Combinations (3^{samples})	Roughly
4 (our example)	81	trivial by hand
20	≈ 3.5 billion	a laptop, seconds
100	$\approx 5 \times 10^{47}$	far beyond any computer on Earth
500 (realistic scale)	astronomically larger still	meaningless to even state

This is only a crude illustration (real attacks are smarter than raw brute force — that's what lattice-reduction algorithms like LLL and BKZ are for), but it captures the right intuition: **the ambiguity introduced by unknown noise compounds multiplicatively with every additional equation**, which is exactly the opposite of ordinary linear algebra, where more equations make life *easier*, not harder.

Example: Decision-LWE: Can You Spot the Real Samples?

Here are two sets of (a_i, b_i) pairs over $\mathbb{Z}_{17}$. One set was generated as genuine LWE samples $b_i = \langle a_i, s \rangle + e_i$ for some fixed secret s; the other set has every b_i replaced by an independent uniformly random value in $\mathbb{Z}_{17}$, with no relationship to a_i at all.

Set X		Set Y	
a_i	b_i	a_i	b_i
$(1,0,0)$	2	$(1,0,0)$	11
$(0,1,0)$	8	$(0,1,0)$	3
$(1,1,1)$	13	$(1,1,1)$	6

Try to decide, just by looking, which set is "real" LWE and which is uniform noise. (Set X is, in fact, the real one — it's the same data as our search-LWE example above.) **The point of this exercise is that you can't tell, and neither can a computer, efficiently** — both sets look like an unremarkable list of numbers in $\mathbb{Z}_{17}$. That indistinguishability, formalized, *is* decision-LWE. It's also exactly the property real encryption schemes lean on: a ciphertext under LWE-based encryption should be computationally indistinguishable from random noise to anyone without the secret key.

2.6.5 Why LWE Is Believed Hard: the Lattice Connection

Remark: LWE Is Bounded-Distance Decoding (BDD), a Cousin of CVP

Define the lattice $\Lambda_q(A) = \{\, y \in \mathbb{Z}_q^m : y \equiv As \pmod q \text{ for some } s \in \mathbb{Z}_q^n \,\}$ (all vectors reachable as A times *some* secret, viewed as a lattice via the ring structure of $\mathbb{Z}_q$ from Lecture 1). The vector $b = As + e$ is, by construction, a point $As \in \Lambda_q(A)$ that has been nudged by a *small* vector e. Recovering s from b is therefore exactly **Bounded-Distance Decoding**: given a target b that is guaranteed to be *unusually close* to some lattice point (closer than a typical random point would be), find that lattice point. This is CVP's easier, promise-bearing cousin — and, exactly as with CVP in Section 2.4.2, solving it reliably requires something like a good basis for $\Lambda_q(A)$. The whole point of choosing A uniformly at random is that no such good basis is visible to an attacker; only whoever *generated* s (or knows a matching trapdoor) has an edge.

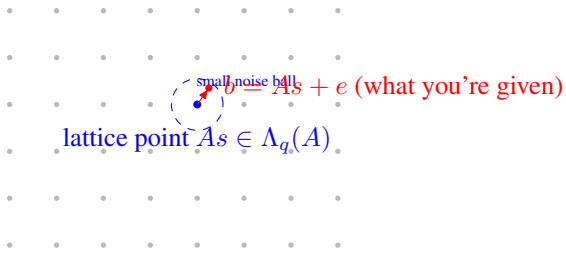


Figure 2.8: LWE as Bounded-Distance Decoding: b is guaranteed to land inside a small ball around some lattice point As — unlike general CVP (Figure 2.5), where the target could be anywhere. That extra promise ("close, not just closest") is what makes BDD tractable *with a good basis* and still believed hard *without one*.

Fact: Regev's Worst-Case-to-Average-Case Reduction (Statement Only)

A landmark 2005 result (Oded Regev) shows: if there existed an efficient algorithm that solves *average-case, randomly-generated* decision-LWE instances, that algorithm could be turned into an efficient algorithm for *worst-case* GapSVP and other hard lattice problems (via a quantum reduction; later work gave classical reductions for related problems). In plain terms: **LWE's hardness is not a bet made up from nothing** — it inherits its credibility from the same worst-case lattice hardness Lecture 1 built toward. This is exactly why cryptographers trust random LWE instances, generated fresh for every key, to be hard: hardness on average is tied by proof to hardness in the worst case. We take this as given; the proof itself is well beyond this course.

Looking Ahead: Search vs. Decision — Also Provably Equivalent

It's also a known theorem (not proven here) that search-LWE and decision-LWE are polynomially equivalent: an efficient solver for one can be converted into an efficient solver for the other. This matters practically because security *proofs* for encryption schemes are usually stated against decision-LWE ("ciphertexts are indistinguishable from random"), while *attacks* usually aim at search-LWE ("recover the actual secret key"). Lessons 5 and 6 will use both framings, so it's worth knowing they're two views of the same underlying hardness.

2.7 From Plain LWE to Ring-LWE

2.7.1 The Problem With Plain LWE: Size

A single LWE public key with security parameter n requires an $m \times n$ matrix A over $\mathbb{Z}_q$ — roughly n^2 numbers. For cryptographically secure parameters (n in the many hundreds), that's an enormous public key, and every encryption/decryption operation costs a full matrix-vector multiplication, $O(n^2)$ work. Real deployed schemes need public keys measured in kilobytes and operations fast enough for a TLS handshake or a smartcard, not this.

Example: Putting Real Numbers On "Too Big"

Take a security-relevant dimension, $n = 1024$, with q a few thousand (so each number needs about 12 bits ≈ 1.5 bytes to store). A plain-LWE public key needs roughly $n^2 = 1{,}048{,}576$ such numbers — over 1.5 megabytes, just for one public key, before even discussing ciphertexts. Compare that to a real ML-KEM-1024 public key, which is about 1,568 *bytes* in total — roughly a thousand times smaller. That gap is exactly what Ring-LWE and Module-LWE close, by replacing an $n \times n$ grid of independent numbers with a much smaller number of *structured* ring elements that each silently represent n numbers at once (Section 2.7.3's remark makes precise how).

2.7.2 The Fix: Move Into a Polynomial Ring

Recall from Lecture 1's "Ring" section and its preview box: Kyber and Dilithium do arithmetic inside $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ — polynomials of degree $< n$ with coefficients in $\mathbb{Z}_q$, where powers of x wrap around using the rule $x^n \equiv -1$.

Definition: The Ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$

Elements of R_q are polynomials $a(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{n-1} x^{n-1}$ with each coefficient $a_i \in \mathbb{Z}_q$. Addition is ordinary coefficient-wise addition mod q. Multiplication is ordinary polynomial multiplication, *followed by reduction*: whenever a term x^n appears, replace it with -1 (and more generally $x^{n+k} \to -x^k$), then reduce every coefficient mod q. This makes R_q a finite ring with exactly q^n elements — and, crucially, a single element of R_q packs n numbers into one object that a computer can multiply in roughly $O(n \log n)$ time using the Number-Theoretic Transform (NTT, a finite-field cousin of the FFT) — we will not need NTT details in this course, just the fact that it's the reason Ring-LWE is fast in practice.

Example: Multiplying Two Small Polynomials in R_q, $n = 4$, $q = 17$

Let $n = 4$, so we reduce using $x^4 \equiv -1$, and work mod $q = 17$. Take $a(x) = 1 + 2x$ and $c(x) = 3 + x^3$ in R_{17}. Ordinary polynomial multiplication first:

$$a(x)\, c(x) = (1 + 2x)(3 + x^3) = 3 + x^3 + 6x + 2x^4.$$

Now reduce using $x^4 \equiv -1$: the term $2x^4$ becomes $2 \cdot (-1) = -2$. So we're left with

$$3 + 6x + x^3 - 2 \;=\; 1 + 6x + 0x^2 + x^3.$$

Finally reduce coefficients mod 17 (here every coefficient is already in $\{0, \ldots, 16\}$, so nothing changes): the product is $1 + 6x + x^3 \in R_{17}$. Notice the "wraparound" step ($x^4 \to -1$) is exactly what folds the high-degree term back down to degree < 4 — this is the polynomial-ring analogue of "carrying" in ordinary mod-q arithmetic.

2.7.3 Ring-LWE, Formally

Definition: Ring Learning With Errors (RLWE)

Fix $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ and an error distribution χ over R_q (coefficients drawn independently from a small distribution). Let $s \in R_q$ be a fixed secret ring element. A search-RLWE sample is a pair $(a, b) \in R_q \times R_q$ where a is uniform random and

$$b = a \cdot s + e \pmod{q}, \qquad e \leftarrow \chi.$$

Search-RLWE asks for s given many such pairs; decision-RLWE asks to distinguish (a, b) from a uniformly random pair in $R_q \times R_q$.

Remark: One Ring-LWE Sample ≈ n Plain-LWE Samples

Compare the two definitions side by side: plain LWE needs an entire matrix $A \in \mathbb{Z}_q^{m \times n}$ to state m equations. A *single* Ring-LWE pair (a, b), because a and s are each secretly a length-n coefficient vector multiplied together via the structured polynomial product above, is algebraically equivalent to about n ordinary LWE-style equations bundled into one ring multiplication. This is exactly where the size and speed win comes from — and exactly why the ring's extra structure is a trade-off: it buys efficiency at the cost of a stronger (though still, so far, unbroken) hardness assumption than plain LWE, since the extra algebraic structure is, in principle, more surface area for a future attack to exploit.

2.8 Module-LWE: the Middle Ground Kyber and Dilithium Actually Use

Definition: Module Learning With Errors (MLWE)

Fix R_q as above and a small integer k (the **module rank**). The secret is now a length-k vector of ring elements, $\vec{s} \in R_q^k$. A sample is $(\vec{a}, b)$ with $\vec{a} \in R_q^k$ uniform random and

$$b = \langle \vec{a}, \vec{s} \rangle + e \pmod{q} = \sum_{i=1}^{k} a_i s_i + e, \qquad e \leftarrow \chi.$$

Remark: MLWE Sits Exactly Between Plain LWE and Ring-LWE

Set $k = 1$ and MLWE collapses to Ring-LWE exactly. Let $n = 1$ (no ring structure at all — just plain integers) and MLWE collapses toward plain LWE. Choosing $k > 1$ with a modest per-ring dimension n gives designers a dial: more of the speed benefit of rings, without committing to the single largest, most algebraically rigid ring dimension. This is precisely why Kyber and Dilithium use MLWE with a moderate fixed $n = 256$ and vary k (Kyber uses $k \in \{2, 3, 4\}$ for its three security levels) instead of varying n directly — a design choice we will see in full in Session 3.

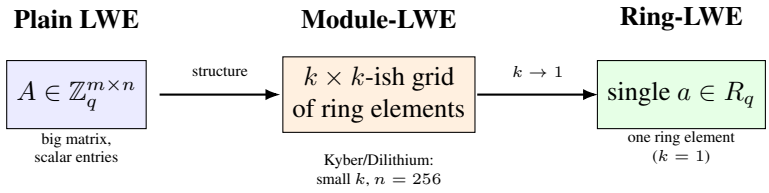


Figure 2.9: Module-LWE is a size/speed dial between the two extremes: no ring structure at all (plain LWE, left) and maximal ring structure (Ring-LWE, right). Kyber and Dilithium sit in the middle deliberately.

2.9 Bridge to Lecture 3: A Real Public Key *Is* an (M)LWE Sample

When we open up Kyber (ML-KEM) in Lecture 3, its key generation will look almost exactly like the MLWE definition above: pick a random public matrix (of ring elements) $\mathbf{A}$, pick a small secret $\vec{s}$ and small error $\vec{e}$, and publish $\mathbf{t} = \mathbf{A}\vec{s} + \vec{e}$ as the public key — literally an MLWE sample, with $\vec{s}$ kept as the secret key. Encryption uses a *second*, independent MLWE-style sample to hide the message inside more noise. Once you can point at a real Kyber public key and say "that's $b = As + e$," the scheme stops looking like magic and starts looking like an application of exactly the object defined in Section 2.8.

2.10 Summary Table

Problem	Given	Find / Decide
SVP	Basis B of lattice L	Shortest nonzero $v \in L$
CVP	Basis B, target $t \notin L$	Closest $v \in L$ to t
SIS	Random $A \in \mathbb{Z}_q^{n \times m}$	Short nonzero x: $Ax \equiv 0$
SIS hash $h_A(x) = Ax$	Same A, input $x \in \{0,1\}^m$	Compressed output in $\mathbb{Z}_q^n$; collisions $\Rightarrow$ SIS solutions
Search-LWE	$(A, b = As + e)$	Recover secret s
Decision-LWE	(A, b)	Is b "$As + e$" or uniform random?
Ring-LWE	$(a, b = as + e) \in R_q^2$	Recover $s \in R_q$
Module-LWE	$(\vec{a}, b) \in R_q^k \times R_q$	Recover $\vec{s} \in R_q^k$

2.11 Full List of Exercises (Assign as Homework Before Lecture 3)

Exercise: Homework Set

1. State, in your own words (2–3 sentences each), the difference between SVP and CVP, and the difference between the search and decision versions of a problem in general.
2. For $n = 1$, $m = 3$, $q = 13$, $A = \begin{pmatrix} 4 & 6 & 9 \end{pmatrix}$: find a nonzero, short $x \in \mathbb{Z}^3$ solving the SIS instance $Ax \equiv 0 \pmod{13}$ (Section 2.5 worked example, above). Confirm your answer by direct computation.
3. Complete the SIS-hash practice exercise in Section 2.5.1, if you have not already: find a genuine colliding pair for h_A and confirm it yields a valid SIS solution.
4. Redo the search-LWE worked example (Section 2.6.4) with secret $s = (2, 9, 1)$ but a *new* sample $a_5 = (2, 0, 1)$ and error $e_5 = 1$. Compute b_5. If you were only given a_5 and b_5 (not s or e_5), could you determine s from this single equation alone? Why not?
5. Explain in your own words why LWE would become *easy* (not hard) if the error e were removed entirely (i.e., $b = As$ exactly). Connect your answer to ordinary linear algebra.
6. In the ring $R_{17} = \mathbb{Z}_{17}[x]/(x^4 + 1)$: compute the product $(2 + x^2)(1 + 3x)$, showing the wraparound step explicitly (Section 2.7.2 worked example, above).
7. In 3–4 sentences: why does moving from plain LWE to Ring-LWE reduce key size roughly from $O(n^2)$ numbers to $O(n)$ numbers? Use the "one Ring-LWE sample $\approx n$ plain-LWE samples" remark (Section 2.7.3) in your explanation.
8. Module-LWE with $k = 1$ is exactly Ring-LWE, and (informally) R_q with $n = 1$ is exactly $\mathbb{Z}_q$. Using these two facts, explain why Module-LWE is fairly described as "a generalization that contains both plain LWE and Ring-LWE as special cases."
9. **Looking ahead:** Kyber-768 uses module rank $k = 3$ with per-ring dimension $n = 256$ and $q = 3329$. Without looking anything up yet, estimate how many total $\mathbb{Z}_q$-coefficients are packed into one Kyber-768 secret key $\vec{s} \in R_q^3$. (You'll check this answer in Session 3.)

Lesson 3: ML-KEM (Kyber) — A Real Scheme, End to End

Parameters, Compression, CPA-Secure Encryption, and the Route to CCA Security

This lecture makes Module-LWE concrete. We fix parameters, define the compression functions Kyber's ciphertexts use, specify the core encryption scheme — Kyber.CPAPKE — algorithm by algorithm, prove that decryption recovers the message, work through the scheme by hand, and then apply the Fujisaki–Okamoto transform that turns Kyber.CPAPKE into the standardized, chosen-ciphertext-secure key encapsulation mechanism **ML-KEM (FIPS 203)**.

3.1 Notation and Parameters

Module-LWE (Lecture 2, Section 2.8) states that, for a small integer k and $R_q = \mathbb{Z}_q[x]/(x^n+1)$, the pair $(\vec{a}, b = \langle \vec{a}, \vec{s} \rangle + e)$ is computationally indistinguishable from uniform random, for secret $\vec{s} \in R_q^k$ and small error e. ML-KEM (Kyber) is this object made concrete: $\vec{a}$ becomes a public $k \times k$ matrix A of ring elements (one MLWE equation per row), $\vec{s}$ becomes the Kyber secret key, and $\vec{t} = A\vec{s} + \vec{e}$ becomes the Kyber public key.

The "/" in $R_q = \mathbb{Z}_q[x]/(x^n+1)$ is a *quotient*, the same construction as $\mathbb{Z}_n = \mathbb{Z}/n\mathbb{Z}$ from Lecture 1, Section 1.3.2 — it has nothing to do with set subtraction ($\{a,b,c\} \setminus \{c\}$). $\mathbb{Z}/n\mathbb{Z}$ declares $n \equiv 0$ and reduces every integer using that fact, leaving the remainders $\{0, \ldots, n-1\}$ as canonical representatives; $\mathbb{Z}_q[x]/(x^n+1)$ does the same thing one level up, with polynomials in place of integers. It declares $x^n + 1 \equiv 0$ — equivalently $x^n \equiv -1$ — and reduces every polynomial product using that rule, leaving the degree-$<n$ polynomials as canonical representatives; two polynomials are considered equal exactly when their difference is a multiple of (x^n+1), just as two integers are equal mod n exactly when their difference is a multiple of n. This is precisely the "$x^n \equiv -1$" wraparound step used in every ring-multiplication example throughout this lecture: it is not an ad hoc trick, it is what the quotient notation means.

The table below fixes the correspondence between Lecture 2's generic notation and the names used from here on.

Lecture 2's MLWE object	**Kyber's name for it**
Module rank k	Same, k — fixed per parameter set
Ring R_q, dimension n	Same, $n = 256$ for every ML-KEM parameter set
Public matrix $A \in R_q^{k \times k}$	Same letter, A — expanded from a short seed ρ
Secret $\vec{s} \in R_q^k$	Same letter, $\vec{s}$ — the Kyber secret key
Error $\vec{e} \in R_q^k$	Same letter, $\vec{e}$ — used only during KeyGen
Sample $b = A\vec{s} + \vec{e}$	Renamed $\vec{t} = A\vec{s} + \vec{e}$ — the Kyber public key, alongside A

Encryption introduces one further object: a second, independent MLWE sample $\vec{u}$, built the same way as $\vec{t}$ but from a fresh secret $\vec{r}$ that only the encryptor ever knows.

3.1.1 Parameter Sets

All three standardized parameter sets share $n = 256$ and $q = 3329$, a prime satisfying $q \equiv 1 \pmod{256}$ (indeed $3329 = 13 \times 256 + 1$). This congruence is what makes the Number-Theoretic Transform (NTT, a finite-field analogue of the FFT) applicable to multiplication in R_q — the reason q takes this

specific value rather than a rounder one. The parameter sets differ only in the module rank k and the noise parameters η_1, η_2, which control the width of the centered binomial noise distribution (Lecture 2, Section 2.6.3).

Parameter set	k	η_1	η_2	Public-key size	Target security
ML-KEM-512	2	3	2	800 bytes	AES-128
ML-KEM-768	3	2	2	1,184 bytes	AES-192 (NIST default)
ML-KEM-1024	4	2	2	1,568 bytes	AES-256

n is held fixed and k is varied instead, for two reasons. First, q and the NTT machinery are tuned to this specific ring; changing n per security level would mean re-deriving that machinery three times over. Second, k is a much finer security knob than n would be: Lecture 2's "MLWE dial" (Figure 2.9) lets $k \in \{2, 3, 4\}$ scale security in small, well-understood steps, rather than jumping between a handful of practical ring dimensions.

3.1.2 Object Shapes

Every quantity below is either a $k \times k$ matrix of ring elements, a length-k vector of ring elements, or a single ring element — never a plain scalar in $\mathbb{Z}_q$. It is worth being explicit that k and n enter in different places: k counts *how many ring elements* are stacked into a vector or matrix, while n is the number of $\mathbb{Z}_q$-coefficients carried *inside* each individual ring element. A public matrix of module rank k is therefore $k \times k$, not $k \times n$: each of its k^2 entries is already a full degree-$<n$ polynomial, so n never appears as an outer matrix dimension at all — it is already accounted for inside every entry. The table below lists the shape of every object used from here on.

Object	**Shape**	**Meaning**
A	$k \times k$ matrix of ring elements	Public, random (expanded from seed ρ)
$\vec{s}, \vec{e}$	length-k vectors of ring elements	Secret key and its keygen noise
$\vec{t}$	length-k vector of ring elements	Public key (alongside A)
$\vec{r}, \vec{e}_1$	length-k vectors of ring elements	Fresh, per-encryption randomness/noise
e_2	a single ring element	Fresh, per-encryption scalar noise
$\vec{u}$	length-k vector of ring elements	First half of the ciphertext
v	a single ring element	Second half of the ciphertext (carries the message)
m	a bit string of length $n = 256$	The plaintext, one bit per coefficient

Matrix–vector and vector–vector products follow ordinary linear-algebra shape rules throughout, with "number" replaced by "ring element" and ordinary multiplication replaced by polynomial multiplication in R_q: a $k \times k$ matrix times a length-k vector gives a length-k vector ($A\vec{s}$); a length-k vector transposed times a length-k vector gives a single ring element ($\vec{t}^T\vec{r}$). When $k = 1$, every vector and matrix below collapses to a single ring element, which is exactly why the $k = 1$ worked example in Section 3.5 is simpler than the general scheme; Section 3.5.1 redoes the same computation with $k = 2$ to show the vector structure explicitly.

3.2 Compression

A ciphertext built from full elements of R_q would store every one of its $n = 256$ coefficients at full precision modulo $q = 3329$, about 12 bits each. Kyber shrinks the *ciphertext* — not the public key, which is sent once per session and whose size is already fixed by k and n alone — by deliberately discarding low-order bits of precision, in a controlled way.

Definition: Compression and Decompression

For an integer $d < \lceil \log_2 q \rceil$, define, for $x \in \mathbb{Z}_q$:

$$\text{Compress}_d(x) = \left\lceil \frac{2^d}{q} \cdot x \right\rfloor \bmod 2^d, \qquad \text{Decompress}_d(y) = \left\lceil \frac{q}{2^d} \cdot y \right\rfloor,$$

where $\lceil \cdot \rfloor$ denotes rounding to the nearest integer. Compression maps a $\mathbb{Z}_q$ value down to a d-bit value; decompression maps it back up to an approximation of the original. Applied coefficient-by-coefficient, both extend to an entire ring element, and further to an entire vector of ring elements.

Example: Compression by Hand, $q = 17$

Take $d = 2$ (values squeeze into $\{0, 1, 2, 3\}$) and $x = 9$: $\text{Compress}_2(9) = \lceil \frac{4}{17} \cdot 9 \rfloor = \lceil 2.12 \rfloor = 2$, and $\text{Decompress}_2(2) = \lceil \frac{17}{4} \cdot 2 \rfloor = \lceil 8.5 \rfloor = 9$ — landing back exactly on the original.
Now compare $d = 2$ against $d = 3$ for $x = 6$. With $d = 2$: $\text{Compress}_2(6) = \lceil \frac{4}{17} \cdot 6 \rfloor = \lceil 1.41 \rfloor = 1$, and $\text{Decompress}_2(1) = \lceil \frac{17}{4} \rfloor = 4$ — a round-trip error of $|6 - 4| = 2$. With $d = 3$: $\text{Compress}_3(6) = \lceil \frac{8}{17} \cdot 6 \rfloor = \lceil 2.82 \rfloor = 3$, and $\text{Decompress}_3(3) = \lceil \frac{17}{8} \cdot 3 \rfloor = \lceil 6.375 \rfloor = 6$ — exact. Doubling the number of representable levels (from 4 to 8) roughly halved the worst-case rounding error, exactly as expected from halving the gap between adjacent representable values.

Adding rounding error on purpose seems to fight Lecture 2's central point, that noise is what makes LWE hard. The resolution is a fixed budget, made precise in Section 3.4: decryption succeeds as long as the *total* noise on a coefficient — the original LWE-style noise from KeyGen and encryption, plus whatever compression adds — stays under $q/4$ in absolute value. Compression spends a small, carefully bounded slice of that budget in exchange for a large reduction in ciphertext size; push d too low and decryption starts failing measurably. Real ML-KEM uses two different depths: d_u for $\vec{u}$ (10 or 11 bits, depending on the parameter set) and a smaller d_v for v (4 or 5 bits). $\vec{u}$ needs more bits because any error in it is multiplied by $\vec{s}$ during decryption (Section 3.4); v tolerates a coarser encoding because it is only ever compared against the two widely separated targets 0 and $\lceil q/2 \rceil$. The worked examples from Section 3.5 onward skip compression entirely, to keep the hand arithmetic focused on the cancellation argument itself.

3.3 The CPA-Secure Scheme: Kyber.CPAPKE

CPA stands for **chosen-plaintext attack** (Section 3.6 gives the full threat-model definition, alongside its stronger counterpart, *CCA*, chosen-ciphertext attack); "CPA-secure" means secure against an attacker limited to that weaker threat model. We now write out the scheme itself, at the module level, using Lecture 2's notation directly. A^T denotes the transpose of the matrix of ring elements, and all arithmetic below is in R_q.

Algorithm 1 Kyber.CPAPKE.KeyGen

1: Generate $A \in R_q^{k \times k}$ uniformly at random (in practice: expanded deterministically from a short public seed ρ, so only ρ needs to be stored or sent)
2: Sample $\vec{s} \in R_q^k$, each coefficient from the small noise distribution (centered binomial, parameter η_1)
3: Sample $\vec{e} \in R_q^k$ the same way
4: Compute $\vec{t} = A\vec{s} + \vec{e} \in R_q^k$
5: **return** public key $pk = (A, \vec{t})$, secret key $sk = \vec{s}$

A is exactly the random matrix every MLWE instance needs (Lecture 2, Section 2.8) and may be made fully public with no loss of security. $\vec{s}$ and $\vec{e}$ are the two quantities that must never become public, both drawn from a distribution deliberately much narrower than the full range $\{0, \ldots, q-1\}$. Line 4 is an

MLWE sample, and it is the only line that touches both A and $\vec{s}$ together — everything in Enc and Dec is built to eventually cancel this exact term. Note that $\vec{e}$ itself is discarded after KeyGen: once it is baked into $\vec{t}$, only $\vec{s}$ needs to survive.

Algorithm 2 Kyber.CPAPKE.Enc$(pk = (A, \vec{t}),\ m \in \{0,1\}^n)$

1: Sample $\vec{r} \in R_q^k$, $\vec{e}_1 \in R_q^k$ from the small noise distribution (parameter η_1), and $e_2 \in R_q$ (parameter η_2)
2: Compute $\vec{u} = A^T\vec{r} + \vec{e}_1 \in R_q^k$
3: Encode the message: $\mathrm{Encode}(m) \in R_q$ has i-th coefficient $\lceil q/2 \rceil \cdot m_i$, so each of the n message bits becomes one coefficient, set to either 0 or $\lceil q/2 \rceil$
4: Compute $v = \vec{t}^T\vec{r} + e_2 + \mathrm{Encode}(m) \in R_q$
5: **return** ciphertext $c = \big(\mathrm{Compress}_{d_u}(\vec{u}),\ \mathrm{Compress}_{d_v}(v)\big)$

Every quantity sampled in line 1 is fresh, generated independently for this one encryption and never reused. Line 2 builds $\vec{u}$, a second MLWE sample structurally identical to KeyGen's $\vec{t} = A\vec{s} + \vec{e}$, with A^T in place of A and the fresh $\vec{r}, \vec{e}_1$ in place of $\vec{s}, \vec{e}$. Line 3 places the message on the two points of $\mathbb{Z}_q$'s "clock face" (Lecture 1, Section 1.6) that sit farthest apart, 0 and $\lceil q/2 \rceil$ — this is what gives the decoder the most room to tolerate noise. Line 4 hides $\mathrm{Encode}(m)$ inside $\vec{t}^T\vec{r}$; to anyone without $\vec{s}$, $\vec{t}^T\vec{r}$ is computationally indistinguishable from uniform random, which is exactly MLWE's hardness guarantee, so v reveals nothing about m without the secret key.

It is worth asking directly why the ciphertext needs two parts, $\vec{u}$ and v, rather than v alone. A itself is never hidden — it is public throughout — so $\vec{u}$ is not hiding A. What it hides is $\vec{r}$, the encryptor's own fresh secret. Without $\vec{u}$, a recipient given only $v = \vec{t}^T\vec{r} + e_2 + \mathrm{Encode}(m)$ would face one equation in two unknowns, m and $\vec{r}$, with no way to separate them: $\vec{t}$ is known, but $\vec{r}$ was never sent, and $\vec{t}^T\vec{r}$ sits on top of the message as an unremovable mask. Sending $\vec{r}$ directly would fix that but break security completely, since $\vec{t}$ is public and anyone could then compute $\vec{t}^T\vec{r}$ and peel it straight off v. $\vec{u}$ resolves both problems at once: it hides $\vec{r}$ behind the same MLWE hardness that protects $\vec{s}$, while reusing the same public matrix A that KeyGen used to build $\vec{t}$. That reuse is exactly what lets the two copies of A cancel during decryption (Section 3.4), giving the decryptor — and only the decryptor — a path back to m. The construction is a Diffie–Hellman-style exchange in MLWE clothing: $\vec{t}$ is the recipient's long-term public share, $\vec{u}$ is the encryptor's one-time public share, and each side can compute its own approximation of the same masked quantity without ever transmitting $\vec{s}$ or $\vec{r}$. Using a fresh $\vec{r}$ on every call also makes the scheme probabilistic — encrypting the same m twice yields different ciphertexts — which is a requirement of any secure public-key scheme, not an optional refinement: with deterministic encryption, an attacker could guess a message, re-encrypt it, and compare the result against a target ciphertext directly.

Algorithm 3 Kyber.CPAPKE.Dec$(sk = \vec{s},\ c = (c_u, c_v))$

1: Recover $\vec{u} = \mathrm{Decompress}_{d_u}(c_u)$, $v = \mathrm{Decompress}_{d_v}(c_v)$
2: Compute $m' = v - \vec{s}^T\vec{u} \in R_q$
3: **return** $\mathrm{Decode}(m')$: for each coefficient of m', output bit 1 if it is closer to $\lceil q/2 \rceil$ than to 0 (mod q), else bit 0

Algorithms 1–3 describe three separate computations; Figure 3.1 puts them back together into the two-party exchange they actually form, showing which party runs which algorithm and exactly what crosses the wire between them.

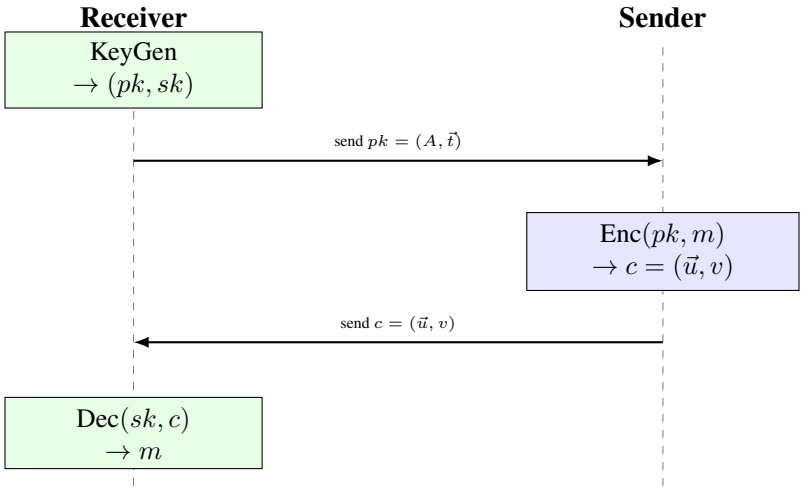


Figure 3.1: The CPA protocol as an exchange between two parties. The Receiver runs KeyGen once and publishes $pk = (A, \vec{t})$; $sk = \vec{s}$ never leaves the Receiver. The Sender runs Enc using only pk and a message m, and sends the resulting ciphertext c back; m and the fresh randomness $\vec{r}, \vec{e}_1, e_2$ used to build c never leave the Sender. Only the Receiver, holding $\vec{s}$, can turn c back into m.

3.4 Correctness of Decryption

Substituting the definitions of $\vec{u}$ and v into $m' = v - \vec{s}^T\vec{u}$ gives

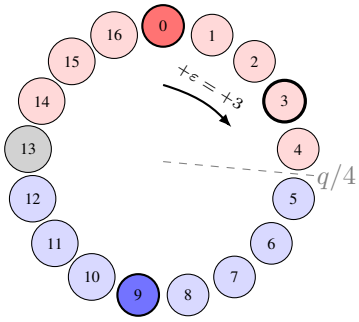
$$\begin{aligned} m' &= \big(\vec{t}^T\vec{r} + e_2 + \mathrm{Encode}(m)\big) - \vec{s}^T\big(A^T\vec{r} + \vec{e}_1\big) \\ &= \vec{t}^T\vec{r} - \vec{s}^T A^T\vec{r} + e_2 - \vec{s}^T\vec{e}_1 + \mathrm{Encode}(m) \\ &= (A\vec{s} + \vec{e})^T\vec{r} - \vec{s}^T A^T\vec{r} + e_2 - \vec{s}^T\vec{e}_1 + \mathrm{Encode}(m) \qquad \text{(substituting } \vec{t} = A\vec{s} + \vec{e}\text{)} \\ &= \vec{s}^T A^T\vec{r} + \vec{e}^T\vec{r} - \vec{s}^T A^T\vec{r} + e_2 - \vec{s}^T\vec{e}_1 + \mathrm{Encode}(m) \\ &= \mathrm{Encode}(m) + \underbrace{\big(\vec{e}^T\vec{r} + e_2 - \vec{s}^T\vec{e}_1\big)}_{\text{total noise, call it } \varepsilon}. \end{aligned}$$

The two copies of $\vec{s}^T A^T\vec{r}$ cancel exactly, using $(A\vec{s})^T = \vec{s}^T A^T$; this cancellation is why the scheme is built around A appearing on both sides, once via $\vec{t}$ and once via $\vec{u}$. Only whoever knows $\vec{s}$ can line the two copies up and subtract them — an eavesdropper who sees only $A, \vec{t}, \vec{u}, v$ has no way to perform this cancellation, which is exactly the asymmetry a public-key scheme needs (Lecture 2, Section 2.1). What remains is the message plus a total noise term ε, built entirely from small quantities.

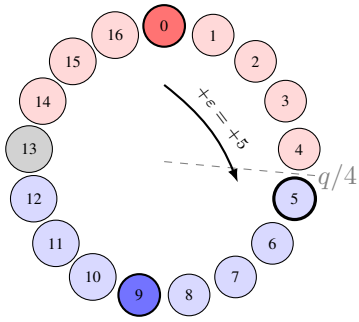
3.4.1 The Noise Budget

Encode places every message bit on one of two poles of $\mathbb{Z}_q$, 0 for bit 0 and $\lceil q/2 \rceil$ for bit 1 — as far apart as the modulus allows. Decode simply asks which pole a coefficient is closer to. The two points exactly midway between the poles, at $q/4$ and $3q/4$, are the decision boundaries: crossing either one flips which bit Decode reports. A coefficient encoding bit 0 therefore still decodes correctly as long as its noise ε_i satisfies $|\varepsilon_i| < q/4$, keeping it on the near side of both boundaries; by the same symmetric argument around the other pole, a coefficient encoding bit 1 tolerates the same margin. The bound $q/4$ is thus not an arbitrary safety number — it is exactly half the gap between the two encoding poles. Once every coefficient of ε stays under this bound, Decode recovers m exactly. Figure 3.2 shows this on the toy ring $q = 17$ used throughout this lecture's worked examples, where the two poles are 0 and $\lceil 17/2 \rceil = 9$ and the boundaries sit at $q/4 = 4.25$ and $3q/4 = 12.75$.

Real parameters stay far from this edge. Take ML-KEM-512 ($k = 2$, $\eta_1 = 3$, $\eta_2 = 2$, $q = 3329$, so $q/4 \approx 832$). Each coefficient of $\vec{e}, \vec{r}, \vec{e}_1$ is bounded by $\pm\eta_1 = \pm 3$, and e_2 by $\pm\eta_2 = \pm 2$, by construction



Decodes correctly. $m'_i = 3$: distance to the bit-0 pole (0) is 3; distance to the bit-1 pole (9) is 6. Still closer to 0 — Decode outputs 0, since $|\varepsilon| = 3 < q/4 \approx 4.25$.



Decodes incorrectly. $m'_i = 5$: distance to the bit-0 pole (0) is 5; distance to the bit-1 pole (9) is only 4. Now closer to 9 — Decode outputs 1, a decryption failure on this coefficient, because $|\varepsilon| = 5 > q/4 \approx 4.25$ crossed the dashed boundary.

Figure 3.2: Decoding a true bit-0 coefficient on $\mathbb{Z}_{17}$. Red nodes decode to 0, blue nodes decode to 1, gray is the exact tie at 13; the bold red and blue nodes are Encode's two poles, 0 and 9. Both panels start at the same pole and add noise ε — the only difference is whether ε stays under the dashed $q/4$ boundary (left) or crosses it (right).

of the centered binomial distribution. Each entry of $\varepsilon = \vec{e}^T\vec{r} + e_2 - \vec{s}^T\vec{e}_1$ sums $n = 256$ such small products across $k = 2$ ring multiplications; summing hundreds of small, roughly independent, roughly zero-mean terms produces a typical magnitude that grows only with the square root of the number of terms, landing in the tens or low hundreds — comfortably inside the $q/4 \approx 832$ margin. This concentration behavior is exactly why ML-KEM's per-ciphertext decryption-failure probability is kept below 2^{-100}: the parameters $n, k, \eta_1, \eta_2, d_u, d_v$ are chosen with this square-root growth explicitly in mind. If ε's coefficients happen to land unusually far from 0 — an accepted, carefully bounded event, not a flaw — decryption outputs the wrong bit; some physical attacks work by deliberately inducing this failure and using the resulting pattern to leak information about $\vec{s}$ (Lesson 5).

The worked examples below use $n = 4$ and single-digit noise, so every number can be checked by hand; they demonstrate the algebraic cancellation exactly, but are too small to show the square-root concentration argument above, which needs the real $n = 256$.

3.5 A Worked Example

Example: Kyber-Style Encryption and Decryption, $k = 1$, $n = 4$, $q = 17$

Take $R_{17} = \mathbb{Z}_{17}[x]/(x^4 + 1)$ and module rank $k = 1$, so every object below is a single ring element rather than a vector — this is Ring-LWE-flavored Kyber, kept at $k = 1$ so the arithmetic stays hand-tractable. Section 3.5.1 redoes the computation with $k = 2$ to restore the vector structure.

KeyGen. Let $A = 3 + 5x + 2x^2 + x^3$ (public), $s = 1 - x$ (secret), $e = 1$ (keygen noise). Then, using $x^4 \equiv -1$,

$$A \cdot s = 4 + 2x + 14x^2 + 16x^3, \qquad t = A \cdot s + e = 5 + 2x + 14x^2 + 16x^3.$$

Public key: (A, t). Secret key: $s = 1 - x$.

Enc. Message $m = (1, 0, 1, 0)$, so $\text{Encode}(m) = 9 + 0x + 9x^2 + 0x^3$ (using $\lceil 17/2 \rceil = 9$). Fresh randomness $r = x$, $e_1 = 1$, $e_2 = -1$. Compute

$$u = A \cdot r + e_1 = 3x + 5x^2 + 2x^3 + x^4 + 1 = 0 + 3x + 5x^2 + 2x^3,$$

$$v = t \cdot r + e_2 + \text{Encode}(m) = (1 + 5x + 2x^2 + 14x^3) + (-1) + (9 + 0x + 9x^2 + 0x^3) = 9 + 5x + 11x^2 + 14x^3.$$

(Compression is skipped, as in Section 3.2.) Ciphertext: $c = (u, v)$.

Dec. Compute $s \cdot u = 2 + 3x + 2x^2 + 14x^3$, then

$$m' = v - s \cdot u = (9 - 2) + (5 - 3)x + (11 - 2)x^2 + (14 - 14)x^3 = 7 + 2x + 9x^2 + 0x^3.$$

Decoding each coefficient (closer to 9 than to 0 means bit 1): $7 \to 1$, $2 \to 0$, $9 \to 1$, $0 \to 0$. **Recovered message:** $(1, 0, 1, 0)$ **— exactly the original.**

As a check on Section 3.4's derivation, ε can be computed two independent ways and must agree. Directly: $m' - \text{Encode}(m) = (7 + 2x + 9x^2) - (9 + 9x^2) = -2 + 2x \equiv 15 + 2x \pmod{17}$. Via the formula ($\varepsilon = e \cdot r + e_2 - s \cdot e_1$, with no transposes needed since $k = 1$): $e \cdot r = x$, $e_2 = -1$, $s \cdot e_1 = 1 - x$, so $\varepsilon = x - 1 - (1 - x) = 2x - 2 \equiv 15 + 2x \pmod{17}$. Both routes agree.

3.5.1 A Second Worked Example ($k = 2$)

Because $k = 1$, the example above never exercises the vector-of-ring-elements structure that real ML-KEM depends on. This example redoes the computation with $k = 2$, producing a genuine matrix–vector product, inner product, and transpose.

Example: Kyber-Style Encryption and Decryption, $k = 2$, $n = 4$, $q = 17$

Still $R_{17} = \mathbb{Z}_{17}[x]/(x^4 + 1)$, now with $k = 2$ and a public matrix with monomial entries,

$$A = \begin{pmatrix} 1 & x \\ x^2 & 1 \end{pmatrix} \in R_{17}^{2\times 2}.$$

KeyGen. Secret $\vec{s} = (1 + x,\ 2)$, keygen noise $\vec{e} = (1,\ 16)$ (i.e. $(1, -1)$). Then

$$\begin{aligned} t_1 &= 1 \cdot (1 + x) + x \cdot 2 + 1 = 2 + 3x, \\ t_2 &= x^2 \cdot (1 + x) + 1 \cdot 2 + 16 = x^2 + x^3 + 18 \equiv 1 + x^2 + x^3 \pmod{17}. \end{aligned}$$

Public key: $(A, \vec{t}) = (A,\ 2 + 3x,\ 1 + x^2 + x^3)$. Secret key: $\vec{s} = (1 + x,\ 2)$.
Enc. Message $m = (1, 1, 0, 0)$, so $\text{Encode}(m) = 9 + 9x$. Fresh randomness $\vec{r} = (1,\ x)$, $\vec{e}_1 = (0,\ 1)$, $e_2 = 0$. With $A^T = \begin{pmatrix} 1 & x^2 \\ x & 1 \end{pmatrix}$,

$$\begin{aligned} u_1 = 1 \cdot 1 + x^2 \cdot x + 0 = 1 + x^3, \qquad u_2 = x \cdot 1 + 1 \cdot x + 1 = 1 + 2x, \\ t_1 r_1 + t_2 r_2 = (2 + 3x) + (1 + x^2 + x^3) \cdot x = (2 + 3x) + (16 + x + x^3) \equiv 1 + 4x + x^3 \pmod{17}, \\ v = (1 + 4x + x^3) + 0 + (9 + 9x) = 10 + 13x + x^3. \end{aligned}$$

Ciphertext: $c = (\vec{u}, v) = \big((1 + x^3,\ 1 + 2x),\ 10 + 13x + x^3\big)$.
Dec. $\vec{s}^T\vec{u} = (1+x)(1+x^3) + 2(1+2x) = (x + x^3) + (2 + 4x) = 2 + 5x + x^3$ (using $x^4 \equiv -1$ to cancel the two constant ± 1's in the first product). Then

$$m' = v - \vec{s}^T\vec{u} = (10 + 13x + x^3) - (2 + 5x + x^3) = 8 + 8x.$$

Decoding: $8 \to 1$, $8 \to 1$, $0 \to 0$, $0 \to 0$. **Recovered message:** $(1, 1, 0, 0)$ **— exactly the original.**
Checking $\varepsilon = \vec{e}^T\vec{r} + e_2 - \vec{s}^T\vec{e}_1$ against direct subtraction: directly, $m' - \text{Encode}(m) = (8 + 8x) - (9 + 9x) = -1 - x \equiv 16 + 16x$; via the formula, $\vec{e}^T\vec{r} = 1 \cdot 1 + 16 \cdot x = 1 + 16x$, $e_2 = 0$, $\vec{s}^T\vec{e}_1 = (1 + x) \cdot 0 + 2 \cdot 1 = 2$, giving $\varepsilon = (1 + 16x) - 2 \equiv 16 + 16x$. Both agree, this time genuinely exercising the vector inner products $\vec{e}^T\vec{r}$ and $\vec{s}^T\vec{e}_1$.

What changes going from $k = 1$ to $k = 2$ is only the bookkeeping: A becomes a real 2×2 matrix, $\vec{t}, \vec{u}, \vec{s}^T\vec{u}$ each become genuine sums of two ring products, and A^T genuinely swaps entries. The cancellation argument, the shape of Encode/Decode, and the outcome are unchanged — Section 3.4's derivation was already written for general k, and this example is the first place that generality becomes visible.

3.6 From CPA-Secure Encryption to a CCA-Secure Key Encapsulation Mechanism

Kyber.CPAPKE, exactly as just described, carries a specific security guarantee and no more: it is secure against a **chosen-plaintext attack (CPA)**, meaning an attacker who only ever sees ciphertexts produced by honestly encrypting messages of their choosing, with no way to submit a ciphertext for decryption and observe the result. Real protocols face a stronger attacker. A **chosen-ciphertext attack (CCA)** additionally lets the attacker submit arbitrary, possibly malformed ciphertexts to a decryption function and observe what comes back — a successful result, an error, a timing difference, or a side-channel trace. This is not specifically about a man-in-the-middle position on the network; any setting in which a decryption function is reachable by an untrusted party creates this threat — a server decrypting client-submitted key material, a device an attacker has physical access to, or a network position that lets an

attacker inject crafted ciphertexts. Essentially every real deployment gives an attacker some version of this access, which is why CCA security, not CPA security, is the baseline every real protocol (TLS, SSH) requires.

Kyber.CPAPKE does not meet this stronger requirement on its own; an attacker who can submit chosen ciphertexts to a decryption oracle and observe even coarse information about the result can, in general, recover the secret key (Lesson 5 works through this in detail). ML-KEM closes this gap through a generic transformation applied on top of Kyber.CPAPKE — not a redesign of the algebra in Section 3.3. The table below summarizes the three distinct objects involved, since they are easy to conflate.

Object	**Role**
Kyber.CPAPKE	The underlying encryption primitive (Section 3.3); CPA-secure only
Fujisaki–Okamoto (FO) transform	The generic recipe that upgrades a CPA-secure PKE into a CCA-secure KEM
ML-KEM	The standardized result: Kyber.CPAPKE run through the FO transform; CCA-secure

Definition: Key Encapsulation Mechanism (KEM)

A KEM replaces "encrypt an arbitrary message" with a narrower interface: $\mathrm{Encaps}(pk)$ outputs both a ciphertext c and a fresh shared secret K; $\mathrm{Decaps}(sk, c)$ recovers the same K. Neither party chooses K directly — it is generated as a by-product of encapsulation, and is then used as a symmetric key for the remainder of the session.

This narrowing is deliberate, and it is exactly what makes the FO transform possible. Plain public-key encryption lets a sender encrypt any message they like, which is precisely the freedom a CCA attacker exploits by choosing (or influencing) that message adversarially. A KEM removes the freedom at the interface level: nobody, not even the honest sender, ever chooses K directly; it falls out as a side effect of a randomized process the sender does not fully control.

Algorithm 4 ML-KEM.Encaps$(pk = (A, \vec{t}))$

1: Sample a fresh random message $m \in \{0,1\}^n$
2: Derive randomness $(\vec{r}, \vec{e}_1, e_2)$ and shared secret K from m (and pk) via a hash function
3: Compute $c \leftarrow$ Kyber.CPAPKE.Enc(pk, m) using the derived randomness
4: **return** ciphertext c, shared secret K

Algorithm 5 ML-KEM.Decaps$(sk = \vec{s},\ c)$

1: Compute $m' \leftarrow$ Kyber.CPAPKE.Dec(sk, c)
2: Re-derive randomness from m' exactly as Encaps would, and recompute $c' \leftarrow$ Kyber.CPAPKE.Enc(pk, m') using it
3: **if** $c' = c$ **then**
4: **return** $K \leftarrow H(m')$
5: **else**
6: **return** $K \leftarrow H(sk, c)$ ▷ implicit rejection
7: **end if**

The idea is to stop encrypting a message the attacker can choose or influence, and instead encrypt a message only the honest sender ever knows, in a way the receiver can independently double-check. Encaps generates a fresh random m, derives both the encryption randomness and the final key K from m by hashing, and encrypts m with Kyber.CPAPKE. Decaps does something a plain decryption function does not: after recovering a candidate m', it re-encrypts m' with the same derived randomness and

checks whether the result reproduces the submitted ciphertext c exactly. If it does, c is one an honest sender could genuinely have produced, and K is derived from m'. If it does not, Decaps does not report an error; it instead returns a different, pseudorandom-looking key K, derived from the secret key and c together (*implicit rejection*), indistinguishable from a legitimate key to an outside observer. Figure 3.3 shows this schematically.

Remark: Why Not Just Return $H(m')$ Directly?

Line 2's re-encryption check and line 6's implicit rejection both add real cost — a full extra CPAPKE.Enc call on every single Decaps invocation — so it is worth asking directly why Decaps does not simply skip both and return $K \leftarrow H(m')$ unconditionally, straight off CPAPKE.Dec's output. The answer is that an m' decrypted from an arbitrary, attacker-chosen ciphertext is not trustworthy: it is exactly the quantity a chosen-ciphertext attacker manipulates to mount the boundary-sweep key-recovery attack Lesson 5 works through in full, by submitting many crafted ciphertexts and learning something about $\vec{s}$ from whatever comes back. Without the check, that "whatever comes back" is $K = H(m')$ itself — and if K is ever used downstream (e.g. in a MAC that later succeeds or fails), the attacker regains exactly the decryption oracle Kyber.CPAPKE was never designed to survive. Line 2 closes that gap by refusing to trust m' until it has been verified to be one that an honest Encaps run could actually have produced. Line 6 then closes the gap the check itself would otherwise open: if Decaps returned a visibly different *kind* of output on failure (an error, a zero, a shorter string), the failure signal itself becomes exactly the one-bit-per-query oracle Section 3.6 already warns about. Skip either line, and Decaps quietly degrades back into the same oracle Kyber.CPAPKE alone was already vulnerable to — the FO transform's entire contribution is these two lines together, not either one alone.

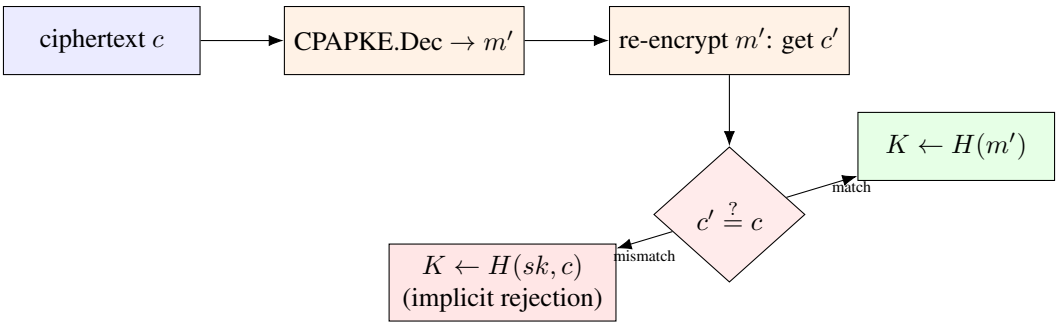


Figure 3.3: ML-KEM's Decaps, schematically. The re-encryption check is the step the Fujisaki–Okamoto transform adds on top of plain Kyber.CPAPKE decryption; both branches must be computed in a way that reveals nothing about which one was taken.

Example: ML-KEM Encapsulation and Decapsulation, $k = 1$, $n = 4$, $q = 17$

Reuse Section 3.5's $k = 1$ keys exactly: $A = 3+5x+2x^2+x^3$, $s = 1-x$, $t = 5+2x+14x^2+16x^3$.

Encaps (honest run). Suppose the fresh random message sampled in line 1 is $m = (1, 0, 1, 0)$. The hash-based derivation in line 2 is not something to compute by hand — in real ML-KEM it is a cryptographic hash function — so for this walkthrough we simply fix its output: $(r, e_1, e_2) = (x,\ 1,\ -1)$, and, as a toy stand-in for $K \leftarrow H(m)$, take K to be m's bitwise complement, $K = (0, 1, 0, 1)$. Line 3 then runs exactly Section 3.5's computation: $u = 0 + 3x + 5x^2 + 2x^3$, $v = 9 + 5x + 11x^2 + 14x^3$. **Output:** ciphertext $c = (u, v)$, shared secret $K = (0, 1, 0, 1)$.

Decaps, on the honest ciphertext. Line 1 runs CPAPKE.Dec exactly as Section 3.5 did, recovering $m' = (1, 0, 1, 0)$ — matching m exactly. Line 2 re-derives randomness from this m': since the derivation is a function of the message alone, it reproduces the same $(r, e_1, e_2) = (x, 1, -1)$, so re-encrypting gives $c' = (u, v)$, identical to c. The check on line 3 passes ($c' = c$), so **Decaps returns** $K = H(m') = (0, 1, 0, 1)$ — the same key Encaps produced.

Decaps, on a tampered ciphertext. Now suppose an attacker intercepts $c = (u, v)$ and bumps v's constant coefficient by 1 before forwarding it, submitting $c^* = (u, v^*)$ with $v^* = 10+5x+11x^2+14x^3$ in place of v. Decrypting: $m'' = v^* - s \cdot u = (10-2)+(5-3)x+(11-2)x^2+(14-14)x^3 = 8+2x+9x^2+0x^3$, which decodes to $(1, 0, 1, 0)$ — *the same message as before*: decoding alone gives no hint anything is wrong. But line 2 re-derives randomness from this $m'' = (1, 0, 1, 0)$, exactly the message case already handled above, so it reproduces $(r, e_1, e_2) = (x, 1, -1)$ and re-encrypts to $c' = (u, v)$, the *original*, untampered ciphertext — not c^*. The check on line 3 now fails ($c' \neq c^*$, since $v \neq v^*$), so implicit rejection triggers: **Decaps returns** $K \leftarrow H(sk, c^*)$, a value determined by the secret key and the submitted ciphertext together, unrelated to $H(m'')$ and, to anyone without sk, indistinguishable from a legitimate key.

The tampered example is deliberately chosen so that Decode alone cannot tell the two ciphertexts apart — both decrypt to the same message. That is exactly the point: the re-encryption check, not decoding, is what catches the corruption, which is why Decaps performs it on every ciphertext rather than trusting the decoded message on its own.

Returning a plain error on mismatch, rather than implicit rejection, would reopen exactly the gap CCA security is meant to close: even a bare accepted/rejected signal, with no timing information at all, lets an attacker submit many chosen, slightly malformed ciphertexts and learn one bit per query — and Lesson 5's boundary-sweep attack (Section 5.2) shows exactly how a string of such bits reconstructs an entire secret key. Implicit rejection removes the distinguishable outcome: every ciphertext, valid or not, produces some K that looks equally random from the outside, with no error message and, if implemented correctly, no timing difference between the two branches. This is also why Decaps performs a full re-encryption on every ciphertext, valid or not, rather than shortcutting for inputs that look obviously malformed — deciding what counts as "obviously malformed" without doing the full check would itself leak information.

Figure 3.4 redraws the two-party picture from Figure 3.1, now with Encaps and Decaps in place of Enc and Dec. The shape of the exchange — pk crossing once, a single ciphertext crossing back — is unchanged; what is new is that neither party ever transmits a message at all. Both sides instead arrive at the same shared secret K independently, the Sender by construction (it generated K itself, alongside c, in line 2 of Encaps) and the Receiver by recomputation (via Decaps) — with K itself never appearing on the wire in either direction.

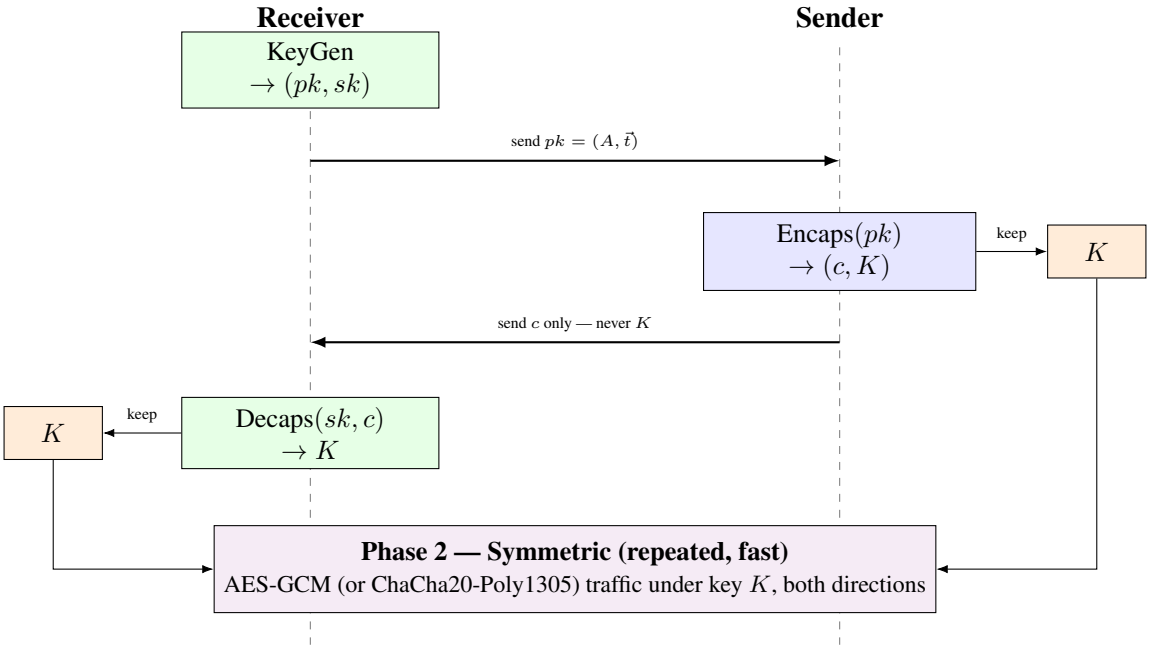


Figure 3.4: The CCA-secure KEM protocol, now including what happens after the handshake. Only pk and c ever cross the wire during Phase 1 — exactly as in the CPA version (Figure 3.1); the shared secret K is computed independently on each side and kept private. If c reaches Decaps unmodified, both K's match (Section 3.6's worked example); if c was tampered with in transit, implicit rejection still produces *some* K on the Receiver's side, just not one that matches the Sender's (Section 3.7.1 covers how that gets caught). Once both sides do hold matching K, Phase 2 hands it to a fast symmetric cipher for the actual application traffic — exactly Lecture 2's hybrid-encryption pattern (Figure 2.2), with ML-KEM supplying Phase 1.

Remark: Where This Step Is Attacked

The re-encryption check is a comparison whose outcome must never leak; if an attacker learns, via timing or power consumption, whether a chosen ciphertext passed or failed it, many such queries can be turned into full key recovery. A second, independent leak sits earlier: the intermediate value m' is computed in full *before* the check ever runs, even for ciphertexts that will ultimately be rejected, so if that computation itself leaks (through the CPAPKE arithmetic touching $\vec{s}$), implicit rejection does not help — the damage already happened one step earlier. Lesson 5 names specific published attacks that exploit exactly this structure.

3.7 From Shared Secret to a Working Channel

Encaps and Decaps only ever establish one thing: a short, fresh, matching value K on both sides (Figure 3.4). K is never used to encrypt application data directly. Exactly as Lecture 2's hybrid-encryption pattern (Section 2.1.2) anticipated, ML-KEM's entire output is handed off to a fast symmetric cipher — typically AES-GCM or ChaCha20-Poly1305 — which does the actual work of encrypting whatever messages the two parties exchange afterward. This is precisely why ML-KEM is a *Key Encapsulation Mechanism* and not a general-purpose encryption scheme: K is deliberately too short-lived and too narrowly scoped to be anything other than a symmetric key, and Figure 3.4's Phase 2 shows exactly where it lands.

In a real deployment, K is not used as an AES key directly either. It is first run through a key-derivation function (a KDF, typically HKDF) to produce one or more separate *traffic keys* — often a different key

for each direction (Receiver→Sender and Sender→Receiver), so a compromise of one direction's key does not automatically expose the other. Since HKDF is a deterministic function of K, both parties, having independently arrived at the same K, independently derive the same traffic keys from it with no further communication needed — and switch to fast symmetric encryption for the rest of the session.

3.7.1 Catching a Mismatched K

Figure 3.4's caption already flags the one case that matters here: if c is tampered with (or simply corrupted) in transit, implicit rejection makes Decaps return *some* K on the Receiver's side — just not the one the Sender actually generated. Nothing about Encaps or Decaps themselves ever detects this; implicit rejection is deliberately built to look exactly like a success, on both branches, precisely to defend against the oracle attacks of Section 3.6 and Lesson 5. So a mismatch has to be caught somewhere *else* — not by a special repair sub-protocol bolted onto ML-KEM, but by the ordinary integrity check every real handshake protocol already performs on itself.

Remark: This Is Not a New Protocol — It Is TLS's Existing One

TLS 1.3 (and every other protocol that embeds ML-KEM the same way) already ends its handshake with a **Finished** message from each side: a MAC, computed under a key derived from that side's own view of the negotiated secret, over the entire handshake transcript so far. If the two sides' K values do not match, the traffic keys they derive do not match either, so the MAC one side computes will not verify against what the other side expects — the Finished check fails, and the connection is simply torn down. There is no separate "did we really agree on K" handshake layered on top of TLS for ML-KEM's benefit; **the Finished exchange already is that check**, for every possible cause of disagreement (a corrupted ML-KEM ciphertext, active tampering, an implementation bug, anything at all) — not a mechanism added specifically because ML-KEM might fail. The response to a failed Finished check is the same regardless of cause: abandon this handshake attempt and start over with a fresh Encaps.

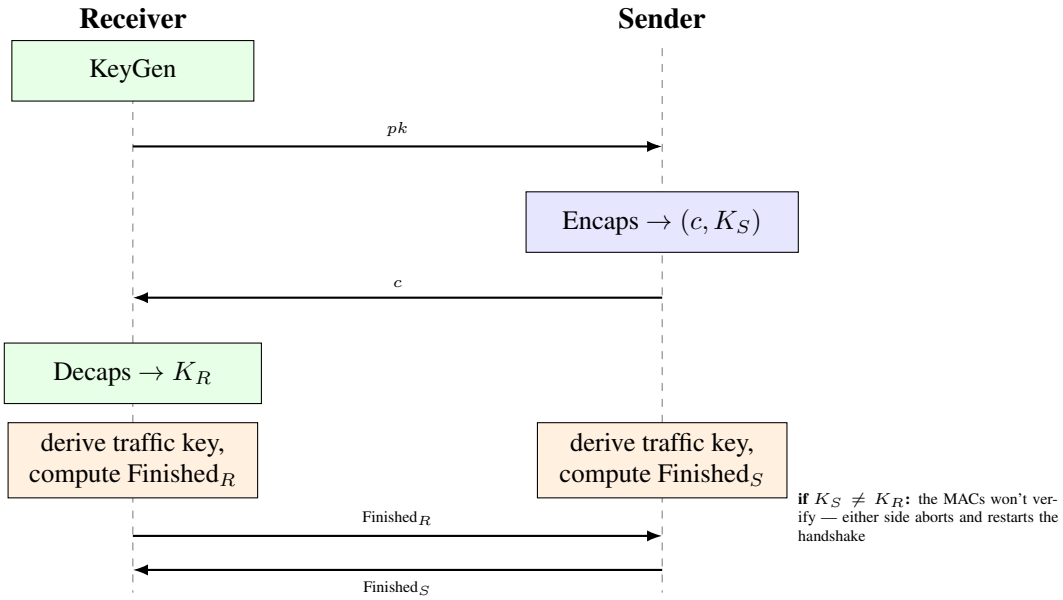


Figure 3.5: Key confirmation as it actually happens: two ordinary MACs (Finished messages), each computed under a key derived from that party's own view of K. A mismatch $K_S \neq K_R$ — for instance, from a tampered ML-KEM ciphertext triggering implicit rejection — makes one or both MAC checks fail, aborting the handshake. This is the same general-purpose mechanism TLS already uses to catch *any* handshake corruption, not something added specifically for ML-KEM.

This buys something beyond robustness: an attacker who tampers with c does not get to watch Decaps "fail" directly — implicit rejection hides that entirely at the KEM layer, exactly as designed. The only thing an on-path attacker can eventually observe is that the connection was torn down after the Finished exchange — a far coarser, far more expensive signal (an entire abandoned handshake, not a single oracle query), and exactly the kind of signal Lesson 5's attacks are built to avoid ever needing.

3.8 Deployment

ML-KEM was finalized by NIST as **FIPS 203** in August 2024, alongside ML-DSA (FIPS 204) and SLH-DSA (FIPS 205).[1] It is already deployed at scale: Chrome enabled hybrid ML-KEM by default in late 2024, Apple's iMessage uses it as part of PQ3, and the major cloud providers (AWS, Google Cloud, Microsoft Azure) support PQC-enabled TLS endpoints. In these deployments ML-KEM runs alongside a classical algorithm such as X25519 rather than replacing it: a hybrid handshake runs both X25519 and ML-KEM and combines the two resulting shared secrets, typically by hashing them together, into the key actually used, so an attacker must break *both* the classical and the lattice problem to recover it. This is a deliberate, conservative hedge during the transition to a comparatively young primitive, not a statement of doubt about the material in this lecture.

3.9 Summary

Object	**What it is, in Lecture 2's language**
Public key $(A, \vec{t})$	An MLWE sample: $\vec{t} = A\vec{s} + \vec{e}$
Secret key $\vec{s}$	The MLWE secret
Ciphertext part $\vec{u}$	A second, independent MLWE sample using fresh randomness $\vec{r}$
Ciphertext part v	Public key mixed with $\vec{r}$, plus the message, plus noise
Decryption	Noise cancels algebraically, leaving message + small residual noise ε
Compression (d_u, d_v)	Deliberate extra rounding error, traded for smaller ciphertexts
Noise budget	Decryption succeeds iff every coefficient of ε stays under $q/4$
CPAPKE $\rightarrow$ ML-KEM	Fujisaki–Okamoto transform: re-encrypt and check, or implicitly reject
Hybrid deployment	ML-KEM run alongside a classical scheme (e.g. X25519), keys combined

[1] NIST, *Module-Lattice-Based Key-Encapsulation Mechanism Standard*, FIPS 203, August 2024.

3.10 Exercises

Exercise: Homework Set

1. Redo the compression worked example (Section 3.2) with $d = 3$ instead of $d = 2$, this time for $x = 9$ (still $q = 17$). Is the recovered value after decompression closer to the original 9 than it was with $d = 2$? Explain why, in general, larger d means smaller compression error.
2. In the noise-cancellation derivation (Section 3.4), identify by name every term that ends up inside ε. Which of these terms come from key generation, and which come from encryption?
3. Using the $k = 1$ worked example in Section 3.5, redo encryption with a *different* message $m = (0, 1, 0, 1)$, keeping the same A, s, e, r, e_1, e_2. Compute u, v, and confirm decryption recovers $(0, 1, 0, 1)$.
4. Using the $k = 2$ worked example in Section 3.5.1, redo encryption with fresh randomness $\vec{r} = (0,\ 1)$ instead of $(1,\ x)$, keeping $\vec{e}_1 = (0, 1)$, $e_2 = 0$, and the same message $m = (1, 1, 0, 0)$. Compute $\vec{u}$ and v, then decrypt and confirm you recover $(1, 1, 0, 0)$ again. (Hint: with $r_1 = 0$, several terms drop out entirely — use that to check your work as you go.)
5. In your own words (4–5 sentences): why is it not enough for ML-KEM's Decaps to simply "return an error" when the re-encryption check fails, instead of the implicit-rejection mechanism in Section 3.6? Think about what an attacker could learn from the mere presence or timing of an error message.
6. Using Section 3.1.2's table, state the shape (matrix, vector, or scalar ring element, and how many entries) of each of the following ML-KEM-768 objects: A, $\vec{s}$, $\vec{t}$, $\vec{u}$, v. (Recall ML-KEM-768 uses $k = 3$.)
7. In 3–4 sentences, distinguish CPA security, CCA security, and the Fujisaki–Okamoto transform (Section 3.6): which is a threat model, which is a security guarantee, and which is a construction? Give one example of a real-world setting, other than a network man-in-the-middle, that hands an attacker chosen-ciphertext access.
8. **Looking ahead:** skim the abstract of one published Kyber side-channel paper of your choosing (we will assign specific ones in Lesson 5) and identify which single line of the algorithms in Section 3.3 of this lecture it targets — KeyGen, Enc, Dec, or the FO check in Section 3.6.

Lesson 4: ML-DSA (Dilithium) and FN-DSA (Falcon)

Signing With Lattices: Fiat–Shamir-with-Aborts and Trapdoor Sampling

Purpose of this lecture. Lecture 3 built an encryption scheme out of MLWE. Today we build *signature* schemes — a different goal (proving you hold a secret, not hiding a message) that leads to genuinely different designs. We cover **ML-DSA (Dilithium)**, which reuses MLWE directly inside a technique called Fiat–Shamir-with-Aborts, and **FN-DSA (Falcon)**, which uses a different lattice (NTRU) and a different technique (GPV trapdoor sampling) entirely. Both sections end by naming the exact step in each algorithm that Lesson 6's physical attacks target — that mapping is the main deliverable of this lecture.

4.1 What a Signature Scheme Needs to Do

Encryption (Lecture 3) hides information; a signature does the opposite — it *proves* something publicly, without hiding anything. A signature scheme has three algorithms: KeyGen (produces a public/secret key pair), $\mathrm{Sign}(sk, \text{message})$ (produces a signature), and $\mathrm{Verify}(pk, \text{message}, \text{signature})$ (accepts or rejects). The security goal is **unforgeability**: without sk, no one should be able to produce a signature that Verify accepts, even after seeing many genuine signatures on messages of their choosing.

Remark: The Core Design Tension

A signature must depend on the secret key (otherwise anyone could forge it) — but it must not depend on the secret key in a way that *leaks* it, even after an attacker collects thousands of signatures. Both schemes in this lecture are, at heart, different engineering answers to that one tension. Keep this framing in mind; it's the thread connecting today's material straight through to Lesson 6.

4.2 A Primer on Fiat–Shamir (Needed Before Dilithium Makes Sense)

Definition: Fiat–Shamir Transform, Informally

Start with a three-move *interactive* protocol between a Prover (who knows a secret) and a Verifier: (1) Prover sends a **commitment** w (built from fresh randomness, independent of the secret); (2) Verifier sends a random **challenge** c; (3) Prover computes a **response** z that combines w's randomness with the secret and c, and the Verifier checks a consistency equation relating w, c, z, and the public key. The Fiat–Shamir transform removes the Verifier: the Prover computes c themself, as a hash of w (and the message being signed), $c = H(w, \text{message})$. Because H behaves unpredictably, the Prover still can't predict c before committing to w — so the security argument survives, but now there's no interaction: the whole transcript (w, c, z) can be published as a non-interactive signature.

Remark: Why This Detour Was Necessary

Every part of Dilithium's Sign algorithm below — the commitment w, the challenge c, the response z — is exactly this three-move shape, with the interaction removed via hashing. Without seeing the general pattern first, Dilithium's algorithm looks like an arbitrary pile of steps; with it, each line has an obvious job.

4.3 ML-DSA (Dilithium): Parameters

Definition: ML-DSA Parameter Sets

All parameter sets share the familiar ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ with $n = 256$, this time with $q = 8{,}380{,}417$ (a different prime from Kyber's — still chosen for NTT-friendliness). The secret is a pair of small vectors $(\vec{s}_1, \vec{s}_2) \in R_q^\ell \times R_q^k$; the public key is built from an MLWE-style sample using them.

Parameter set	(k, ℓ)	Signature size	Public-key size
ML-DSA-44	(4,4)	2,420 bytes	1,312 bytes
ML-DSA-65	(6,5)	3,309 bytes	1,952 bytes
ML-DSA-87	(8,7)	4,627 bytes	2,592 bytes

4.4 ML-DSA: KeyGen, Sign, Verify

Algorithm 6 ML-DSA.KeyGen

1: Generate $A \in R_q^{k \times \ell}$ uniformly at random (again, from a short public seed)
2: Sample small $\vec{s}_1 \in R_q^\ell$, $\vec{s}_2 \in R_q^k$ (coefficients bounded by a small η)
3: Compute $\vec{t} = A\vec{s}_1 + \vec{s}_2 \in R_q^k$
4: **return** $pk = (A, \vec{t})$, $sk = (\vec{s}_1, \vec{s}_2)$

Remark: Yes, This Is an MLWE Sample Too

Line 3 is identical in shape to Kyber's KeyGen (Lecture 3). The two schemes start from the *same* hard problem; what differs entirely is what you do with the key afterward.

Algorithm 7 ML-DSA.Sign$(sk = (\vec{s}_1, \vec{s}_2),\ \text{msg})$ — Simplified

1: Sample a masking vector $\vec{y} \in R_q^\ell$ with somewhat larger small coefficients than $\vec{s}_1, \vec{s}_2$ (derived deterministically from sk and msg via a PRF, not from fresh randomness — see remark below)
2: Compute the commitment $\vec{w} = A\vec{y} \in R_q^k$
3: Compute the challenge $c = H(\vec{w}, \text{msg})$ — a specially-shaped "sparse" small polynomial
4: Compute the response $\vec{z} = \vec{y} + c\vec{s}_1$
5: **Rejection check:** if any coefficient of $\vec{z}$ (or of a related quantity built from $\vec{w} - c\vec{s}_2$) is too large in absolute value, **discard everything and go back to step 1** with fresh $\vec{y}$
6: Once the check passes, **return** signature $(\vec{z}, c)$ (in the real scheme, a compact *hint* $\vec{h}$ is also included, letting the verifier reconstruct $\vec{w}$'s high-order bits exactly — omitted here for simplicity)

Algorithm 8 ML-DSA.Verify$(pk = (A, \vec{t}),\ \text{msg},\ (\vec{z}, c))$ — Simplified

1: Check that every coefficient of $\vec{z}$ is within the allowed bound (reject if not)
2: Recompute $\vec{w}' = A\vec{z} - c\vec{t}$
3: Accept if $c = H(\vec{w}', \text{msg})$; reject otherwise

Remark: Why Verification Works: the Algebra

Substitute: $\vec{w}' = A\vec{z} - c\vec{t} = A(\vec{y} + c\vec{s}_1) - c(A\vec{s}_1 + \vec{s}_2) = A\vec{y} + cA\vec{s}_1 - cA\vec{s}_1 - c\vec{s}_2 = A\vec{y} - c\vec{s}_2 = \vec{w} - c\vec{s}_2$. The two copies of $cA\vec{s}_1$ cancel — the same style of cancellation as Kyber's decryption in Lecture 3. $\vec{w}'$ isn't *exactly* $\vec{w}$; it's $\vec{w}$ shifted by a small quantity $c\vec{s}_2$. Real ML-DSA's "high bits" / hint mechanism (omitted above) is specifically engineered so that this small shift doesn't change the *high-order* bits of $\vec{w}$, so $H(\vec{w}', \text{msg})$ still reproduces the same challenge c the signer used. Our simplified version above sidesteps this by just checking exact equality of the full hash input in a toy example (Section 4.6) small enough that no shift occurs.

4.5 Why Rejection Sampling Exists (This Is the Important Part)

Remark: The Problem Rejection Sampling Solves

Look at line 4 above: $\vec{z} = \vec{y} + c\vec{s}_1$. If we *always* output $\vec{z}$, its distribution would depend — ever so slightly — on the secret $\vec{s}_1$, because $c\vec{s}_1$ is added on every single signature. Collect enough signatures on enough messages, and that slight dependence becomes a statistical attack surface: an attacker who sees many $(\vec{z}, c)$ pairs could, in principle, average out the masking $\vec{y}$ and recover information about $\vec{s}_1$. **Rejection sampling closes this gap**: $\vec{y}$ is deliberately sampled from a range wide enough that, *conditioned on being accepted*, $\vec{z}$'s distribution is statistically independent of $\vec{s}_1$ — indistinguishable from $\vec{z}$ having been sampled uniformly from the accepted range, with no trace of the secret at all. The rejected attempts simply never get published; only "safe-looking" signatures ever leave the signer.

Fact: Deterministic Signing, and Why It's a Double-Edged Sword

Step 1 above notes that $\vec{y}$ is derived *deterministically* from (sk, msg) via a pseudorandom function, rather than from fresh random bits each time. This mirrors a well-known fix (RFC 6979-style) for a classical signature failure mode: if two different messages were ever signed with the *same* $\vec{y}$ by accident (e.g. a broken random-number generator), the two resulting equations $\vec{z}_1 = \vec{y} + c_1\vec{s}_1$ and $\vec{z}_2 = \vec{y} + c_2\vec{s}_1$ can be subtracted to isolate $\vec{s}_1$ directly — a catastrophic, complete key-recovery, and a real failure class in classical ECDSA history (e.g. the Sony PlayStation 3 signing-key incident). Determinism eliminates the accidental version of this bug. But it introduces a new one: **if a fault attack can force the same $\vec{y}$ to be reused across two different signing operations** (by corrupting the PRF's internal state, or replaying an earlier internal value), the exact same subtraction attack becomes available again — this time caused deliberately rather than by accident. This is precisely the "fault attacks on determinism" line item flagged for Lesson 6: determinism is a defense against one failure mode and an attack surface for another, and real published Dilithium fault attacks exploit exactly this tension.

4.6 A Fully Worked Toy Example

Example: ML-DSA-Style Signing, $k = \ell = 1$, $n = 4$, $q = 17$ (Verified by Hand)

Reusing $R_{17} = \mathbb{Z}_{17}[x]/(x^4 + 1)$ and $A = 3 + 5x + 2x^2 + x^3$ from Lecture 3.
KeyGen. Let $s_1 = 1$ (constant polynomial) and $s_2 = 1 - x$ (small). Then

$$t = A \cdot s_1 + s_2 = A + (1 - x) = (3 + 1) + (5 - 1)x + 2x^2 + x^3 = 4 + 4x + 2x^2 + x^3.$$

Sign. Mask $y = x$. Commitment: $w = A \cdot y = A \cdot x = 16 + 3x + 5x^2 + 2x^3$ (computed exactly as in Lecture 2's ring-multiplication example). For this toy example, take the challenge to be the simplest possible sparse polynomial, $c = 1$ (constant). Response: $z = y + c \cdot s_1 = x + 1 = 1 + x$. (No rejection triggered — coefficients of z are small.) Signature: $(z, c) = (1 + x,\ 1)$.
Verify. Recompute $w' = A \cdot z - c \cdot t$:

$$\begin{aligned} A \cdot z &= A \cdot (1 + x) = A + A \cdot x = (3, 5, 2, 1) + (16, 3, 5, 2) \\ &= (19, 8, 7, 3) \equiv (2, 8, 7, 3) \bmod 17, \\ c \cdot t &= 1 \cdot t = (4, 4, 2, 1), \\ w' &= (2 - 4,\ 8 - 4,\ 7 - 2,\ 3 - 1) = (-2, 4, 5, 2) \equiv (15, 4, 5, 2) \bmod 17. \end{aligned}$$

Compare to the original $w = (16, 3, 5, 2)$: they are *not* identical — $w' = w - c \cdot s_2 = w - (1, -1, 0, 0)$, matching the algebra of Section 4.4's remark exactly (subtract $s_2 = 1 - x = (1, -1, 0, 0)$ from $w = (16, 3, 5, 2)$: $(16 - 1, 3 - (-1), 5, 2) = (15, 4, 5, 2) = w'$, confirmed). In this toy example we accept the signature by checking this exact algebraic identity holds (the real scheme's hint mechanism is what lets a verifier who does *not* know s_2 still confirm this identity, via the high-order bits of w instead of w itself — omitted here for tractability, as flagged in Section 4.4).

Exercise: Practice

Using the same A, s_1, s_2: sign the same message again, but this time with $y = 1$ (instead of $y = x$) and the same challenge $c = 1$. Compute the new z and confirm $w' = w - c \cdot s_2$ still holds with the new $w = A \cdot y$. Then, pretending you are an attacker who somehow learned that the *same* $y = x$ was used for two different messages with challenges $c_1 = 1$ and $c_2 = 2$ respectively, write down the two equations for z_1 and z_2 and show algebraically how subtracting them isolates s_1 — this is the calculation behind the factbox in Section 4.5.

4.7 FN-DSA (Falcon): a Different Lattice, a Different Technique

Falcon abandons MLWE entirely and uses a different hard problem on a different kind of lattice — worth seeing, precisely because your project needs to compare attack surfaces *across* schemes, not just understand one.

Definition: NTRU Lattices

Fix $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ as before. An NTRU key pair consists of two *short* polynomials $f, g \in R_q$ (the secret), from which the public key $h = g \cdot f^{-1} \bmod q$ is computed (assuming f is invertible mod q). The **NTRU lattice** associated to h is

$$\Lambda_h = \{\, (u, v) \in R_q^2 : u + v \cdot h \equiv 0 \pmod{q} \,\}.$$

(f, g) (suitably arranged) is a *short vector* in Λ_h — in fact, a short enough one to serve as a trapdoor *basis* for the whole lattice, not just a single short vector: this is the "all vectors short together" property Lecture 1's optional Minkowski's Second Theorem box was pointing toward.

Definition: The GPV Framework (Gentry–Peikert–Vaikuntanathan)

Given a trapdoor (short basis) for a lattice Λ, and a target point $t = H(\text{msg})$ (hashed into the same space as Λ), **sample** a lattice point $v \in \Lambda$ close to t, using the trapdoor to guide a randomized search — specifically, sampling from a discrete Gaussian distribution (Lecture 2, Section 2.6.3) centered near t. The signature is (essentially) the short offset $s = t - v$. Verification recomputes t from the message, checks s is short, and checks $s + v'$ (reconstructed from the public key and s) lands back on a valid target. Unlike Dilithium's commit-challenge-response shape, this is a single sampling step against a target — no interactive protocol being flattened, just a direct trapdoor-guided search.

Remark: Falcon's KeyGen/Sign/Verify, at a Glance

KeyGen: generate short f, g (this step itself is delicate number theory — specific polynomial sampling techniques — which we do not detail here); compute $h = gf^{-1} \bmod q$; the trapdoor is (f, g) (or an equivalent short basis derived from them), the public key is h. **Sign:** hash the message to a target point, then use the trapdoor with GPV sampling (Falcon's specific method is called *fast Fourier sampling*, chosen for speed) to find a short vector (s_1, s_2) with $s_1 + s_2 h \equiv H(\text{msg}) \pmod{q}$; output (s_1, s_2) (compressed) as the signature. **Verify:** recompute $s_1 = H(\text{msg}) - s_2 h \bmod q$ from the public h and the received s_2, and check that (s_1, s_2) is short enough to plausibly come from the trapdoor.

Looking Ahead: Why Falcon's Sampler Is the Star of Lesson 6

Two things make Falcon's signing step uniquely attack-prone compared to Dilithium's. First, GPV sampling needs to sample from a discrete Gaussian *exactly* (or very close to it) — an approximate or biased sampler can leak information about the trapdoor (f, g) through the statistical shape of many signatures, even without any single signature looking wrong. Second, Falcon's fast Fourier sampling is implemented using **floating-point arithmetic**, for speed — and floating-point operations have historically been far harder to make constant-time (free of secret-dependent timing or power variation) than the simple integer comparisons Dilithium's rejection sampling relies on. Put together: Dilithium's main defense (rejection sampling) is a young technique but algorithmically simple to implement safely; Falcon's main mechanism (Gaussian trapdoor sampling) is a mathematically older and better-understood technique, but a much harder *implementation* target to make side-channel-resistant. This is exactly the gap that produced the published single-trace and sampler-leakage attacks on Falcon that Lesson 6 will walk through.

4.8 Standardization Status

ML-DSA was finalized as **FIPS 204** alongside ML-KEM and SLH-DSA in August 2024, and is NIST's recommended general-purpose signature standard today.[1] FN-DSA (Falcon), by contrast, remains a **draft** standard (to become FIPS 206): as of mid-2026 it has not yet been finalized, with publication expected in late 2026 or early 2027, and major certificate authorities have stated they will not deploy it in production until it is.[2] Worth stating plainly in the proposal's related-work section: attacking a finalized federal standard (ML-DSA) and attacking a still-draft one (FN-DSA) carry different framing, even though both are technically live research targets today.

4.9 Comparing the Three Schemes So Far

	ML-KEM (Kyber)	**ML-DSA (Dilithium)**	**FN-DSA (Falcon)**
Hard problem	Module-LWE	Module-LWE	NTRU lattice (SIS/GPV-flavored)
Purpose	Key encapsulation	Signature	Signature
Core technique	MLWE encryption + FO transform	Fiat–Shamir-with-Aborts	GPV trapdoor sampling
Key defense mechanism	Implicit rejection (Decaps)	Rejection sampling	Exact Gaussian sampling
Riskiest implementation step	Re-encryption check	$\vec{y}$ generation / bound check	Floating-point sampler
Standard status (mid-2026)	FIPS 203, finalized	FIPS 204, finalized	FIPS 206, draft

Looking Ahead: Where This Leads: Lessons 5 and 6

You now have, for all three schemes, both the correct algorithm *and* a named "riskiest step." Lesson 5 opens by formalizing the general distinction between attacking the math (everything in Lectures 1–4) versus attacking an implementation of it, then unpacks ML-KEM's one named riskiest step into two concrete, worked attacks. Lesson 6 completes the picture with the remaining four — two for ML-DSA, two for FN-DSA — six risky steps in total across the two lessons. At that point, every attack should read as "targeting a specific line of an algorithm you can already write out from memory," not as an isolated trick.

[1] NIST, *Module-Lattice-Based Digital Signature Standard*, FIPS 204, August 2024.
[2] NIST Initial Public Draft, FIPS 206, submitted for public review in late 2025.

4.10 Exercises

Exercise: Homework Set

1. In your own words, explain the Fiat–Shamir transform to someone who has never seen an interactive proof — use the commit/challenge/response structure from Section 4.2, but invent your own non-cryptographic analogy (e.g. a game show, a magic trick) to illustrate why removing the interactive Verifier and replacing it with a hash still works.
2. Complete the practice exercise in Section 4.6 (two-signature key-recovery calculation) if you have not already.
3. Compare Kyber's re-encryption check (Lecture 3, Section 3.6) and Dilithium's rejection sampling (Section 4.5 above): both exist to prevent a secret-dependent quantity from becoming visible to an attacker. In 4–5 sentences, describe what each mechanism is protecting against, and why the two mechanisms look so different from each other despite that similarity of purpose.
4. Falcon's signature is a pair (s_1, s_2) with $s_1 + s_2 h \equiv H(\text{msg}) \pmod{q}$. Suppose $h = 5$ (a toy scalar stand-in, not a real ring element) and $H(\text{msg}) = 12 \pmod{17}$. If $s_2 = 2$, what must s_1 be for the equation to hold? Is this pair unique, or could other (s_1, s_2) pairs also satisfy the equation — and if so, what property (from Section 4.7) picks out the one Falcon actually outputs?
5. **Looking ahead:** using the comparison table in Section 4.9, predict — before Lessons 5–6 reveal the real answer — which of the three schemes' riskiest steps you would guess is *easiest* to defend against with constant-time coding practices alone, and which would need dedicated hardware countermeasures. Briefly justify your guess; we will revisit it directly there.

Lesson 5: Physical Attacks I — ML-KEM (Kyber)

Attacking the Math vs. the Implementation, and Two Ways to Break Kyber Without Breaking LWE

Purpose of this lecture. Lectures 1–4 built the mathematics: worst-case lattice problems, SIS, LWE, and three real, standardized schemes built on top of them. None of that math is what most published attacks on lattice cryptography actually break. This lecture turns to the other half of the threat model: what an attacker can learn by watching *how* a scheme is computed — its timing, its power draw, its electromagnetic emissions — or by deliberately corrupting that computation with a fault, rather than by attacking the underlying hard problem at all. We build the general framework once, then spend the rest of the lecture on ML-KEM (Kyber) specifically: its one named "riskiest step" from Lecture 3's summary table, unpacked into two concrete, fully worked attacks. Lesson 6 completes the picture with ML-DSA (Dilithium) and FN-DSA (Falcon).

5.1 Attacking the Math vs. Attacking the Implementation

5.1.1 Two Threat Models

Every attack on a cryptographic scheme falls into one of two categories, and it matters enormously which one you're doing.

Definition: Cryptanalysis vs. Implementation Attacks

A **cryptanalytic** (mathematical) attack works entirely from public information — the public key, some ciphertexts or signatures — and unlimited computation, exactly as if the scheme were an abstract mathematical object with no physical existence at all. Lattice reduction (LLL, BKZ) attacking an LWE instance directly is cryptanalysis. An **implementation attack** instead exploits *how* a specific piece of software or hardware computes the algorithm: its timing, its power consumption, its electromagnetic emissions, or its behavior when deliberately corrupted. It says nothing about whether the underlying math is broken — and, as this lecture and the next will show, a scheme can have a rock-solid worst-case hardness proof and still fall apart in seconds against the right implementation attack.

Remark: Why This Course Cares

Nothing in Lectures 1–4 is wrong or weakened by anything in this lecture. GapSVP, LWE, and SIS are exactly as hard as Lecture 2 said. What changes is the question: instead of "can an attacker solve the math problem," it's "can an attacker learn something about the secret *without ever solving it*, just by watching the computer compute?" Both threat models have to be defended against simultaneously — a scheme immune to the first but not the second is not secure in practice, which is exactly why implementation security is its own, entire subfield, and exactly why your thesis has a whole project's worth of material to work with.

Implementation attacks split further, along a second axis: whether the attacker only *observes*, or actively *interferes*.

Definition: Passive vs. Active Implementation Attacks

A **passive** attack (side-channel analysis) only observes a physical side-effect of a correct computation — how long it took, how much power it drew, what it radiated electromagnetically, which cache lines it touched — without disturbing the computation itself. An **active** attack (fault injection) deliberately corrupts the computation — via a voltage glitch, a clock glitch, an electromagnetic pulse, or a laser pulse aimed at the chip — and learns something from *how the output differs* from what a correct execution would have produced.

5.1.2 The General Recipe: Leak, Then Combine

Almost every implementation attack you will ever read about, however exotic the physical mechanism, follows the same two-step shape.

Remark: The Recipe

Step 1 — Leak. Find some observable (a timing difference, a power trace, a fault's effect on the output) that depends, even very slightly, on the secret. One observation rarely reveals the whole secret — often just one bit, or one noisy equation involving it, or a small bias in a distribution that should have been secret-independent. **Step 2 — Combine.** Repeat Step 1 many times — once per chosen ciphertext, once per captured power trace, once per induced fault — and combine the results. Sometimes this combination is purely algebraic (solve a system of equations, exactly the way Lecture 4's practice exercise solved for $\vec{s}_1$ from two equations). Sometimes it's statistical (average many power traces against a guessed secret byte and look for correlation — the classical technique is called **Differential** or **Correlation Power Analysis**, DPA/CPA). Occasionally it's a hybrid: use the leak to pin down *most* of the secret, then hand the small remaining uncertainty to a lattice-reduction algorithm like BKZ (Lecture 2) to finish the job — turning a partial side-channel leak into full key recovery even when no single observation was clean enough to do it alone.

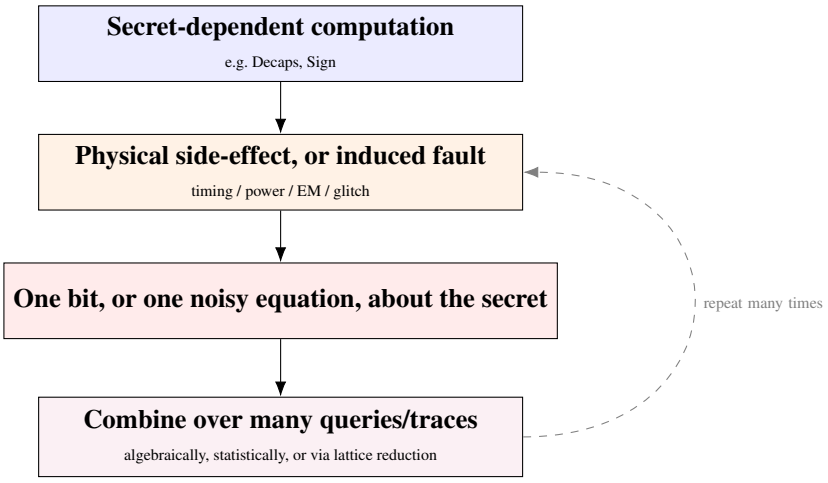


Figure 5.1: Every attack in this lecture and the next is one instance of this same two-step loop: leak a little, then combine a lot. What differs between attacks is only what fills each box.

Example: A Toy Timing Leak, Before Any Lattices

Here's the recipe at its simplest, with no cryptography-specific machinery at all. Suppose a program checks a secret 4-digit PIN against a guess by comparing digits one at a time and returning `false` the instant it finds a mismatch:

$$\texttt{for } i = 1..4\texttt{: if guess}[i] \neq \texttt{secret}[i]\texttt{: return false} \quad \texttt{return true}$$

Leak: a guess that's wrong in digit 1 exits almost immediately; a guess that happens to get digit 1 *right* takes measurably longer, since the loop proceeds to check digit 2. Timing alone reveals *how many leading digits were correct* — one bit of information ("was this guess's timing longer than that guess's?") per query. **Combine:** try all 10 values for digit 1, keep whichever one measurably takes longer; that's digit 1, solved with at most 10 queries instead of the 10^4 a truly blind guesser would need. Repeat for digit 2, then 3, then 4: the total cost drops from 10^4 to about 4×10. This exact bug — an early-exit comparison where it should have been constant-time — has been found in real production cryptographic code more than once. Section 5.3 below is the same idea, aimed at Kyber's `Decaps`.

5.2 A Chosen-Ciphertext Oracle Near the Decoding Boundary

Recall Kyber's decryption step, exactly as Lecture 3 wrote it: given secret key $\vec{s}$ and ciphertext $c = (u, v)$ (we work with the uncompressed $k = 1$ toy version from Lecture 3, Section 3.3, to keep the arithmetic hand-tractable), decryption computes $m' = v - s \cdot u$ and then **decodes** each coefficient: output bit 1 if that coefficient is closer to $\lceil q/2 \rceil$ than to 0 (mod q), else bit 0.

Remark: The Idealized Oracle for This Section

For this section only, imagine an attacker who can submit any ciphertext $c = (u, v)$ of their choosing and simply *read off* the decoded bits of m' — as if they had a copy of the receiver's decryption function sitting on their own desk. This is a stronger oracle than a real attacker gets against Kyber's actual `Decaps` (which never outputs decoded plaintext bits at all — that's exactly what the FO transform in Lecture 3, Section 3.6, is for). We use it here purely to see the underlying mechanism cleanly, with real numbers; Section 5.3 shows how a real attacker gets an equivalent oracle out of the actual, FO-transformed scheme.

Example: Recovering $\vec{s}$ Coefficient by Coefficient, $q = 17$ (Verified by Hand)

Reuse Lecture 3's exact toy secret, $s = 1 - x = (1, 16, 0, 0)$ in $R_{17} = \mathbb{Z}_{17}[x]/(x^4 + 1)$ (recall $-1 \equiv 16 \pmod{17}$) — unknown to the attacker, who wants to recover it. **Choose** $u^* = 1$ (the constant ring element). Since 1 is the multiplicative identity, $s \cdot u^* = s \cdot 1 = s$ exactly, with no computation needed. So

$$m' = v^* - s \cdot u^* = \left(v_0^* - 1,\ v_1^* - 16,\ v_2^*,\ v_3^*\right) \bmod 17 = \left(v_0^* - 1,\ v_1^* + 1,\ v_2^*,\ v_3^*\right) \bmod 17.$$

Recovering $s_0 = 1$. Fix $v^* = (v_0^*, 0, 0, 0)$ and sweep $v_0^* = 1, 2, 3, \ldots$, watching only the first coefficient of m'. Working out Decode for $q = 17$ directly from the defbox rule shows it outputs 1 exactly for m'-values $\{5, \ldots, 12\}$ and 0 for $\{0, \ldots, 4\} \cup \{13, \ldots, 16\}$ (with 13 an exact tie, unused below). So:

v_0^*	1	2	3	4	5	**6**
$m'_0 = v_0^* - 1$	0	1	2	3	4	**5**
Decode	0	0	0	0	0	**1 (flip!)**

The flip happens at $v_0^* = 6$. Since decode flips to 1 exactly when m'_0 first reaches 5, we have $6 - s_0 \equiv 5 \pmod{17}$, so $s_0 = (6 - 5) \bmod 17 = 1$ — **exactly the true value.**

Recovering $s_1 = 16$. Same idea on the second coordinate: $m'_1 = v_1^* + 1$.

v_1^*	1	2	3	**4**
$m'_1 = v_1^* + 1$	2	3	4	**5**
Decode	0	0	0	**1 (flip!)**

Flip at $v_1^* = 4$, so $s_1 = (4 - 5) \bmod 17 = -1 \bmod 17 = 16$ — again exactly right. The same sweep on coordinates 2 and 3 recovers $s_2 = 0$ and $s_3 = 0$ (try it), reconstructing the entire secret $s = 1 - x$ from nothing but the decoded bits of chosen ciphertexts — never touching lattice reduction, never solving LWE.

Remark: Making the Sweep Fully General

The worked sweep above starts at $v_i^* = 1$ and happens to hit the flip within a handful of queries — convenient for a by-hand example, but not guaranteed for every possible value of s_i (if s_i itself already sits inside the "decode to 1" zone, the very first probe can already read 1, with no visible flip yet). The fully general, always-correct version: sweep v_i^* across the *entire* range $0, \ldots, q - 1$ (at most q queries, or $O(\log q)$ with binary search, since the boundary positions are fixed and known in advance) and specifically locate the point where Decode goes from 0 to 1 as v_i^* increases by one — as opposed to the far boundary, where it goes back from 1 to 0. Get that direction right and the same equation, $s_i = (v_i^* - 5) \bmod q$, holds unconditionally. Either way: at most $O(n \log q)$ oracle queries recover an entire secret polynomial of degree n exactly.

5.3 From "See the Bit" to "See Match or Mismatch": The Real Oracle in `Decaps`

Real Kyber never lets an attacker read m''s decoded bits directly — that is exactly the problem the Fujisaki–Okamoto transform (Lecture 3, Section 3.6) exists to solve. `Decaps` only ever outputs a symmetric key K: either $H(m')$, if re-encrypting the recovered m' reproduces the submitted ciphertext exactly, or a pseudorandom-looking $H(sk, c)$ if it doesn't (*implicit rejection*). So where does an attacker's oracle actually come from?

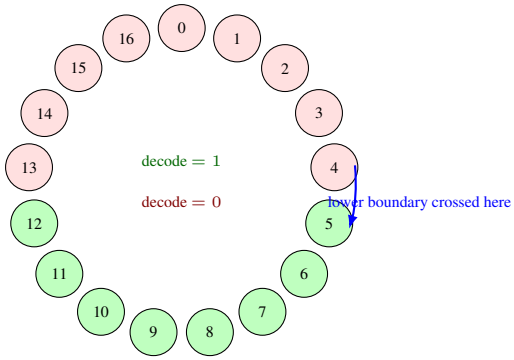


Figure 5.2: The 17 possible values of a coefficient of m', arranged in a circle exactly as modular arithmetic wraps around (Lecture 1, Section 1.6). Decode outputs 1 for the green arc $\{5, \ldots, 12\}$ and 0 for the red arc — a single contiguous region on each side. Sweeping v_i^* walks a probe around this circle; the moment it crosses from red into green pins down s_i exactly.

Remark: The Bridge

An attacker who cannot see K itself can often still learn *which branch* `Decaps` took — via **timing** (the match-check and the two key-derivation paths are natural places for a non-constant-time implementation to take measurably different amounts of time), **power** (the re-encryption step touches different data on each branch), or, in some deployment models, an explicit **protocol-level signal** that a handshake failed. That single bit — match vs. mismatch — is a coarser version of the same information Section 5.2 used directly: for a ciphertext crafted to sit exactly at a decode boundary, whichever side of that boundary the true decoding lands on determines *both* the decoded bit *and*, almost always, whether re-encryption reproduces c. Getting a coarser, binary signal instead of the exact decoded bits costs the attacker more queries, not a different attack — the boundary-sweep procedure and the underlying algebra are unchanged.

Looking Ahead: A Second, Independent Leak Hiding in the Same Step

Lecture 3's preview box already flagged this precisely: the intermediate value m' is computed, in full, *before* the match-check ever runs — for every ciphertext, including ones that will ultimately be rejected. If that computation itself leaks (say, via power analysis of the $\vec{s}^T\vec{u}$ multiply inside CPAPKE.Dec, independent of any timing difference in the comparison afterward), an attacker gets information about $\vec{s}$ regardless of whether implicit rejection kicks in. Implicit rejection hides *which branch was taken*; it does nothing to hide what happened while computing m' in the first place. This is exactly why constant-time coding of the comparison alone is not a complete fix (Section 5.5 returns to this).

5.4 Decryption Failures as a Naturally Occurring Version of the Same Leak

Section 5.2's attacker deliberately crafted ciphertexts to sit at a decode boundary. A second, related attack surface doesn't need to construct anything so precise — it exploits noise that was already there.

Recall from Lecture 3, Section 3.4: decryption recovers m correctly as long as the total noise term ε stays under $q/4$ in absolute value; ML-KEM's real parameters keep the *probability* of a decryption failure astronomically small (below 2^{-100}), but not literally zero. That tiny residual failure probability is not secret-independent.

Remark: Why Failures Correlate With the Secret

ε is built from $\vec{e}, \vec{r}, e_2, \vec{s}, \vec{e}_1$ (Lecture 3's derivation). An attacker who can influence any of the public, attacker-chosen quantities in that mix ($\vec{r}$, e_2, and, via the ciphertext, effectively the compression rounding too) — while $\vec{s}$ stays fixed across many queries — controls the *distribution* of ε relative to the fixed, unknown $\vec{s}$. Sending many ciphertexts engineered to push ε as close to the $q/4$ boundary as compression and encoding allow, and recording which ones happen to fail, produces a dataset whose failure pattern is a statistical function of $\vec{s}$ — the same underlying idea as Section 5.2's boundary sweep, but read off through *failure rate over many trials* instead of a single crisp decode bit. This is precisely what Lecture 3 called Kyber's **central-reduction leak**: published decryption-failure and plaintext-checking-oracle attacks against LWE-based encryption schemes follow exactly this template, differing mainly in how cleverly the chosen ciphertexts are constructed to maximize how much each query reveals.

Remark: The Two Kyber Attack Surfaces, Side by Side

- **Attack surface 1 (Sections 5.2–5.3):** the FO re-encryption check's match/mismatch outcome, and the m'-before-the-check leak underneath it — exploited via *deliberately crafted* boundary ciphertexts.
- **Attack surface 2 (this section):** the natural decryption-failure margin — exploited *statistically*, over many ciphertexts, without needing a single ciphertext to sit exactly on a boundary.

Both are instances of the same recipe from Section 5.1.2: leak a little (one bit, or one failure/-success outcome) per query, combine over many queries. Together they make up Kyber's share of the "six risky steps" promised back in Lecture 4 — Lesson 6 supplies the remaining four, for Dilithium and Falcon.

5.5 Countermeasures, Kyber-Specific

Remark: Constant-Time Comparison and Decoding

The most direct fix for Section 5.3's leak is making the re-encryption check — and the Decode step feeding it — take exactly the same time and touch exactly the same memory regardless of whether the values match, typically via bitwise operations (AND/OR/XOR across the full byte string, then a final constant-time reduction to one bit) instead of a short-circuiting equality test like the toy PIN-checker in Section 5.1.2. Reference implementations of ML-KEM do exactly this.

Remark: Why That Alone Isn't Enough

Constant-time comparison closes the *match-vs-mismatch* leak, but, as the preview box in Section 5.3 stressed, it does nothing about power or EM leakage from computing m' itself, *before* any comparison happens. That requires a different, harder tool: **masking** — splitting every secret-dependent intermediate value into two or more random shares, computed on separately, so that no single physical measurement ever sees anything statistically correlated with $\vec{s}$ alone. Masking arithmetic mod q (and the rounding/compression steps built on top of it) is substantially harder to get right than masking the bitwise operations of a cipher like AES. Lesson 6 picks this up properly, once Dilithium's and Falcon's attack surfaces are on the table too and a single, shared countermeasures discussion can cover all three schemes at once.

5.6 Bridge to Your Thesis

Looking Ahead: A Starting Checklist for Kyber

For any implementation you might analyze, three questions from this lecture translate directly into a research starting point: (1) Is the FO re-encryption check implemented in constant time, and does that constant-time property survive compiler optimization? (2) Does the CPAPKE decryption arithmetic itself ($\vec{s}^T\vec{u}$) show measurable data-dependent timing or power variation, independent of the comparison afterward? (3) Under repeated, adversarially chosen ciphertexts, does the empirical decryption-failure rate show *any* detectable dependence on the ciphertext's structure? A clean, well-documented answer to any one of these — even a purely negative one, confirming a specific implementation resists a specific attack — is exactly the kind of floor-level contribution your thesis's "Analysis" framing was built to accommodate; a genuinely novel variant of any of them is the ceiling.

5.7 Summary Table

Concept	**What it is**	**Where it lives**
Cryptanalysis	Attacks the hard problem itself	Public info only, unlimited compute
Implementation attack	Attacks how the algorithm is computed	Timing / power / EM / faults
Passive attack	Observes without disturbing	Side-channel analysis (SCA)
Active attack	Deliberately corrupts the computation	Fault injection
Leak, then combine	The general recipe (Fig. 5.1)	Every attack in this lecture
Boundary-sweep oracle	Craft c at a decode boundary	Kyber attack surface 1
Decryption-failure leak	Exploit the natural noise margin	Kyber attack surface 2
Constant-time comparison	Fixes the match/mismatch leak only	Necessary, not sufficient
Masking	Splits secrets into random shares	Fixes leaks *before* the check

5.8 Exercises (Assign as Homework Before Lesson 6)

Exercise: Homework Set

1. In your own words (2–3 sentences), explain the difference between a cryptanalytic attack and an implementation attack, and give one example of each that does *not* appear in this lecture.
2. Redo the toy PIN-checker timing analysis (Section 5.1.2) for a 6-digit PIN instead of 4. How many queries does the timing attack need in the worst case, versus a blind brute-force guesser? Express both as a function of the number of digits.
3. Using the same toy Kyber instance ($q = 17$, secret $s = 1-x$) as Section 5.2's worked example: complete the sweep for coordinates 2 and 3 by hand, and confirm you recover $s_2 = 0$ and $s_3 = 0$.
4. Pick any $s_i \in \{6, \ldots, 13\}$ (a value for which the naive sweep starting at $v_i^* = 1$ shows decode$= 1$ immediately, with no visible flip). Using the fully general procedure from the "Making the Sweep Fully General" remark, find the correct boundary crossing and recover s_i — confirming the formula still works once you sweep the full range and identify the correct (entering) direction.
5. In 3–4 sentences: why does making the re-encryption check constant-time *not* fully close Kyber's attack surface, even though it closes the specific leak described in Section 5.3?
6. **Looking ahead:** Section 5.4 describes Kyber's decryption-failure attack surface at the conceptual level only. Find one published paper (search terms like "decryption failure attack LWE encryption" or "plaintext-checking oracle Kyber" are a good start) and identify, in 3–4 sentences, which specific part of the ciphertext that paper's authors chose to manipulate, and why.

Lesson 6: Physical Attacks II — ML-DSA (Dilithium) and FN-DSA (Falcon)

Four More Attack Surfaces, Countermeasures for All Three Schemes, and Your Thesis's Starting Map

Purpose of this lecture. Lesson 5 built the general framework — cryptanalysis vs. implementation attacks, passive vs. active, and the "leak, then combine" recipe — and used it on ML-KEM's one named riskiest step, split into two concrete attacks. This lecture finishes the job: two attacks on ML-DSA (Dilithium)'s $\vec{y}$-generation/bound-check step, and two on FN-DSA (Falcon)'s floating-point sampler — the remaining four of the "six risky steps" Lecture 4 promised. We then build one shared countermeasures toolkit covering all three schemes at once, and close with a single map of all six attack surfaces to help you scope your own thesis contribution.

6.1 ML-DSA, Attack Surface 3: Fault Attacks on Determinism

Recall the factbox from Lecture 4: Dilithium derives its masking vector $\vec{y}$ *deterministically* from (sk, msg) via a PRF, precisely to avoid the classical failure mode where an accidentally-reused $\vec{y}$ across two signatures lets an attacker subtract the two signing equations and isolate $\vec{s}_1$ directly. Determinism kills the *accidental* version of that bug. It does not kill the *deliberate* one.

Remark: How a Fault Forces Reuse

$\vec{y}$ is computed from (sk, msg) by a PRF, then never touched again during that signing call. A fault — a voltage or clock glitch timed to hit the PRF call, or a skipped instruction that leaves a stale register from a previous signing operation in place — can make the device use the *same* $\vec{y}$ for two different messages, without the signer's software ever noticing anything went wrong. Both signatures still verify correctly; nothing about the output looks broken.

Example: Recovering s_1 From Two Faulted Signatures, $q = 17$ (Verified by Hand)

Reuse Lecture 4's exact toy instance: $R_{17} = \mathbb{Z}_{17}[x]/(x^4+1)$, $A = 3 + 5x + 2x^2 + x^3$, secret $s_1 = 1$ (unknown to the attacker). The legitimate first signature, exactly as Lecture 4 computed it: mask $y = x$, challenge $c_1 = 1$, response

$$z_1 = y + c_1 s_1 = x + 1 \cdot 1 = 1 + x.$$

Now suppose a fault forces the *same* $y = x$ to be reused when signing a second, different message, whose Fiat–Shamir challenge comes out as $c_2 = 2$ (different message $\Rightarrow$ different hash input $\Rightarrow$ a different challenge, even with $w = Ay$ unchanged). The second response:

$$z_2 = y + c_2 s_1 = x + 2 \cdot 1 = 2 + x.$$

The attacker sees only the two public signatures, $(z_1, c_1) = (1 + x,\ 1)$ and $(z_2, c_2) = (2 + x,\ 2)$ — never y or s_1 directly. Subtract:

$$z_1 - z_2 = (1+x) - (2+x) = -1 \equiv 16 \pmod{17}, \qquad c_1 - c_2 = 1 - 2 = -1 \equiv 16 \pmod{17}.$$

Since $z_1 - z_2 = (y + c_1 s_1) - (y + c_2 s_1) = (c_1 - c_2)s_1$, the masking term y — the one thing the attacker never learns directly — cancels out completely, exactly like Lecture 3's noise-cancellation trick. Solving for s_1 (using $(-1)^{-1} \equiv -1 \pmod{17}$, since $(-1) \times (-1) = 1$):

$$s_1 = (z_1 - z_2) \cdot (c_1 - c_2)^{-1} = (-1) \cdot (-1) = 1 \pmod{17} \quad \textbf{— exactly the true secret.}$$

This is precisely the calculation Lecture 4's practice exercise asked you to set up symbolically; here it is completed with real numbers, and it recovers the secret in full from just two faulted signatures.

Remark: Why This Is a Complete Break, Not a Partial Leak

Unlike Section 6.1's sibling attacks in Lesson 5, which needed many queries combined statistically or via a boundary sweep, this one needs exactly **two** faulted signatures and pure linear algebra — no averaging, no lattice reduction, no repeated probing. That makes it, coefficient for coefficient, one of the most devastating attacks in this entire two-lesson sequence *when the fault succeeds*: the entire secret $\vec{s}_1$ falls out in one subtraction. The real engineering difficulty is entirely on the attacker's fault-injection side (precisely timing a glitch to hit the right instruction), not on the algebra — which is already fully solved, by you, above.

6.2 ML-DSA, Attack Surface 4: Bypassing the Rejection-Sampling Check

Recall from Lecture 4, Section 4.5: $\vec{z} = \vec{y} + c\vec{s}_1$ is only *published* if every coefficient stays within an allowed bound; anything outside it is discarded and resampled with fresh $\vec{y}$. The entire point is that, *conditioned on being published*, $\vec{z}$'s distribution carries no statistical trace of $\vec{s}_1$. A fault that skips or corrupts that bound check breaks the conditioning — without breaking the arithmetic that produced $\vec{z}$ in the first place.

Remark: Why Skipping the Check Reveals a Mean Shift

$\vec{y}$ is sampled from a range built wide and symmetric around $\vec{0}$ specifically so that $\mathbb{E}[\vec{y}] = \vec{0}$. Every legitimate published $z_i = y_i + cs_{1,i}$ therefore has $\mathbb{E}[z_i] = \mathbb{E}[y_i] + cs_{1,i} = cs_{1,i}$ — a mean shifted by exactly the secret-dependent quantity $cs_{1,i}$. Rejection sampling normally hides this: conditioning on landing inside a window safely *interior* to y_i's full range barely perturbs a symmetric distribution, as long as the shift $cs_{1,i}$ is small relative to the safety margin between the accepted window and y_i's natural spread — which is exactly why real parameters choose $\vec{y}$'s range to be comfortably wider than $\vec{z}$'s allowed bound. **Bypass the check, and that safety margin no longer matters**: every z_i, rejected-in-the-real-scheme or not, gets published, and the raw, unconditioned mean shift $cs_{1,i}$ is sitting right there in the data, waiting to be averaged out.

Example: A Small Illustration of the Averaging Step

Suppose an attacker, exploiting a bypassed check, collects five faulted signatures that all happen to share the same challenge $c = 1$ (or normalizes each observed z_i by dividing out its own c first), and reads off one coefficient's value each time: $3,\ 5,\ 2,\ 4,\ 3$. The sample average is $(3+5+2+4+3)/5 = 17/5 = 3.4$, already landing close to a true secret coefficient of, say, $s_{1,i} = 3$ or 4. Five samples is far too few for real confidence — exactly Lesson 5's "leak a little, combine a lot" recipe (Lesson 5's Figure 5.1) again, this time with the combining step being a literal running average instead of a boundary sweep or an algebraic subtraction. Collecting hundreds or thousands of faulted signatures, exactly as real published attacks on rejection-sampling implementations do, drives the noise down and the estimate of $s_{1,i}$ into certainty.

6.3 FN-DSA, Attack Surface 5: Sampler Non-Exactness

Recall from Lecture 4, Section 4.7: Falcon's GPV framework signs by sampling a lattice point from a discrete Gaussian centered near a hashed target, using the trapdoor (f, g) to guide the search. The security argument depends on that sampler being *exact* — indistinguishable from a true discrete Gaussian, with no data-dependent quirks at all.

Remark: The Biased-Coin Analogy

Imagine a coin advertised as fair (50% heads) that is actually, due to a subtle manufacturing flaw tied to its specific weight distribution, 53% heads. Flip it once, and you learn nothing about the flaw. Flip it a thousand times, and the sample frequency — roughly 530 heads instead of 500 — reveals the bias clearly, via nothing more than counting. A discrete Gaussian sampler that is subtly *approximate* rather than exact behaves the same way: each individual signature reveals nothing, but the statistical shape of many signatures — which values get sampled slightly more or less often than a true Gaussian would produce — can correlate with the specific trapdoor (f, g) that built the sampler, exactly as the coin's bias correlated with its physical flaw.

Example: A Toy Illustration of Detecting the Bias

Suppose a correct, exact sampler over $\{-1, 0, 1\}$ should produce each value with probability $\{0.25,\ 0.5,\ 0.25\}$ — a coarse discrete-Gaussian stand-in. A specific flawed implementation, tied to one particular trapdoor, instead produces $\{0.20,\ 0.50,\ 0.30\}$ — a small, hard-to-notice skew toward $+1$. An attacker who collects many signatures and tallies the frequency of each sampled value will, over enough signatures, see the count for $+1$ pull measurably above a quarter of the total — the same counting argument as the biased coin, now applied coefficient by coefficient across many signatures, and correlated against different trapdoor hypotheses to pin down which one produced this specific skew.

6.4 FN-DSA, Attack Surface 6: Floating-Point Timing and Power Leakage

Falcon's fast Fourier sampling is implemented with floating-point arithmetic for speed (Lecture 4's preview box). Floating-point hardware and software are, historically, far harder to make constant-time than the plain integer bound-checks Dilithium's rejection sampling relies on — and the reason is concrete enough to see directly.

Example: A Toy Floating-Point Leak: Rounding to the Nearest Integer

A common way to round a floating-point value x to the nearest integer checks its fractional part f against 0.5:

$$\texttt{if } f \geq 0.5\texttt{: round up} \quad \texttt{else: round down.}$$

On many real floating-point units, the two branches take measurably different time or power, exactly like Lesson 5's PIN-checker — except here, the branch outcome doesn't just leak "was this guess partially right," it leaks **which side of** 0.5 **the fractional part of the actual sampled Gaussian value fell on**. That's not a side detail: it's a direct, per-sample bit of the number the trapdoor sampler was supposed to keep hidden. A single signature already involves many such roundings; a full power trace of one signing operation can leak enough of the sampler's intermediate values to reconstruct large parts of (f, g) directly — which is exactly why **single-trace** key-recovery attacks (needing only one signature's physical trace, not thousands) are the headline result in Falcon's published attack literature, in sharp contrast to the many-query attacks earlier in this lecture.

Remark: Why This Is Harder to Fix Than It Sounds

"Just make the rounding constant-time" undersells the problem. Floating-point units built into real hardware handle rounding, subnormal numbers, and special values (like exactly 0.5) with timing and power characteristics baked into the silicon — not something a software patch can fully erase without abandoning hardware floating-point support altogether. Section 6.5 discusses the real trade-off this forces.

6.5 Countermeasures for All Three Schemes

Lesson 5 introduced constant-time coding and previewed masking for Kyber alone. With all six attack surfaces now on the table, one shared toolkit covers all three schemes — each technique defends against a different piece of the "leak, then combine" recipe.

Remark: Constant-Time Coding, Scheme by Scheme

The baseline defense everywhere: make timing and control flow independent of secret data. For **Kyber**, this means the FO re-encryption comparison (Lesson 5). For **Dilithium**, it means the rejection-sampling bound check itself must not branch in a way that leaks which coefficients failed — and, separately, the PRF call generating $\vec{y}$ must be hardened against the instruction-skip and glitch attacks of Section 6.1 (redundant computation, glitch detectors, or verifying $\vec{y}$ was freshly derived before using it). For **Falcon**, constant-time coding runs into the hardest version of this problem: the real fix usually means replacing floating-point sampling with a carefully designed **fixed-point or integer-only** sampler, trading some speed for a much smaller, more auditable timing surface — an active area of implementation research in its own right.

Remark: Masking

Masking splits every secret-dependent value into two or more random shares, computed on separately, so no single physical measurement ever correlates with the secret alone (Lesson 5). Masking Kyber's mod-q arithmetic and compression is already harder than masking a bitwise cipher like AES; masking Dilithium's rejection-sampling comparison adds the extra requirement that the *bound check itself* must be evaluated on shares without ever reconstructing the unmasked value; and masking a *true discrete Gaussian sampler*, as Falcon needs, is harder still and remains an active research problem — there is no simple, universally agreed-upon masked Gaussian sampler the way there is a masked AES S-box.

Remark: Blinding and Shuffling

Two lighter-weight techniques, often used alongside masking rather than instead of it. **Blinding** multiplies or adds a single random value to an intermediate computation and removes it later, rather than fully secret-sharing every step — cheaper than masking, but generally offering a weaker security guarantee. **Shuffling** randomizes the *order* in which independent operations happen — which polynomial coefficient gets processed first, say — so that a power trace can no longer be aligned to a fixed position and averaged cleanly across many traces, directly undercutting the "combine" half of the recipe from Lesson 5's Figure 5.1 without changing what gets computed at all.

Fact: TVLA: How the Field Actually Measures "Is This Leaking?"

Test Vector Leakage Assessment (TVLA) is the standard methodology for evaluating whether a given implementation leaks at all, without needing to mount a full key-recovery attack first. The core idea: collect power or EM traces for two carefully chosen sets of inputs (e.g., a fixed secret-dependent value vs. a fresh random one each time) and run a statistical test (typically Welch's t-test) comparing the two sets, trace point by trace point. A large, consistent statistical difference is evidence of a leak, even if no one has yet turned that leak into a working attack. This is the standard vocabulary you'll want for writing up your own results — reporting "no statistically significant leakage detected via TVLA" is a precise, citable claim, not just "we didn't find anything."

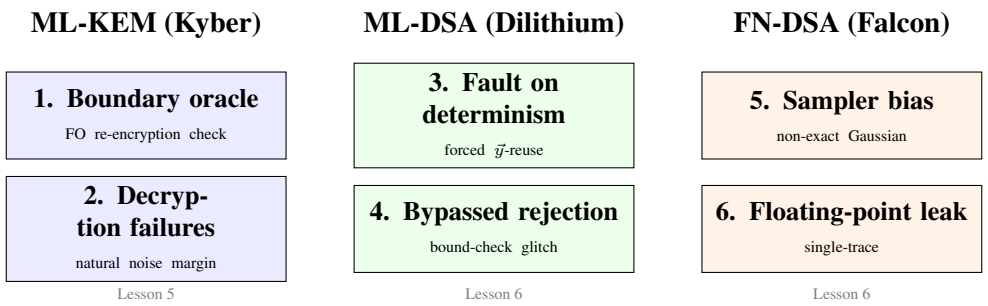


Figure 6.1: All six attack surfaces from Lessons 5–6, side by side. Every box is one instance of the same recipe as Lesson 5's Figure 5.1: find a leak, then combine many observations — algebraically (surface 3), statistically (surfaces 1, 2, 4, 5), or via a single rich trace (surface 6).

6.6 Bridge to Your Thesis: The Full Six-Surface Picture

Looking Ahead: A Starting Checklist, Now Covering All Three Schemes

Extend Lesson 5's Kyber-only checklist with the four questions this lecture adds: (4) Is Dilithium's $\vec{y}$-derivation hardened against instruction-skip and glitch faults, and does the bound check itself run in constant time? (5) Does a given Falcon implementation's sampler pass a statistical exactness test against a true discrete Gaussian, at a sample size large enough to detect a small bias? (6) Does a power or EM trace of a *single* Falcon signature show detectable, sample-correlated structure around its floating-point rounding operations? As before, a careful, well-documented answer to any one of these six questions — confirming a leak *or* confirming its absence via TVLA — is a complete, thesis-worthy floor-level contribution; combining a partial leak from one surface with the lattice-reduction techniques of Lecture 1 to finish a full key recovery is the kind of result that would make a genuinely strong thesis chapter on its own.

6.7 Summary Table

Attack surface	Combine step	Queries needed
1. Kyber boundary oracle	Algebraic sweep	$O(n \log q)$
2. Kyber decryption failures	Statistical	Many
3. Dilithium fault-on-determinism	Algebraic (subtraction)	**2**
4. Dilithium bypassed rejection	Statistical (averaging)	Many
5. Falcon sampler bias	Statistical (frequency count)	Many
6. Falcon floating-point leak	Single rich trace	**1**
Constant-time coding	Fixes timing/branch leaks	—
Masking	Fixes value-correlation leaks	—
Blinding, shuffling	Cheaper, weaker alternatives	—
TVLA	Detects leaks before an attack exists	—

6.8 Exercises

Exercise: Homework Set

1. Using the same toy Dilithium instance as Section 6.1: suppose instead the two observed challenges were $c_1 = 1$ and $c_2 = 3$ (still with the same faulted $y = x$). Compute z_1, z_2, and confirm the subtraction-and-divide procedure still recovers $s_1 = 1$.
2. In your own words (3–4 sentences): why does Attack Surface 3 (Section 6.1) need only two signatures while Attack Surface 4 (Section 6.2) needs many? Connect your answer to whether each attack's "combine" step is algebraic or statistical.
3. Section 6.3's biased-coin toy used probabilities $\{0.20, 0.50, 0.30\}$ instead of the correct $\{0.25, 0.50, 0.25\}$. If an attacker collects 400 signatures and the sampler is indeed the flawed one, roughly how many times would you expect to see the value $+1$ sampled? How does that compare to what a correct, unbiased sampler would produce over the same 400 signatures?
4. Compare Section 6.4's single-trace Falcon attack to Lesson 5's Kyber attacks (which need many chosen ciphertexts). In 3–4 sentences, explain why a defender might reasonably treat a single-trace attack as a *more* urgent risk to fix than a many-query attack, even though the many-query attack is, in some sense, more thoroughly demonstrated in the literature.
5. **Looking ahead:** using Figure 6.1 and the checklist in Section 6.6, write a half-page sketch (this can be genuinely for your own thesis planning, not just a classroom exercise) of which one or two of the six attack surfaces you would prioritize investigating first, and why — consider available equipment, how well-studied each surface already is in the published literature, and which countermeasure from Section 6.5 you would evaluate against.